

ITIL and Devops

-
-

By

Preetha

ITIL

- ITIL, formerly an acronym for **Information Technology Infrastructure Library**, It is a set of best practices for delivering IT services—it standardizes the selection, planning, delivery, and support of IT services to maximize efficiency and maintain predictable levels of service.
- ITIL describes **processes, procedures, tasks, and checklists** which are not organization-specific nor technology-specific, but can be applied by an organization towards strategy, delivering value, and maintaining a minimum level of competency.
- It allows the organization to establish a baseline from which it can plan, implement, and measure.
- **It is a library which is a collection of guidelines or publications to deliver the services to customer.**

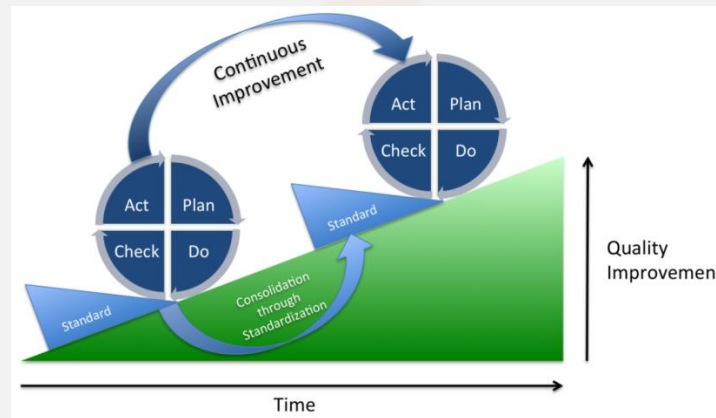
- **Services:** A means of delivering value to customers without requiring the customer to own specific costs and risks.
- Some key ITIL concepts and principles are:
 - Delivering maximum value to customers
 - Optimizing resources and capabilities
 - Offering services that are useful and reliable
 - Planning processes with specific goals in mind
 - Defining roles clearly for each task.

- Reducing IT costs
- Increased Quality and Decrease Risk
- Improved Decision Making
- Increased Productivity
- ITIL provides clear and effective communication.
- ITIL embraces a practical approach to service management do what works.

- ITIL was developed by the [UK Government's Central Computer and Telecommunications Agency \(CCTA\)](#) in the 1980s, due to the growing influence of IT in the government and other industries to form a set of recommendations.
- It recognized that, without standard practices, government agencies and private sector contracts had started independently creating their own IT management practices.
- ITIL was built around [a process model-based](#) view of controlling and managing operations often credited to W. Edwards Deming and his plan-do-check-act (PDCA) cycle.

PDCA Cycle:

- It is also known as the Deming circle, which is a continuous feedback system for continuous improvement of any process/operation.



Phase	Description	Purpose
Plan	Identify a goal or process to improve. Analyze the current situation and develop a plan to implement change.	Set clear objectives and determine how to achieve them.
Do	Implement the plan on a small scale.	Execute the change safely and in a controlled way.
Check	Monitor and measure the results against expected outcomes. Analyze any deviations.	Verify if the plan worked as intended.
Act	If successful, implement on a larger scale. If not, refine the plan and repeat.	Institutionalize the change or go back to planning.

- The IT Infrastructure Library originated as [a collection of books](#), each covering a specific practice within IT service management. After the initial publication [in 1989–96](#), the number of books quickly grew within ITIL Version 1 to more than 30 volumes. CCTA released ITIL v1 in 1989.
- In 1989, ITIL's goal was to **standardize IT service management (ITSM)**.
- This initial iteration gave organizations an [overview of how to streamline services and helped admins start thinking about best practices](#).
- Structure: Over 30 volumes of processes and practices

- **ITIL Version 2:**
- In 2000/2001, to make ITIL more accessible, ITIL Version 2 consolidated the publications into nine logical "sets" that grouped related process-guidelines to match different aspects of IT management, applications and services.
- In April 2001, the CCTA was merged into the OGC (Office of Government Commerce), an office of the UK Treasury. [In 2006, the ITIL Version 2 glossary was published.](#)
- In 2009, the OGC officially announced that ITIL Version 2 certification would be withdrawn and launched a major consultation as per how to proceed.
- ITIL v2 offered admins [a more applicable and uniform structure for service support and delivery](#), and it included actual processes for organizations to follow.

ITIL Version 3:

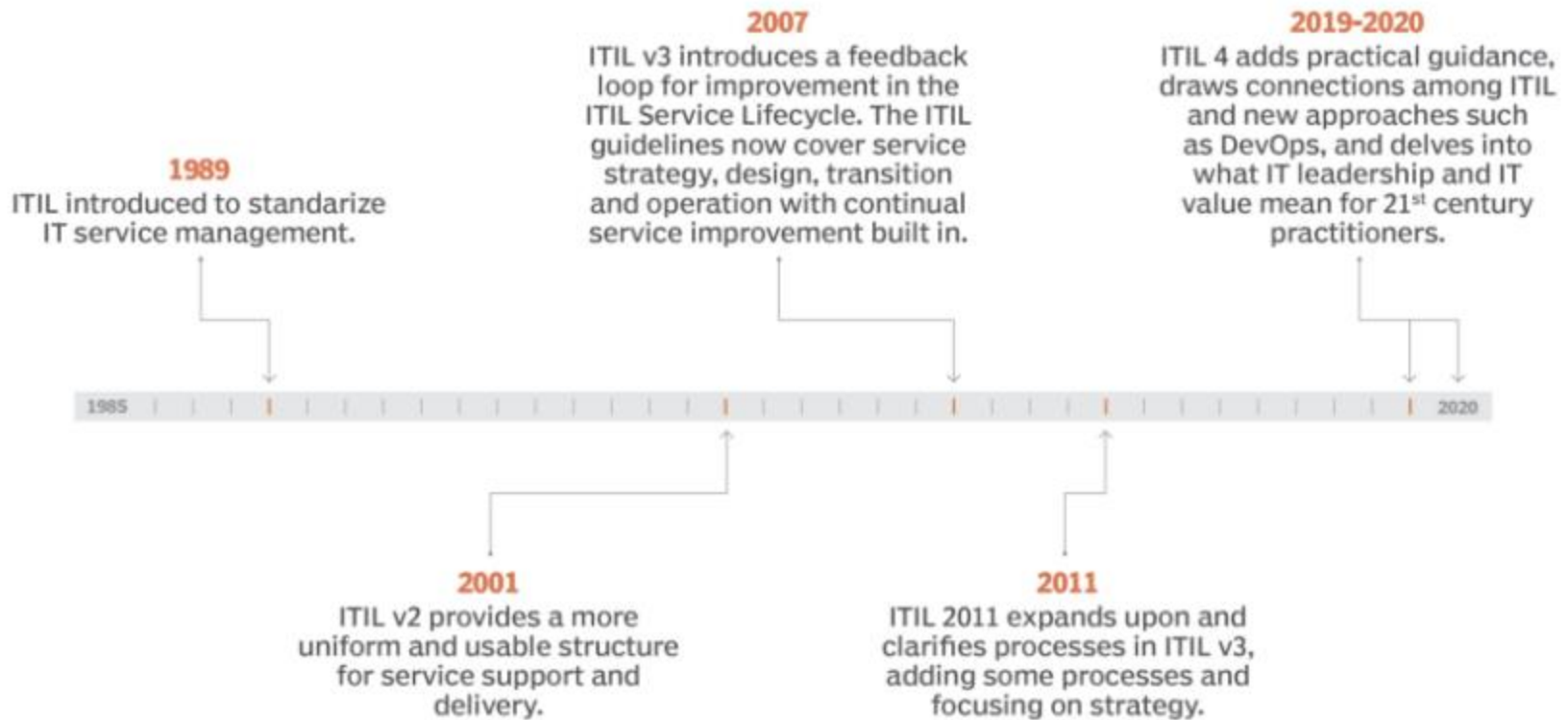
- In [May 2007](#), this organization issued ITIL Version 3 (also known as the ITIL Refresh Project) consisting of [26](#) processes and functions, now grouped into only 5 volumes, arranged around **the concept of Service lifecycle** structure. ITIL Version 3 is now known as [ITIL 2007 Edition](#).
- Key Concepts: Service as a lifecycle, not isolated processes
- Emphasis on value creation, customer focus, and feedback loops

ITIL 4 (2019 – Present)

- **Modernized Framework** for the **digital era**
- **Major Enhancements:**
 - Integrates **Agile, DevOps, Lean, and Cloud** practices
 - Introduces the **Service Value System (SVS)** and **Service Value Chain**
 - Replaces "processes" with broader **34 practices**

Version	Release Year	Focus	Key Feature
ITIL v1	Late 1980s	Government IT Standardization	30+ volumes
ITIL v2	2001	IT Process Standardization	Service Support & Delivery
ITIL v3	2007	Service Lifecycle Management	5 Lifecycle Phases
ITIL 2011	2011	Update of v3	Clarifications and improvements
ITIL 4	2019	Modern, Agile, Value-Centric	SVS, Value Chain, Practices

ITIL evolution



- A set of specialized capabilities for delivering value to customers in the form of services.
- The capabilities take the form of functions and processes for managing services over a lifecycle, with specializations in strategy, design, transition, operation and continual improvement.
- Capabilities: The specialized skills or abilities an organization applies to resources in order to create value.

- **Value**-Service strategy defines a unique approach for delivering better value. According to customers' needs, service should consist of two elements –
 - Utility
 - Warranty



1. Utility (Fitness for Purpose)- The functionality offered by a service to meet a particular need.

- Ensures the **service delivers the desired outcome**.
- Removes constraints or enhances customer performance.

- **Examples:**

- Email service allowing users to send/receive messages.
- A server that processes financial transactions.

2. Warranty (Fitness for Use)-The assurance that the service will perform as expected, under agreed conditions.

- Ensures the **service is delivered reliably** and consistently.
- Involves **availability, capacity, continuity, and security**.

- **Examples:**

- 99.9% system uptime
- Data backed up every hour
- Support available 24/7

- **Processes:** These are defined as structured sets of activities designed to achieve a specific objective. The four basic characteristics of processes are:
 1. They transform inputs into output.
 2. They deliver results to a specific customer or stakeholder.
 3. They are measurable.
 4. They are triggered by specific events.
- **Functions:** Self-contained subsets of an organization intended to accomplish specific tasks. They usually take the form of a team or group of people and the tools they use.
- **Resources:** Defined as the raw materials which contribute to a service, such as money, equipment, time, staff, etc.

- **Objective:** IT service providers implement *ITIL processes* to ensure their services are delivered in a **customer-focused, quality-driven and economical way**.
- It deals with the development or improvement of process.
- **Role :** A role is a set of responsibilities defined in a process and aligned to person or team.

Role of process:

1. Process owner role-Responsible for goal of result of the process.
2. Process manager role-Responsible for design and management of process.
3. Process operative role –Responsible for defined activities within the process.

- A **service-level agreement (SLA)** is a commitment between a service provider and a client. Particular aspects of the service – quality, availability, responsibilities – are agreed between the service provider and the service user. [\[1\]](#)
- The most common component of an SLA is that the services should be provided to the customer as agreed upon in the contract.
- Service-Level Agreement (SLA) **clearly defines what the customer should receive and what the provider must deliver** in terms of service performance.

- **It is a tool that helps organizations implement ITIL effectively.** It improves service delivery, efficiency, and accountability by supporting ITIL processes.
- The HP Service Manger is one of the applications acquired by HP when it purchased **peregrine Systems** in 2005.
- The application was originally known as PNMS(Perigrine Network Management System).
- It consists of following functionalities such as Request Management , Call management and Change Management.

Functionality

Request Management

Call Management

Change Management

Purpose

Handles and tracks service requests from users (e.g., software installation).

Logs and manages user calls related to incidents or issues.

Manages the lifecycle of changes in IT systems with proper approval and tracking.

- OTRS stands for **Open Source Ticket Request System**.
- It is a service management. **Suite that comprises ticketing, workflow automation and notification which is widely used by IT service management.**
- OTRS offers many options to automate tasks based on events, time, external systems and defined processes.
- OTRS also includes the possibility to add individual information types to tickets and help agents to lower their error rate when working with tickets by allowing only defined activities for tickets in specific states.

- Service lifecycle helps to amplify the service management approach and to achieve better understanding of its structure.
- The entire framework of ITIL is divided into five broad components/categories:
 - ITIL Service Strategy
 - ITIL Service Design
 - ITIL Service Transition
 - ITIL Service Operation
 - ITIL Continual Service Improvement



- **Service Strategy** provides guidance on how to design, develop and implement IT services as strategic assets and organizational capabilities.
- TIL Service Strategy gives an explanation regarding how IT Services will be utilized to enable IT to achieve the required objectives.
- The Service Strategy process determines which services the IT organization is to offer and what capabilities need to be developed. Its ultimate goal is to make the IT organization think and act in a strategic manner.

Example-Constructing a house. The idea or thought of constructing a house come under service strategy. In IT industry an organisation ABC plans to achieve 5 billion dollars by 2020.

- The four building block of ITIL strategy stage:
- 1. **Perspective**-It is your vision and direction for the services you provide.
Example: “We want to offer fast, secure, and reliable cloud hosting services to startups.”
- 2. **Position**- Position is how you will differentiate from your competitors ie,what is your unique value.
Example: “We are the only cloud host offering free support to startups in their first year.”
- 3. **Plan**-Describes how you will achieve the established goal and objectives.it focuses on budget and improvement of plans.
Example: “We will invest ₹10 lakhs to improve uptime to 99.99% by year-end.”

- 4. **Pattern**-The fundamental and ongoing actions your organization will take to run smoothly over time.this includes processes, policies, schedules, budgets, and management systems.
- **Example:** “We follow a weekly incident review, monthly performance report, and quarterly customer survey.”

Block	What It Focuses On	Key Question Answered
Perspective	Vision and purpose	“Where are we going?”
Position	Market uniqueness	“What makes us different?”
Plan	Strategy to achieve goals	“How will we get there?”
Pattern	Daily operations and routines	“What do we consistently do?”

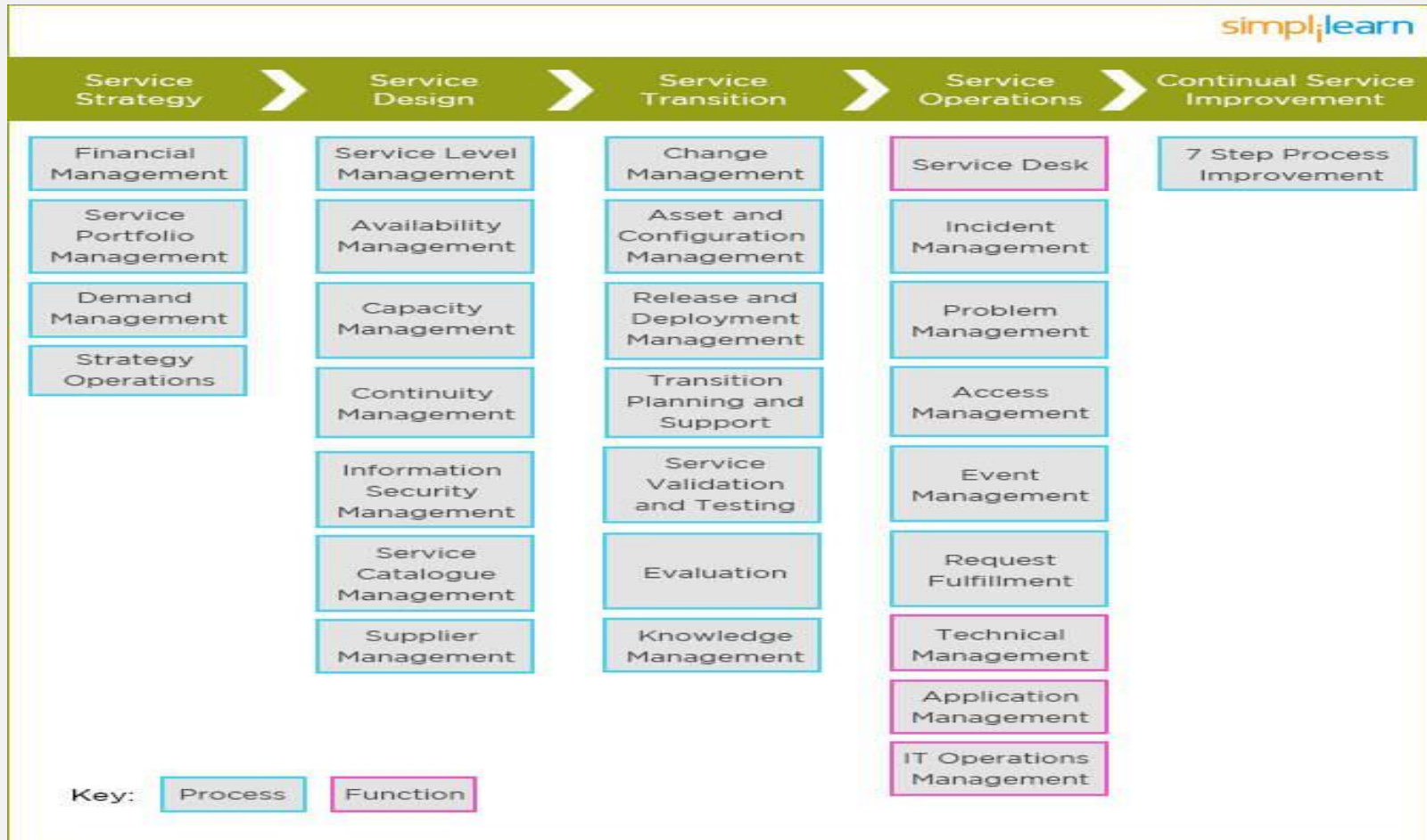
- The Service Design lifecycle phase is about **the design of services and all supporting elements** for introduction into the live environment.
- **Service Design** provides a **blueprint for the services**. It not only includes designing of a new service but also devises changes and improvements to existing ones.
- The '**Four Ps of Service Design**' represent areas which should be taken into consideration when designing a service. They are:
 - **People:** Human resources and organizational structures required to support the service.
 - **Processes:** Service Management Processes required to support the service.
 - **Products:** Technology and other infrastructure required to support the service.
 - **Partners:** Third parties which offer additional support required to support the service

- **Service Transition** provides guidance for the improvement of capabilities and transitioning services into operations.
- In the Service Transition stage, services and processes designed in the **Service Design stage are transitioned into a live environment.**
- **To build and deploy IT services.** Service Transition also makes sure that changes to services and Service Management processes are carried out in a coordinated way.
- Service Transition introduces the concept of the **Service Knowledge Management System** which has the ability to support organizational learning and assist in **increasing the effectiveness and efficiency of all stages of the service lifecycle**, helps to support decision-making and improve the management of services.

- **Service Operation** Service operation ensures that services are being provided efficiently and effectively as per SLAs. It includes monitoring services, resolving incidents, fulfilling requests and carrying out operational tasks.
- provides guidance on **service delivery and support** so as to ensure value for the customer and service provider. IT service operation consists of periodic activities, infrastructure, and processes that deliver value to businesses using technology.
- Once the house is constructed buying house hold articles like light,fan,TV etc. and using it . You may under danger like fire accident in the kitchen this is called incident which has to manage like calling fire engine.
- In IT industry monitoring the environment and restoring the incidents came under service operation.

- **Continual Service Improvement** provides guidance on creating value for customers through better service management
Step-wise improvement.
- Continuous Service Improvement (CSI) deals with measures to be taken to [improve the quality of services by learning from past successes and failures](#).
- Its purpose is to align and realign IT Services to the changing needs by identifying and implementing improvements to the changing business needs.

Process and Functions



Service Strategy Process

- **Strategy Management for IT Services**
- The goal of strategic management is to ensure IT services and their management align with the organization. During this stage, you will assess, define, and execute strategies for your service offerings.

1. Assessment

- Evaluate the **current market position** of the business or IT service provider.
- Identify **opportunities** and **constraints** affecting your services.
- Analyze:
 - Your **existing service offerings**
 - **Current vs. target customers**
 - **Competitor services and strategies**

- **2. Definition**

Based on the assessment, define:

- Strategic **goals** for the IT service provider
 - **Target customer segments**
 - **Recommended services** to meet business and customer needs
- Develop a strategy that supports business growth and service differentiation.

- **Execution**
- The final step is implementation. This step is all about planning your strategy for the successful execution of your strategic initiatives.
- Communicate and implement the strategy across the organization.
- Align services, processes, and capabilities with strategy.
- Monitor and adjust the strategy as needed

- **Financial management-** IT Financial Management provides a means of understanding and managing costs and opportunities associated with services in financial terms. IT Financial Management includes three basic activities:
 - **1. Accounting**
 - Tracking how money is actually spent by a Service Provider
 - Provides detailed records of **where, how, and on what** money is spent.
 - Helps in analyzing cost efficiency and identifying wastage.
 - **2. Budgeting**
 - Planning how the money will be spent by a Service Provider
 - Involves **forecasting and planning** future expenditures.
 - Aligns financial resources with strategic and operational objectives.
 - Ensures funding is available to support planned service delivery and improvements.

- **3. Charging**

- Aims to **recover costs** by billing customers for the services used.
- Promotes **responsible usage** of resources by making costs transparent.
- Can be used to **incentivize efficiency** and support cost optimization.

- **Service Portfolio Management** - Service portfolio **explains the business needs and the response from providers to those needs**. The service portfolio is a combination, or we can say set of services managed by a service provider.
- Service portfolio management (SPM) monitors your services in the pipeline from start to finish. The goal of SPM is to ensure each service **aligns with your service management strategy and organizational objectives**.
- By monitoring your services end-to-end, you can more effectively justify service needs based on concrete business value.

- Three categories of services which are there in Service portfolio are:
 - Service pipeline:** services which are proposed or which are in development phase.
 - Service catalogue:** services which are live or ready for deployment.
 - Retired services:** Services which are no longer functional.
- In IT service management, the service **portfolio shows the investment**, which are made in an organization's services, it also represents **the value that services in return provide**.
- **Demand Management Process**-The Demand Management process is concerned with understanding, anticipating and influencing customer demand.
- Accurately understanding and adapting to service demand ensures **businesses avoid inadequate or excessive service capacity—both of which impact costs and customer satisfaction**.

- **Analyzing**
- Analyze current customer use of services by tracking service desk data (e.g., number of incidents, requests, and problems), as well as network usage and uptime.
- **Anticipating**
- Communicate with your customers about their forecasted needs, track trends and rely on your data analysis to make educated predictions about future service requirements.
- **Influencing**
- Businesses sometimes need to influence customer service consumption to mitigate risks and expenses. For example, if a customer exceeds their expected service usage, this can add significant costs for the business to meet that demand.

- **Business Relationship Management**
- Process Objective: To maintain a positive relationship with customers. Business Relationship Management identifies the needs of existing and potential customers and ensures that appropriate services are developed to meet those needs.

- **Design coordination**-The ITIL design coordination process ensures all design processes, activities, and resources are coordinated effectively throughout the entire service design stage.
- It is also responsible for producing the service design package (SDP).
- Which describes how a new or changed service should be designed, tested, and operated.

- **Service Catalog Management**-The Service Catalog is the subset of the Service Portfolio **which contains services currently available to customers and users**. The Service Catalog is often the only portion of the Service Portfolio visible to customers. **The Service Catalog commonly acts as the entry portal for all information services in the live environment**
- The catalog supports **service delivery and user awareness**, forming a foundation for other ITIL processes.
- **Service Level Management** is the process charged with securing and managing agreements between customers and the service provider regarding the levels of performance (utility) and levels of reliability (warranty) associated with specific services.
- Service Level Management results in the creation of Service Level Agreements(SLAs) between customers and the provider.

- **Availability Management process**-Ensures IT services are available and functioning when needed. It also ensures new or changed service meet availability goals and doesn't affect the existing services.
- Availability Management is responsible for ensuring that all IT infrastructure, processes, tools, roles etc. **are appropriate for the agreed availability targets.**
- In ITIL v4, availability is defined as the ability of a system, service, or configuration item to perform its function when required
- **Capacity Management** –It deals with ensuring that cost-effective capacity exists at all times which meets or exceeds the needs of the business as established in Service Level Agreements.
- Capacity Management considers all resources required to deliver the IT service, and plans for short, medium and long term business requirements.
- In ITIL, capacity is defined as the maximum throughput a service, system, or device can handle.

- Capacity Management is divided into three major activities:
 - **Business Capacity Management (BCM)**-deals with forecasting and developing plans for future business needs.
 - It means making sure that **IT systems are ready to handle future business demand** — by analyzing trends, predicting needs, and planning resources in advance.
 - This aspect involves understanding the current and future business requirements for IT services. It aims to align IT resources with business needs and ensure that the IT infrastructure can support the organization's goals.

- **Service Capacity Management (SCM)** is a sub-process of Capacity Management that focuses on the **performance and resource needs of specific IT services**.
 - It ensures that **resources such as CPU, memory, and storage** are **sufficient** to meet the service's performance targets as defined in **Service Level Agreements (SLAs)**.
 - It involves **monitoring how each IT service uses resources**, and planning for variations in demand to maintain smooth performance.
- **Component Capacity Management (CCM)** focuses on **optimizing the use of individual IT components**, such as servers, storage, network links, and databases.
 - It deals with **monitoring and managing the capacity** of each hardware and software component.
 - The goal is to **prevent bottlenecks and performance issues** by ensuring that no component becomes overloaded.

- **IT Service Continuity Management (ITSCM)** ensures that the **IT service provider can continue to deliver the minimum agreed level of service**, even during or after a **major disruption or disaster**.
 - It reduces the **risk of disaster events** to an acceptable level.
 - It involves **planning, preparing, and testing** for the **recovery of IT services**.
 - Supports the organization's **business continuity plan (BCP)**.

- **IT Security Management**-IT Security Management focuses on the protection of five basic qualities of information assets:
 1. Confidentiality: Assurance that the asset is only available to appropriate parties.
 2. Integrity: Assurance that the asset has not been modified by unauthorized parties.
 3. Availability: Assurance that the asset may be utilized when required.
 4. Authenticity: Assurance that the transactions and the identities of parties to transactions are genuine .
 5. Non-Repudiation: Assurance that transactions, once completed, may not be reversed without approval.

- **Supplier Management**-Supplier Management is the process charged with obtaining value for money from third-party suppliers. Supplier Management handles supplier evaluation, contract negotiations, performance reviews, renewals, and terminations.
- To ensure that all contracts with suppliers support the needs of the business, and that all suppliers meet their contractual commitments

- The objective of the Service Transition process is to build and deploy IT services by also making sure that changes to services and Service Management processes are carried out in a coordinated way.
- **Process**
- **Change Management**-The objective of this processing activity is to control the lifecycle of all the changes. The primary objective of Change Management is to enable beneficial changes to be made with minimum disruption to IT services and helping people to adapt to change.
- is a process which assures that the changes which are taking place in the organization are monitored throughout the stages so that no scope of error is left unturned. To ensure that changes are implemented effectively and efficiently.
- **Change Evaluation**-The objective of the change evaluation process is to assess major changes, like the introduction of a new service or a substantial change to an existing service, before those changes are allowed to proceed to the next phase in their lifecycle.

- The primary goal of this process is to determine whether the changes have achieved their intended outcomes and to identify any issues that need to be addressed.
- **Project Management** (Transition Planning and Support)
This process is aimed at planning and coordinating use of resources to deploy a major release within the predicted cost, time and quality estimates.
- **Application Development**-This process includes the development and maintenance of custom applications as well as the customization of products from software vendors. To make available applications and systems which provide the required functionality for IT services.
- **Release and Deployment Management**-It includes planning, designing, building, testing and deploying new software and hardware components in the live environment. The primary goal is to ensure that the integrity of the live environment is protected and that the correct components are released.

- **Service Validation and Testing**-This ensures that deployed releases and the resulting services meet customer expectations, and to verify that IT operations are able to support the new service.
- **Service Asset and Configuration Management**-The objective is to maintain information about Configuration Items required to deliver an IT service, including their relationships.
- **Knowledge Management**-The objective is to gather, analyze, store and share knowledge and information within an organization. **The primary purpose of Knowledge Management is to improve efficiency by reducing the need to rediscover knowledge.**
- Knowledge is composed of the concepts, experiences, ideas, insights values and judgments of individuals

- Service operation ensures that services are being provided efficiently and effectively as per SLAs. It **includes monitoring services, resolving incidents, fulfilling requests and carrying out operational tasks.**

Process

- **Event Management**-Event management can be described as the process that monitors all events that occur through the IT infrastructure to allow for normal operation and also to detect and escalate exception conditions.
- The objective is to make sure CIs and services **are constantly monitored and to filter and categorize Events in order to decide on appropriate actions.** It ensures all operations run smoothly and that each event is handled in a timely manner with the appropriate response.
- **Incident Management**-Incident management is the process of restoring **normal service operations as quickly as possible** following an incident . Incident Management ensures that the agreed levels **of service quality** are maintained.

- **Request Fulfilment-** The purpose of Request Fulfilment process is to manage the lifecycle of all service requests from the users.
- To maintain user and customer satisfaction through efficient and professional handling of all service requests
- The objective is to fulfil Service Requests, which in most cases are minor Changes (e.g. requests to change a password) or requests for information.
- **Access Management-**The objective is to grant authorized users the right to use a service and also ensure to prevent access to unauthorized users.
- Access Management is something also referred to as Rights Management or Identity Management. To manage access to services based on policies and actions defined in Information Security Management
- **Problem Management-**The process objective is to manage the lifecycle of all problems. Problem management aims to identify and resolve the root causes of recurring incidents..

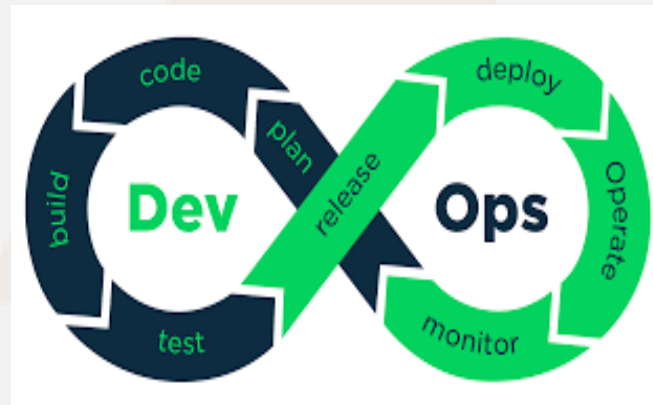
- The primary objectives of Problem Management are to prevent Incidents from happening and to minimize the impact of incidents that cannot be prevented
- **IT Operations Control**-This function is responsible for managing organization's day-to-day operational activities. This includes job scheduling, backup and restoring activities, print and output management, and routine maintenance .
- It ensures that the IT environment remains stable and available.
- **Technical Management**-Technical Management provides technical expertise and support for the management of the IT infrastructure.

- The objective of this component is to use methods from quality management to learn from past successes and failures.
- The Continual Service Improvement process aims to continually improve the effectiveness and efficiency of IT processes and services in line with the concept of continual improvement adopted in ISO 2000.
- CSI offers a mechanism for the IT organization to measure and improve service levels, the technology, efficiency and effectiveness of processes used in the overall management of services.

- **Service Review**-The objective of Service Review is to review business services and infrastructure services on a regular basis.
- The aim of this process is to improve service quality where necessary and to identify more economical ways of providing a service where possible.
- Service reviews typically involve examining service-level agreements (SLAs), customer feedback, service availability, reliability, capacity, and overall customer experience.
- **Process Evaluation**-The objective of Process Evaluation is to evaluate processes on a regular basis. This includes identifying areas where the targeted process metrics are not reached, and holding regular benchmarking, audits, maturity assessments and reviews.
- Process reviews involve analyzing the process flow, documentation, roles and responsibilities, process metrics, compliance with policies and standards, and opportunities for process improvement.

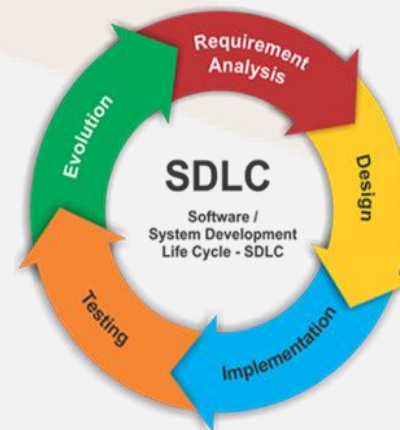
- **CSI Initiatives**-The objective of CSI initiatives is to define **specific initiatives aimed at improving services and processes**, based on the results of service reviews and process evaluations.
- The resulting initiatives are either **internal initiatives**, pursued by the **service provider on his/her own behalf**, or initiatives which require the **customer's cooperation**.
- **Monitoring of CSI Initiatives**-The objective is to verify if improvement **initiatives are proceeding according to plan and to introduce corrective measures where necessary**.

Introduction to Devops

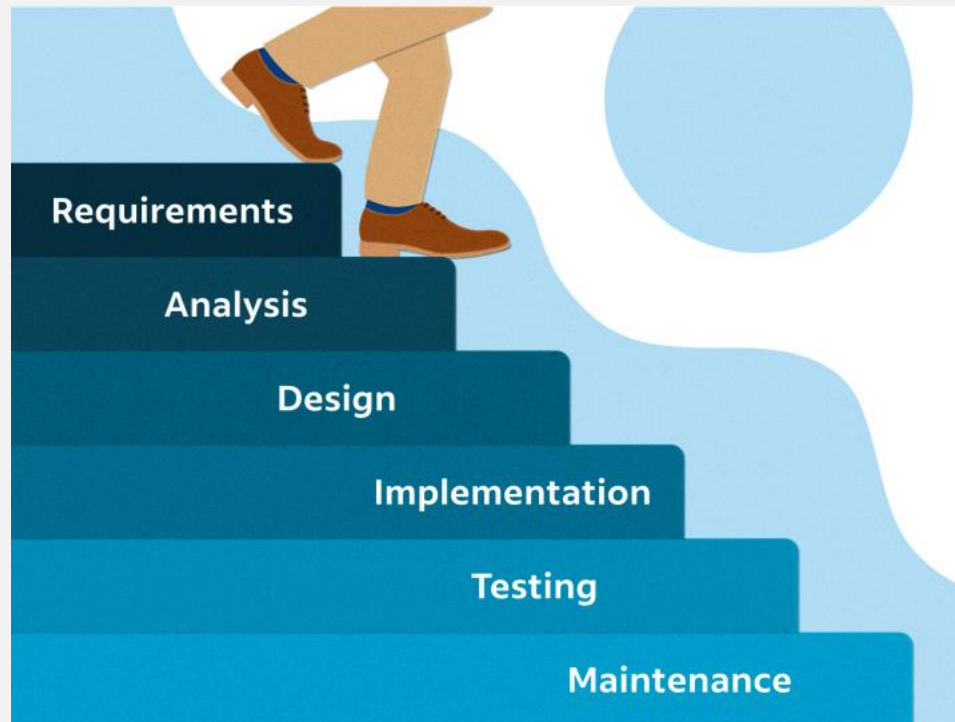


SDLC(Software Development Life Cycle)

- ▲ SDLC is a process followed for a software project, within a software organization. It consists of a detailed plan describing how to develop, maintain, replace and alter or enhance specific software.
- ▲ The life cycle defines a methodology for improving the quality of software and the overall development process.



Waterfall Model



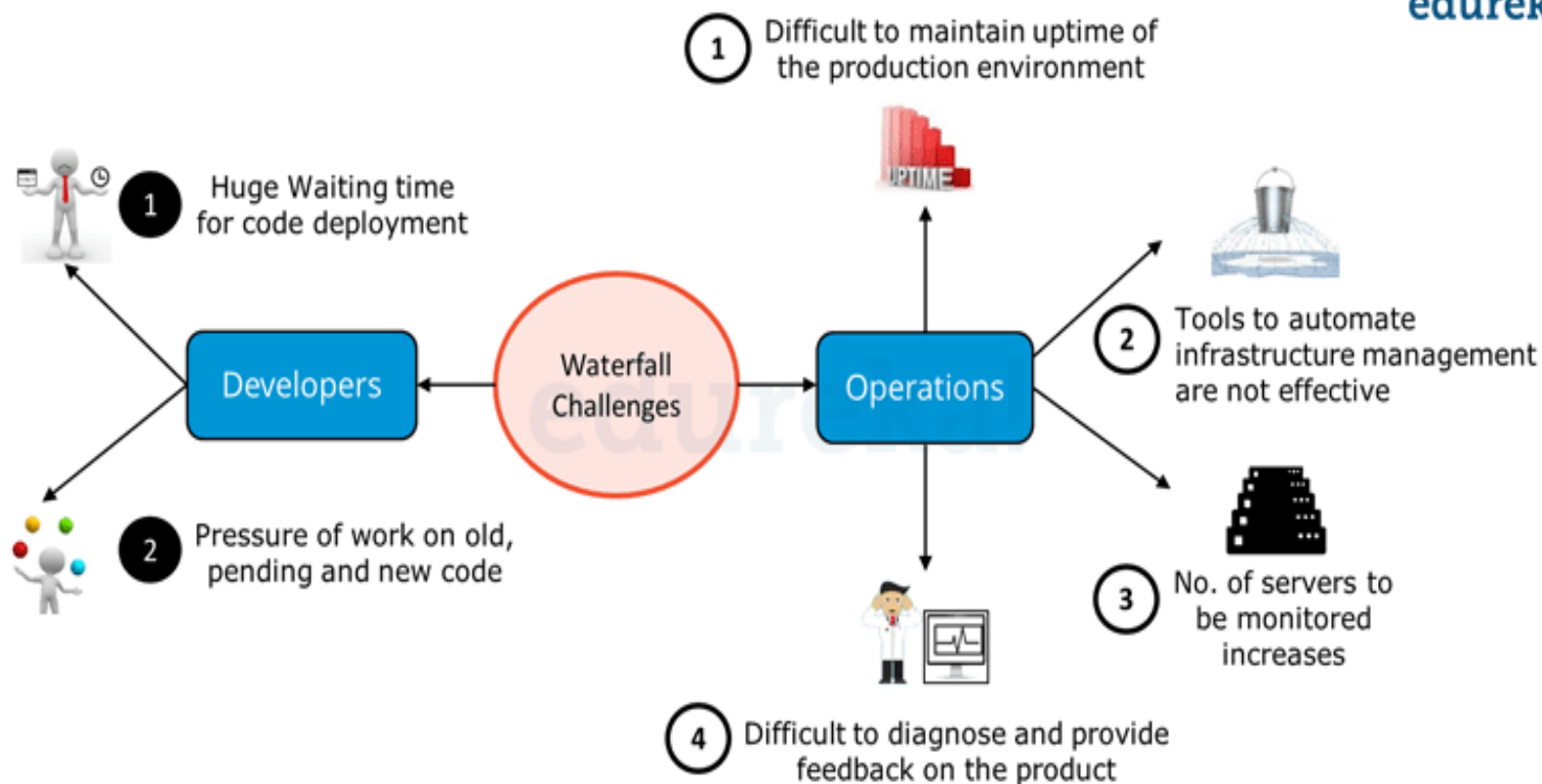
Waterfall Model

edureka!



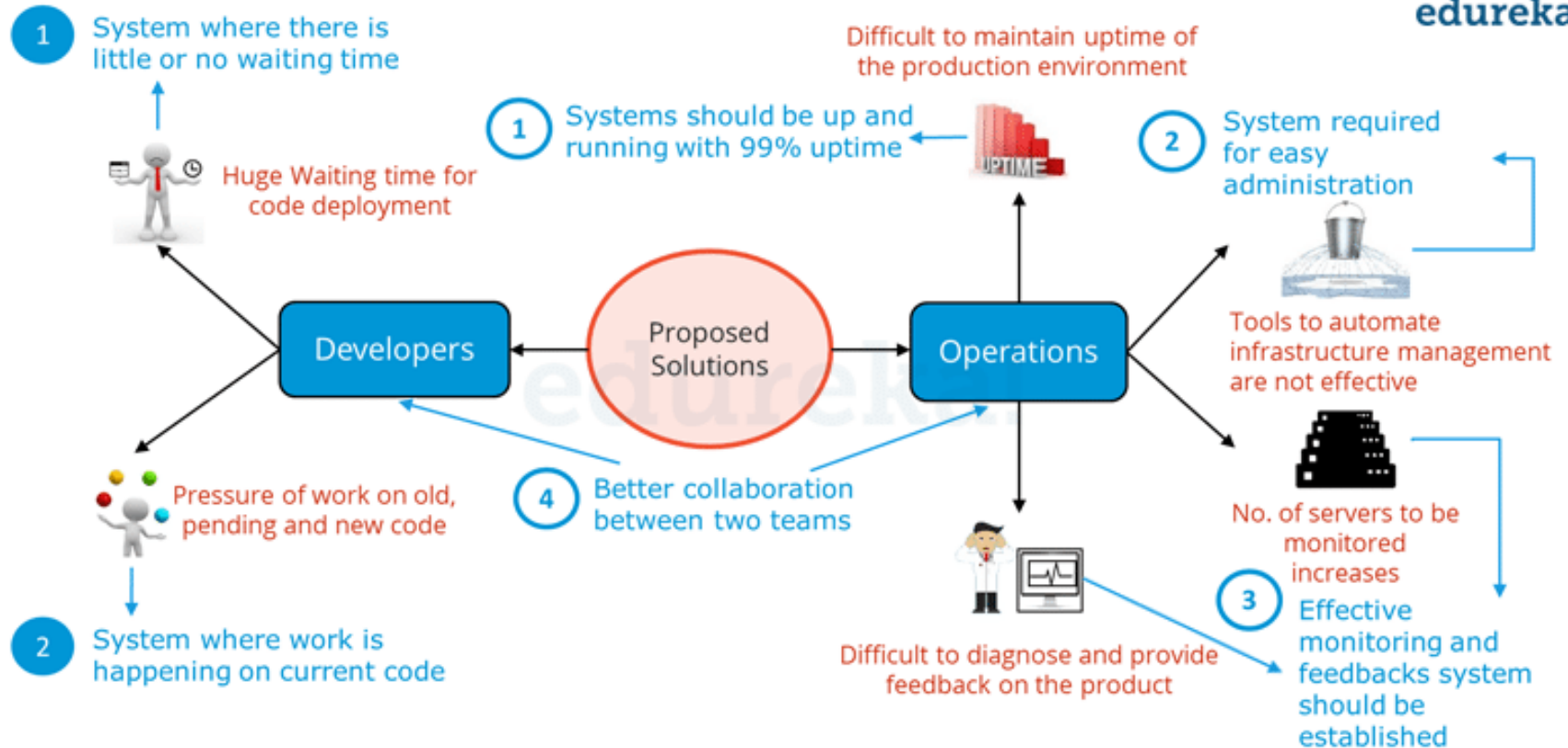
- ▲ In the above diagram you will see the phases it will involve:
- ▲ In phase 1 – Complete Requirement is gathered and SRS is developed
- ▲ In phase 2 – This System is Planned and Designed using the SRS
- ▲ In phase 3 – Implementation of the System takes place
- ▲ In phase 4 – System is tested and its quality is assured
- ▲ In phase 5 – System is deployed to the end users
- ▲ In phase 6 – Regular Maintenance of the system is done

Waterfall Model Challenges



- ▲ Developers point of view there were majorly two challenges:
 - 1 After Development, the code deployment time was huge.
 - 2 Pressure of work on old, pending and new code was high because development and deployment time was high.
- ▲ Operation team
 - 1 It was difficult to maintain 100% uptime of the production environment.
 - 2 Infrastructure Automation tools were not very affective.
 - 3 Number of servers to be monitored keeps on increasing with time and hence the complexity.
 - 4 It was very difficult to provide feedback and diagnose issue in the product.

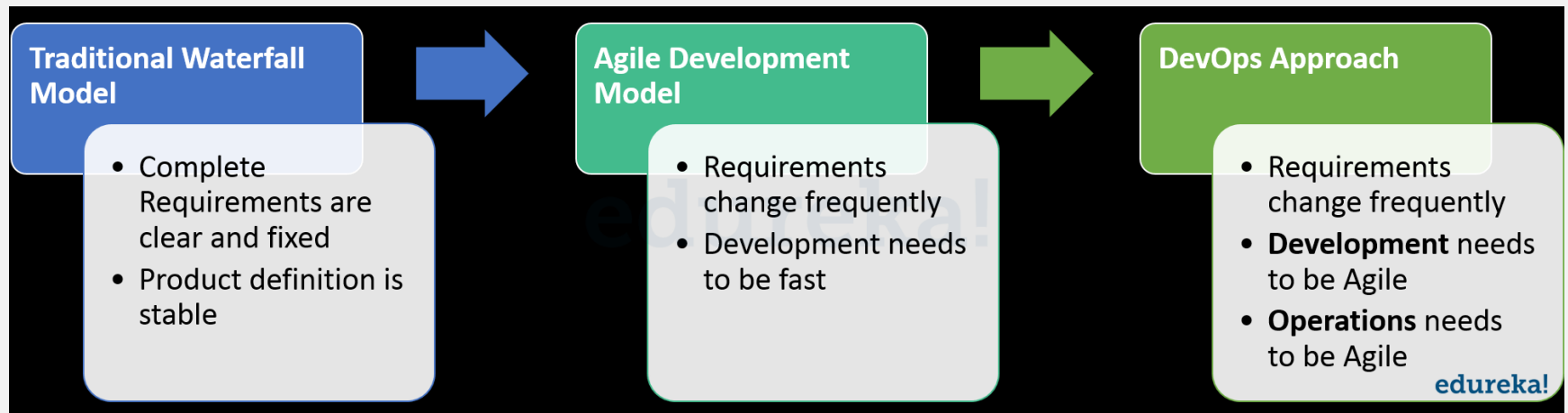
Solutions



Agile Model



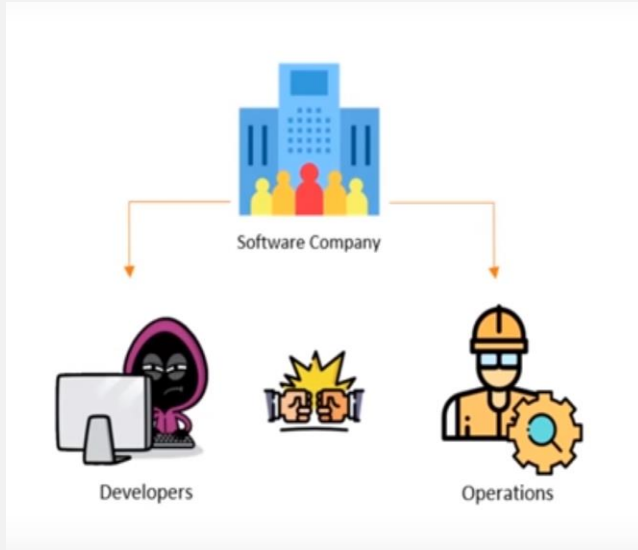
Evolution of Software Development



What is Devops?

- ▲ DevOps is a methodology not a technology.
- ▲ Is a culture which promotes collaboration between Development and Operations Team to deploy code to production faster in an automated.
- ▲ The word 'DevOps' is a combination of two words 'development' and 'operations.'
- ▲ DevOps helps to increases an organization's speed to deliver applications and services. It allows organizations to serve their customers better and compete more strongly in the market.

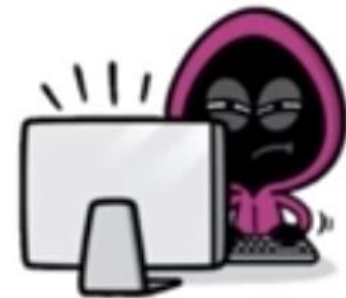
Why DevOps?



- Although, the software quality was improved.
- We still had a lack of efficiency among Dev team.
- Dev and Ops teams are apart of development team.

Who is Developer and Ops?

- A developer is someone who writes and debugs code to applications.
- Once the development activity is completed, then developer will pass the code to Ops team.



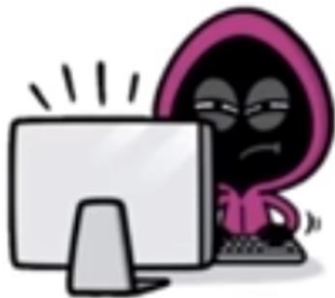
Developer

- The operations team role to test the code , and provide feedback to developers in case of any bugs – The feedback would be passed to developer, who developed the code.
- The Operations team has quite a significant role in the DevOps environment. A huge chunk of DevOps involves configuring, managing, provisioning and monitoring servers.



Operations

Problems with Dev and Ops.



Developer



Operations

Solved using DevOps!



How is DevOps different from traditional IT?

Traditional IT	Devops
Less Productive	More Productive
Skill Centric Team	Team is divided into specialized silos
More Time invested in planning	Smaller and Frequent releases lead to easy scheduling and less time in planning
Difficult to achieve target or goal	Frequent releases, with continuous feedback makes achieving targets easy

- ▲ The idea began in 2008 with a discussion between Patrick Debois and Andrew Clay Shafer concerning the concept of agile infrastructure. However, the idea only started to spread in 2009 with the advent of the first DevOpsDays event held in Belgium.
- ▲ DevOps is such an idea. But, it didn't look that way at first. Yet, DevOps has taken off in only a few years to become a major influence on software development and deployment.

▲ Who is a DevOps Engineer?

- ▲ DevOps Engineer is somebody who understands the Software Development Lifecycle and has the outright understanding of various automation tools for developing digital pipelines (CI/ CD pipelines).
- ▲ DevOps Engineer works with developers and the IT staff to oversee the code releases. They are either developers who get interested in deployment and network operations or sysadmins who have a passion for scripting and coding and move into the development side where they can improve the planning of test and deployment.

Devops Goals

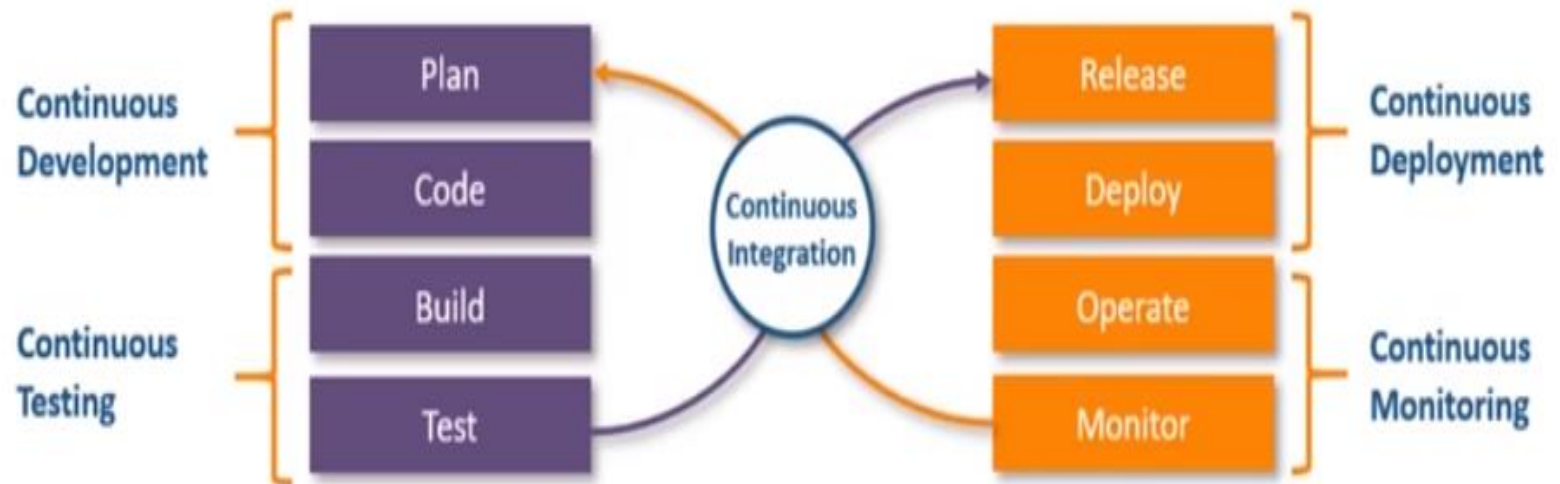
- ▲ Fast Development, Quality Assurance, Deployment .
- ▲ Faster time to market.
- ▲ Iteration & Continuous Feedback (strong and continuous communication between stakeholders — the end users and customers, product owners, development, quality assurance, and production engineers).

Devops Lifecycle

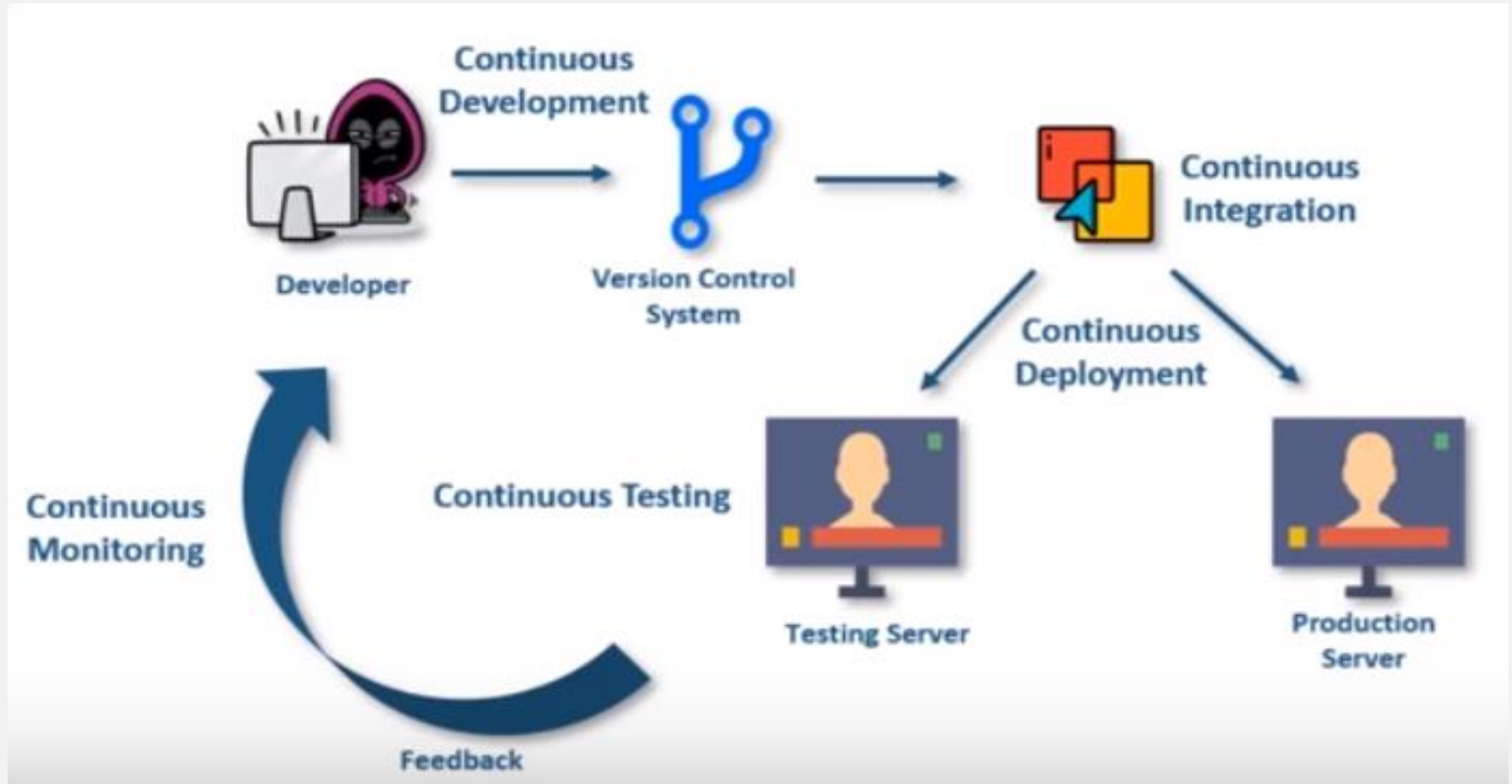


It is a continuous process – continuous development, testing, integration, deployment and monitoring.

How DevOps works ?



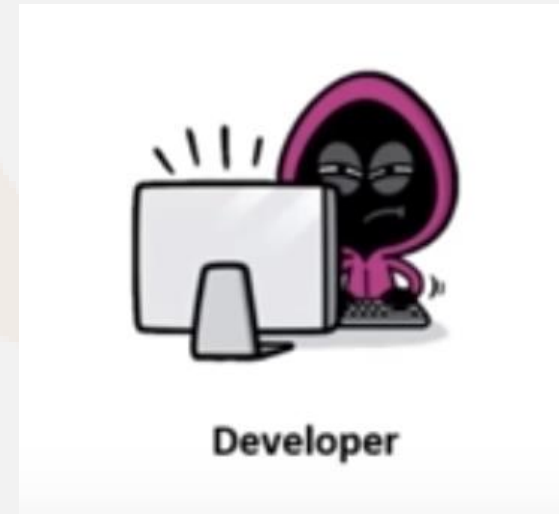
How DevOps works ?



- ▲ This stage involves committing code to version control tools such as **Git** for maintaining the different version of code.

Benefits of CD:

- Faster delivery of new features
- Better Quality Product
- Less Resource Requirement
- Increased Productivity



Faster delivery of new features

- ▲ With continuous development, new features are developed and deployed quickly with automated processes in place.
- ▲ As a result, the speed that companies are able to test new version of it, what they developed.
- ▲ If any , bugs can be identified and addressed almost immediately.
- ▲ By releasing new features rapidly, companies are able to generate a return on investment considerably faster than they would using other deployment methods.

Better Quality Product

- ▲ Product quality means to incorporate features that have a capacity to meet client needs.
- ▲ Gives customer satisfaction by improving products and making them free from any deficiencies or defects.
- ▲ While methods like the waterfall approach require software developers to assume which features are important to the user.
- ▲ If we see in DevOps – where in we can deliver the features or bug fix frequency.

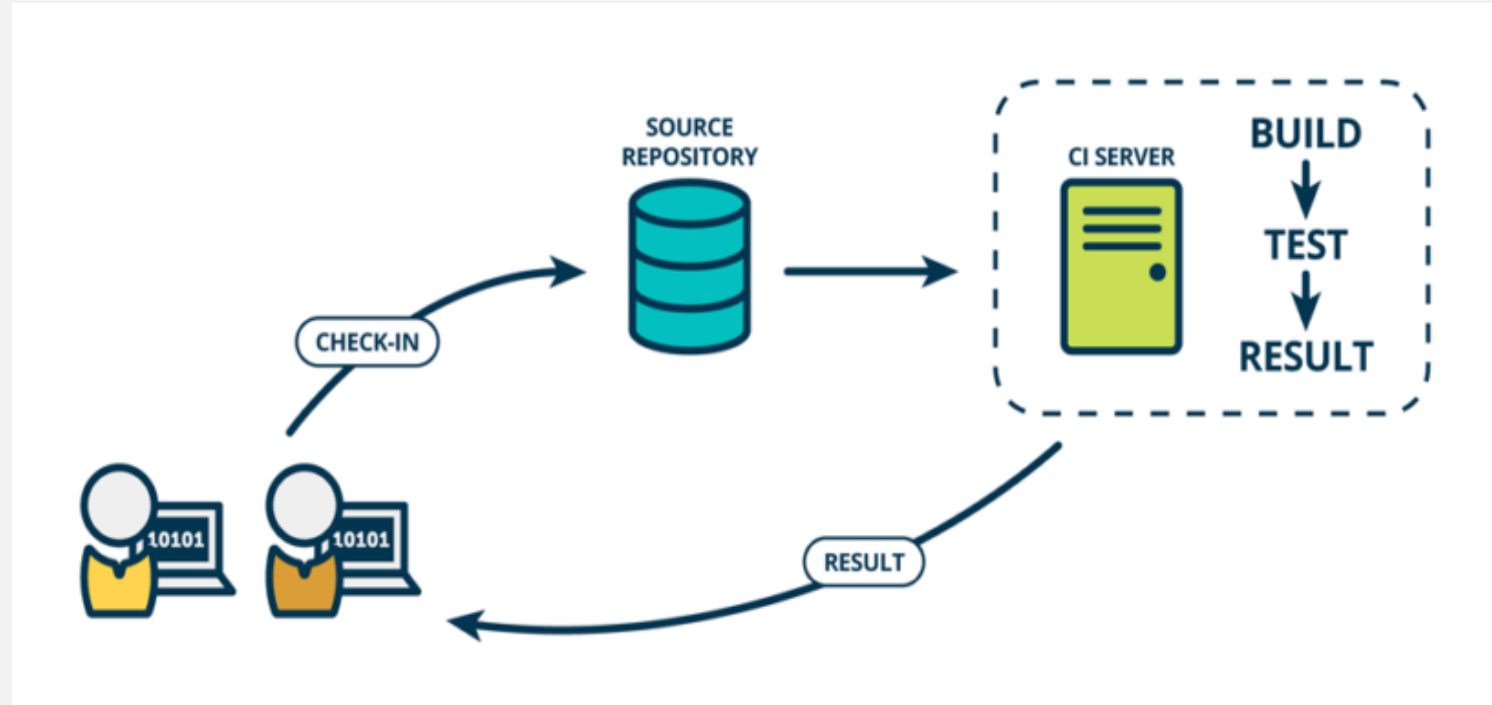
Less Resource Requirement

- ▲ Large updates can require a considerable amount of Dev in coding, deploying and testing.
- ▲ Increase overall efficiency and decrease the amount of time and Dev that is necessary to properly deploy new software features.

Increased Productivity

- ▲ Since the feedback loop is expedited, developers are given a full development workflow.
- ▲ By receiving continuous feedback from various stakeholders throughout the process.
- ▲ Developers are always provided with further tasks to better optimize the software.

Continuous Development



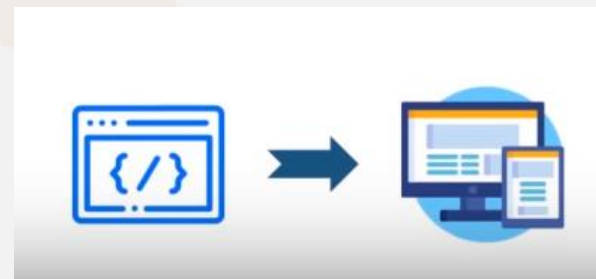
Continuous integration

- ▲ This stage is a critical point in whole DevOps lifecycle.
- ▲ It deals with integrating the different stages of the DevOps lifecycle.
- ▲ Means **CI** is acting like a gateway in whole DevOps process.



Continuous Deployment

- ▲ Continuous delivery is a software development practice where code changes are automatically prepared for a release.
- ▲ Deploying the source code to particular server like UAT,PROD etc..
- ▲ Its involves to ensure software are configured , pushing the build on the environment , where the application can works perfectly.
- ▲ We do have few DevOps tools to take care of configuration management like **Ansible, chef** etc.



Continuous testing

- ▲ This stage deals with automated testing of the application pushed by developer. if there is an error , the message is sent back to the integration tool, then tool turns the notification the developer about the error.
- ▲ If the test is success , then the tested code pushes to PROD server or any.
- ▲ Continuous Testing needs automated end-to-end testing solutions that integrate existing development processes while excluding errors and enabling continuity throughout SDLC.



Continuous monitoring

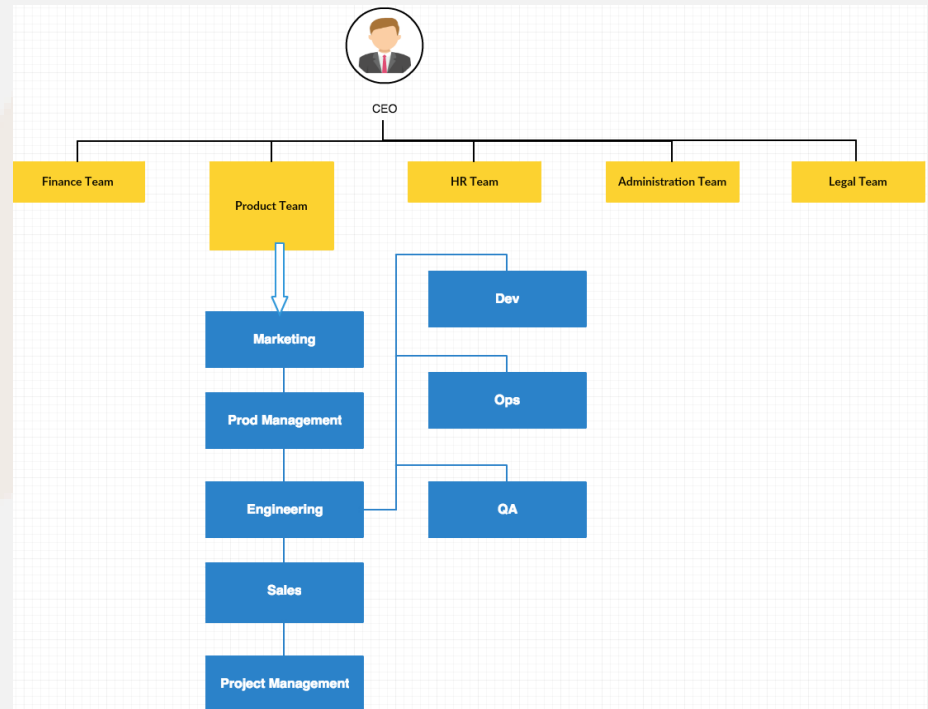
Application monitoring is a process that ensures that a software application processes and performs in an expected manner and scope.

This technique routinely identifies, measures and evaluates the performance of an application and provides the means to isolate and rectify any abnormalities or shortcomings.



Stakeholders for implementing DevOps

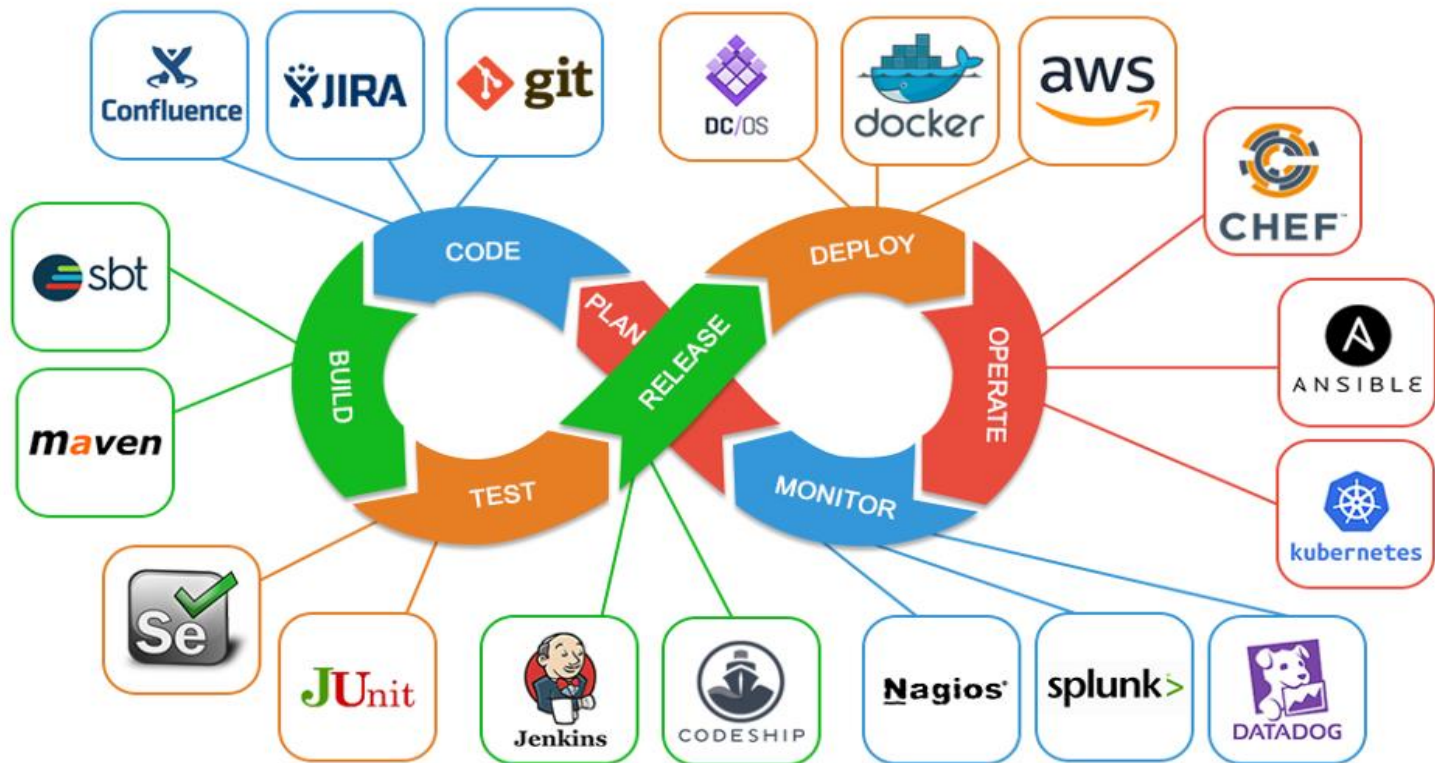
The Blue blocks are the departments focused on the Product. These are the people who are the stakeholders in the DevOps implementation.



Why do they care about devops?

- ▲ **Product Management** : They are interested in testing the different variants of the product and evolve the best of it, in order to do that they need a good devops support so that they can see the changes right away in prod with all the bells and whistles (tracking, monitoring, alerting).
- ▲ **Project Management** : Time is of essence for them. When they know that if devops mindset is in place they can smile as they know that code is shipped from the developers laptop to production on a shuttle.
- ▲ **Marketing** : They need to know that the product evolution in this shop is on steroids. We can overhaul the product and ship it overnight.
- ▲ **Engineering** : Is the technical team of developers and managers who write the code and create the front end, so the clearer the guidance they get upfront, the better. That doesn't mean micromanaging from the product **team**.
- ▲ **Customer Success** : ETA's for resolving issues in minutes, amazing monitoring and ability to change prod in minutes and maintain the stable state sounds like a recipe to delight the customers, hence devops is important for the customer success teams.

- ▲ Git and GitHub – Source code management (Version Control System)
- ▲ Jenkins – Automation server, with plugins built for developing CI/ CD pipelines
- ▲ Chef – Configuration Management and Deployment
- ▲ Ansible – Configuration Management and Deployment
- ▲ Nagios – Continuous Monitoring
- ▲ Maven - Build automation tool



Lean vs Agile

- ▲ “Lean” and “Agile” are terms references to software development methodologies, project management, or organizational styles.
- ▲ Agile and Lean are wildly popularized in the software development space for helping teams deliver faster and more sustainably.

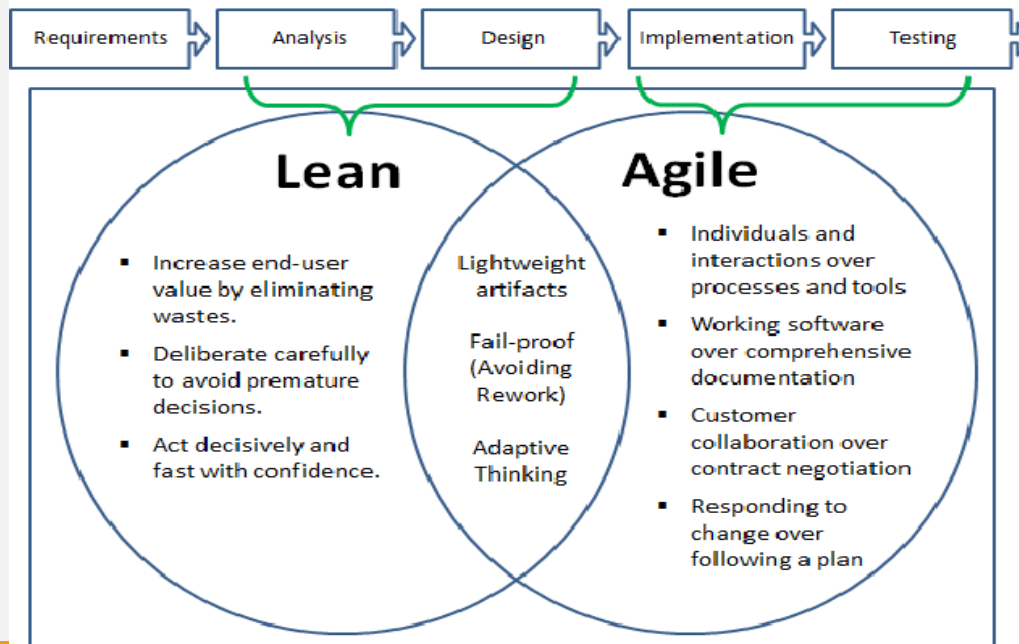
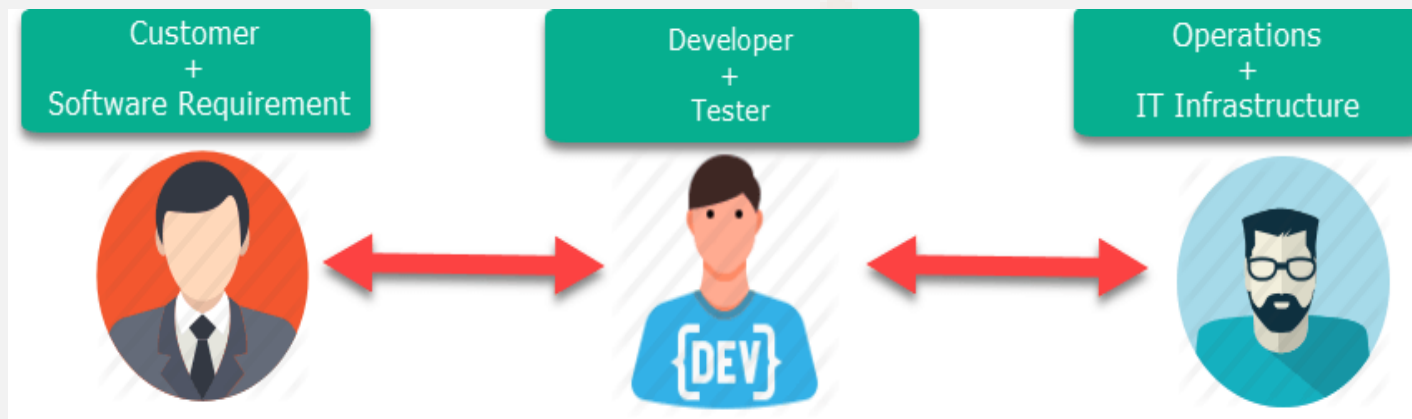


Figure 1. Lean and agile landscape

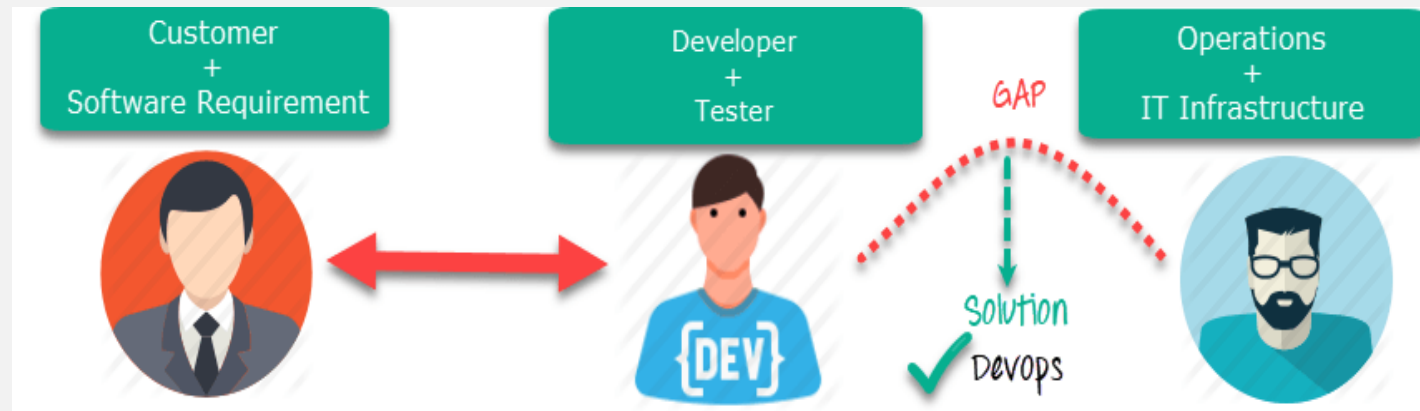
▲ Stakeholders and communication chain in a typical IT process.



- ▶ Agile addresses gaps in Customer and Developer communications.



▲ DevOps addresses gaps in Developer and IT Operations communications





Git

What is repository?

- ▲ In software development, a repository is a central file storage location.
- ▲ It is used by version control systems to store multiple versions of files. While a repository can be configured on a local machine for a single user, it is often stored on a server, which can be accessed by multiple users.
- ▲ A repository contains three primary elements — a **trunk**, **branches**, and **tags**.
- ▲ The **trunk** contains the current version of a software project.
- ▲ **Branches** are used to store new versions of the program. A developer may create a new branch whenever he makes substantial revisions to the program. If a branch contains unwanted changes, it can be discontinued. Otherwise, it can be merged back into the trunk as the latest version.
- ▲ **Tags** are used to save versions of a project, but are not meant for active development. For example, a developer may create a "release tag" each time a new version of the software is released.

GIT

- ▲ Git is created by Linus Torvald.
- ▲ Git handles merge different source code from different people and its maintaining the version of source code.
- ▲ Its called as **distributed version control system**.
- ▲ **Git** (/git/) is a distributed version-control system for tracking changes in source code during software development.



Short history of GIT

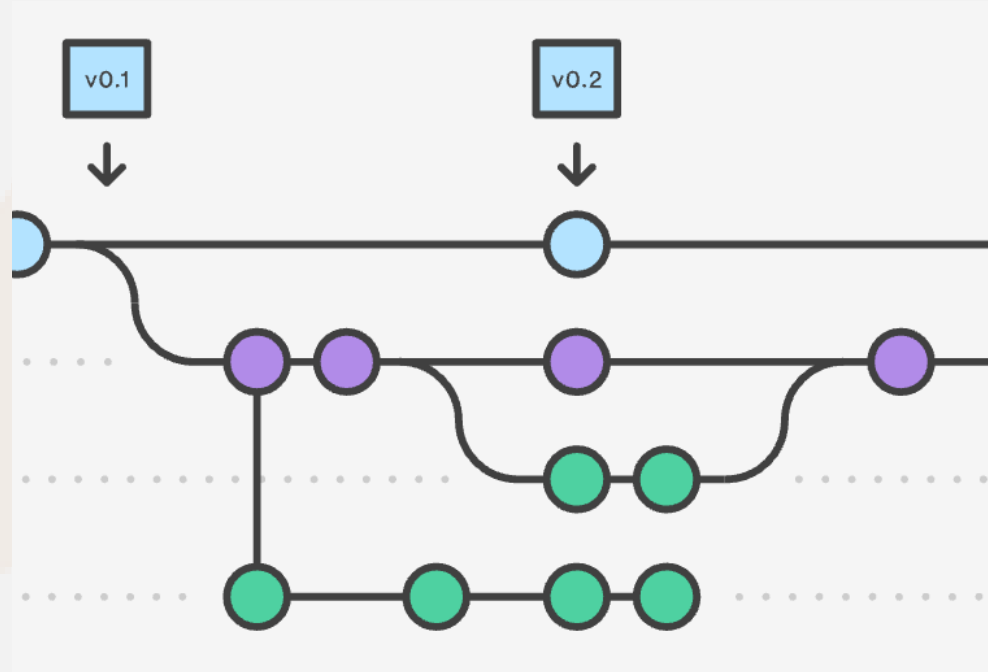
- ▲ The Linux kernel is an open source software project of fairly large scope. For most of the lifetime of the Linux kernel maintenance (1991–2002), changes to the software were passed around as patches and archived files.
- ▲ In 2002, the Linux kernel project began using a proprietary DVCS called BitKeeper.



- ▲ It's important to note that the terms "**checkin**" and "**checkout**" have different meanings depending on the type of SCM system.
- ▲ **Checkin** code means to upload code to mentioned/required branch repository so that its administrator/reviewer can review the code and finally update/merge the project version.
- ▲ **Checkout** code is the exactly opposite which means to download a copy of code from the repository.

Version Control System (VCS)

- ▲ Version Control System (VCS) is a software that helps software developers to work together and maintain a complete history of their work.
- ▲ Functions of a VCS –
 - Allows developers to work simultaneously.
 - Does not allow overwriting each other's changes.
 - Maintains a history of every version.



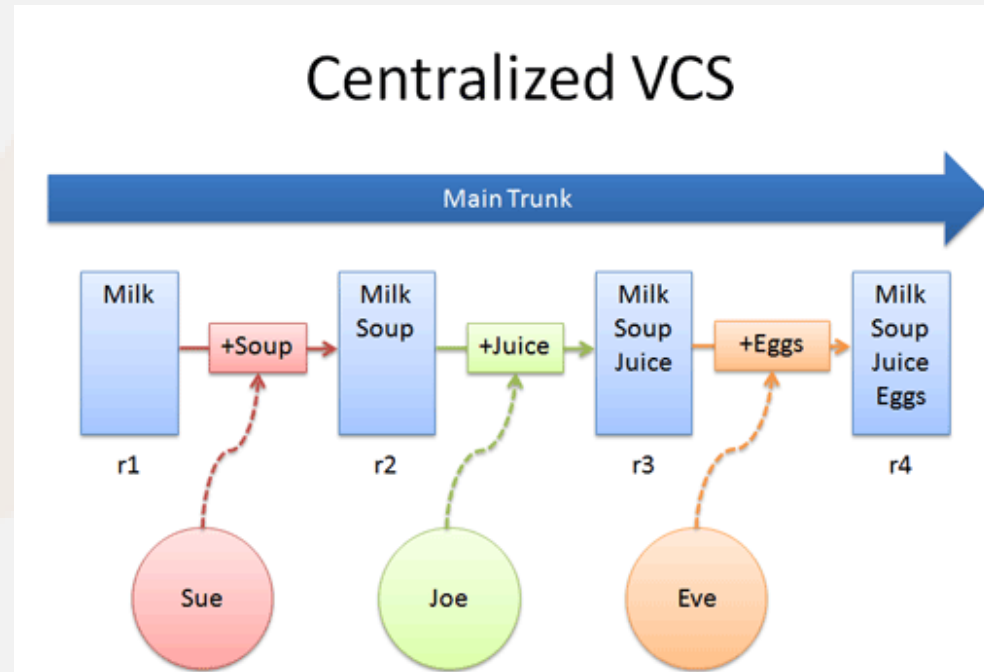
Types of VCS –

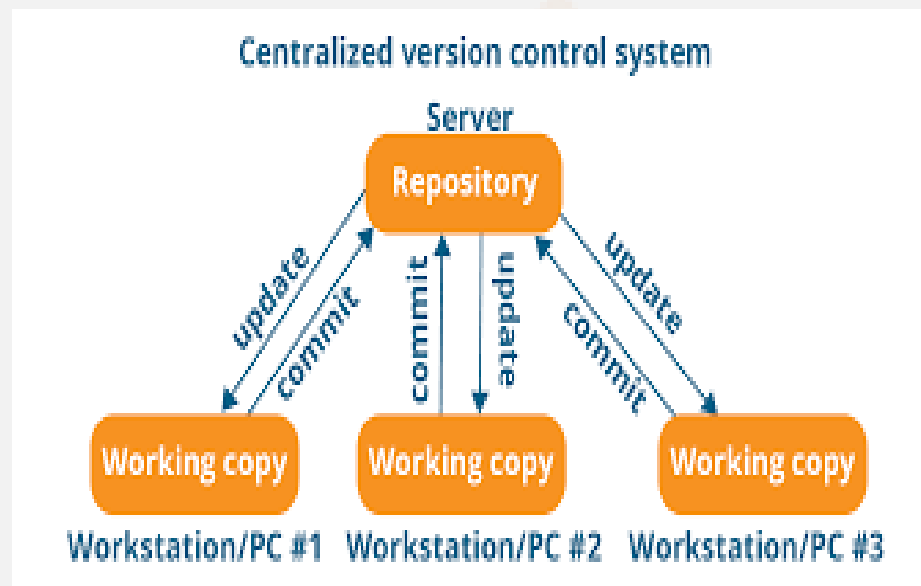
- ▲ Centralized version control system (CVCS).
- ▲ Distributed/Decentralized version control system (DVCS).



Centralized version control system (CVCS)

- ▲ Centralized version control system (CVCS) uses a central server to store all files and enables team collaboration.
- ▲ But the major drawback of CVCS is its single point of failure, i.e., failure of the central server. Unfortunately, if the central server goes down Developers won't "Check in or Check out". Since the CVCS server is down and not accessible.

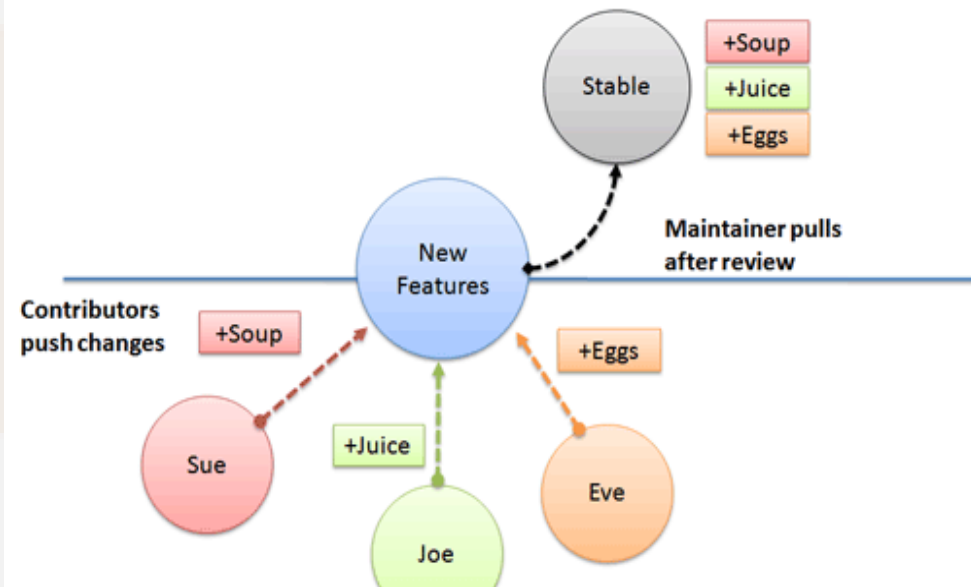


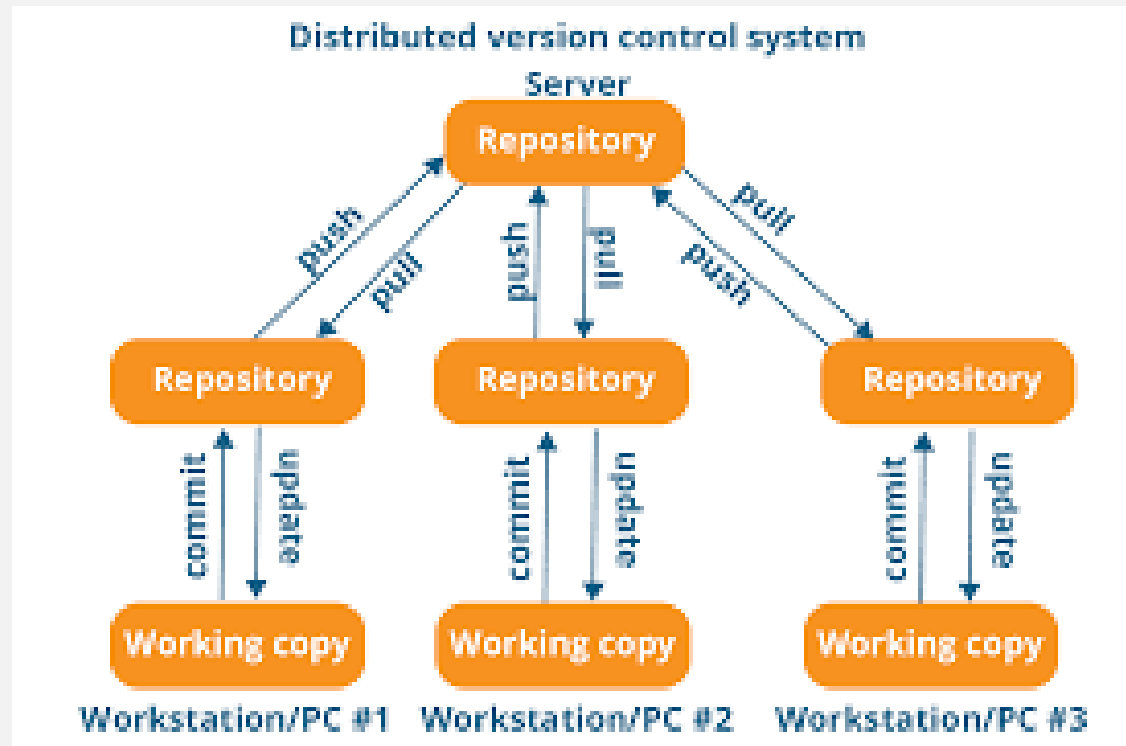


DISTRIBUTED VERSION CONTROL SYSTEM(DVCS)

- ▲ DVCS clients not only check out the latest snapshot of the directory but they also fully mirror the repository. If the server goes down, then the repository from any client can be copied back to the server to restore it. Every checkout is a full backup of the repository.
- ▲ Git does not rely on the central server and that is why you can perform many operations when you are offline.
- ▲ You can commit changes, create branches, view logs, and perform other operations when you are offline. You require network connection only to publish your changes and take the latest changes.

Distributed Push/Pull Model





- ▲ Centralized systems like TFVC, Subversion, and Clearcase use "exclusive" checkouts.
- ▲ For example, on GitHub, cloning means (similar to) checking-out code and pull request means (similar to) checking-in code.
- ▲ Distributed systems like git have a "checkout" command, but it means something completely different. git checkout is used to switch between branches when working with a local repository.
- ▲ In Git terms, a "checkout" is the act of switching between different versions of a target entity. The git checkout command lets you navigate between the branches created by git branch. Checking out a branch updates the files in the working directory to match
- ▲ There is no such git checkin command in git

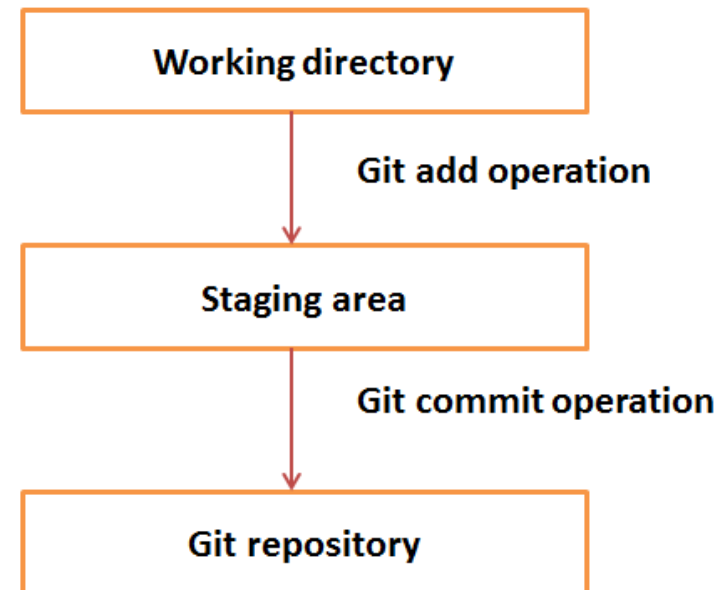
Advantages of Git

- ▲ Free and open source
 - ▲ Fast and small
 - ▲ Security
 - ▲ No need of powerful hardware
 - ▲ Easier branching
- 

- ▲ **Local Repository:** Every VCS tool provides a private workplace as a working copy.
- ▲ Developers make changes in their private workplace and after commit, these changes become a part of the repository. Git takes it one step further by providing them a private copy of the whole repository.
- ▲ **Working Directory and Staging Area or Index:** The working directory is the place where files are checked out.
- ▲ Committing is the process in which the code is added to the **local repository**. Before committing the code, it has to be in the **staging area**. The staging area is there to keep track of all the files which are to be committed. But Git uses a different strategy.
- ▲ Whenever you do commit an operation, Git looks for the files present in the staging area. Only those files present in the staging area are considered for commit and not all the modified files.

Workflow of Git

1. You modify a file from the working directory.
2. You add these files to the staging area.
3. You perform commit operation that moves the files from the staging area. After push operation, it stores the changes permanently to the Git repository.



▲ **git add**

Puts current working files into the stage (aka index or cache)

▲ **git checkout**

Replaces the current working files with files from a branch.

▲ **git checkout -b**

Creates a new local branch from the current branch's tip.

▲ **git clone**

Clone an existing repository into a new directory.

▲ **git commit**

Commits staged changes to a local branch

▲ **git commit -a**

Commits all modified files to a local branch (shorthand for "git add" followed by "git commit" for each modified file)

▲ **git fetch**

Downloads changes from a remote repository into the local clone

▲ **git merge**

Merges files from a given branch into the current branch.

▲ **git pull**

Fetches remote changes ***on the current branch*** into the local clone, and merges them into the current working files.

▲ **git push**

Uploads changes from ***all*** local branches to the respective remote repositories.

Trunk Based development

- ▲ In the trunk-based development model, all developers work on a single branch with open access to it. They commit code to it and run it.
- ▲ where all developers (for a particular deployable unit) commit to one shared branch under source-control.





- There is a system administrator whose responsibility is to maintain the upstate of all the systems in the network. There may be many systems that need maintenance, which is why it is okay if one of them goes down. That one system can be handled.
- However, if there are several issues in multiple systems at once, due to the existence of only a single source of contact, it would be extremely difficult to maintain all the systems at once leading to a severe condition.



- ▲ Chef is a configuration management tool for dealing with machine setup on physical servers, virtual machines and in the cloud. Many companies use Chef software to control and manage their infrastructure.
- ▲ Chef is a open source tool developed by Opscode and its written in Ruby and Erlang.
- ▲ We use Ruby to automate the process.
- ▲ If we look at DevOps life cycle – we could see that Chef is falls under operations and deployments.



Large companies changing their infrastructure that requires to be configured and maintained time to time.



That's when we use chef to automate the complete process

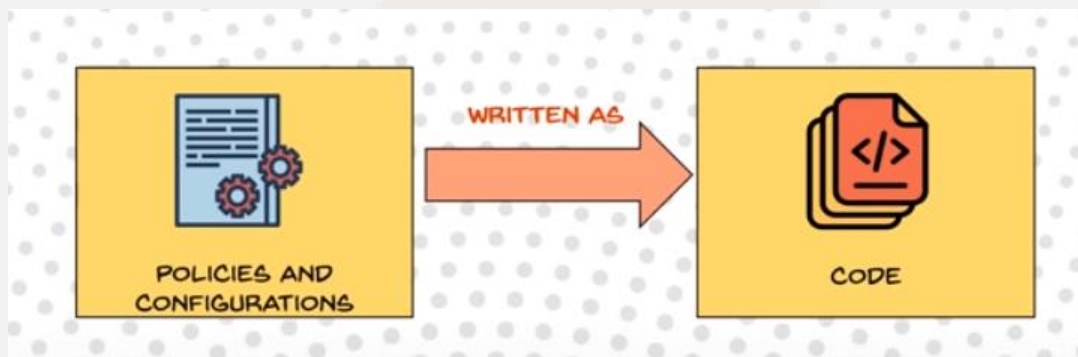
Why to use Chef?

- ▲ Chef can easily manage a large number of servers with fewer employees.
- ▲ It allows continuity in the deployment process from building to testing, until the end.
- Continuous deployment
- ▲ Software is deployed continuously enabling a company to keep the infrastructure up to date as per current market requirements.
- Adapts to cloud
- ▲ Chef easily adapts to most of the cloud services exists in the current market.

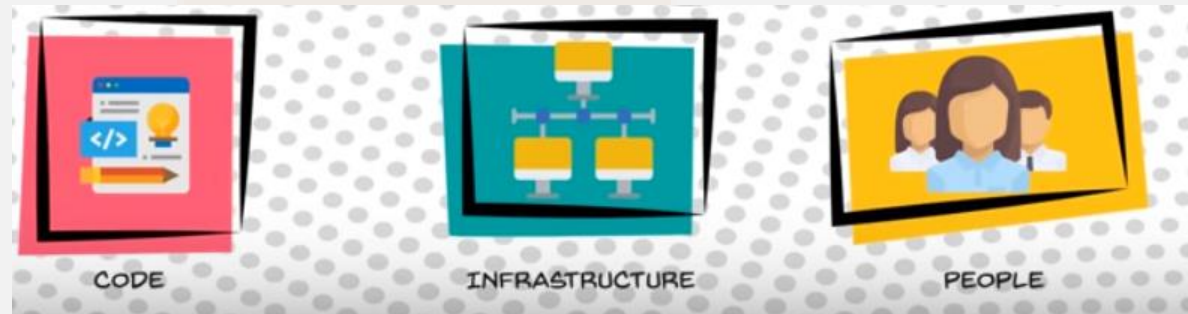
What is chef infrastructure?

- ▲ **Chef Infra** is a powerful automation platform that transforms **infrastructure** into code.
- ▲ Whether you're operating in the cloud, on-premises, or in a hybrid environment, **Chef Infra** automates how **infrastructure** is configured, deployed, and managed across your network, no matter its size.

- ▶ **Infrastructure as code** (IaC) is the process of managing and provisioning computer data centers through machine-readable **definition** files, rather than physical hardware configuration or interactive configuration tools.



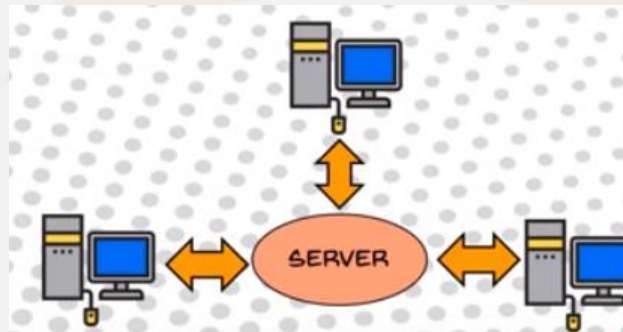
- ▲ Configuration management is a collection of engineering practices that provides systematic way to manage the entities for efficient deployment.
- ▲ Entities are:
 - Code
 - Infrastructure
 - People



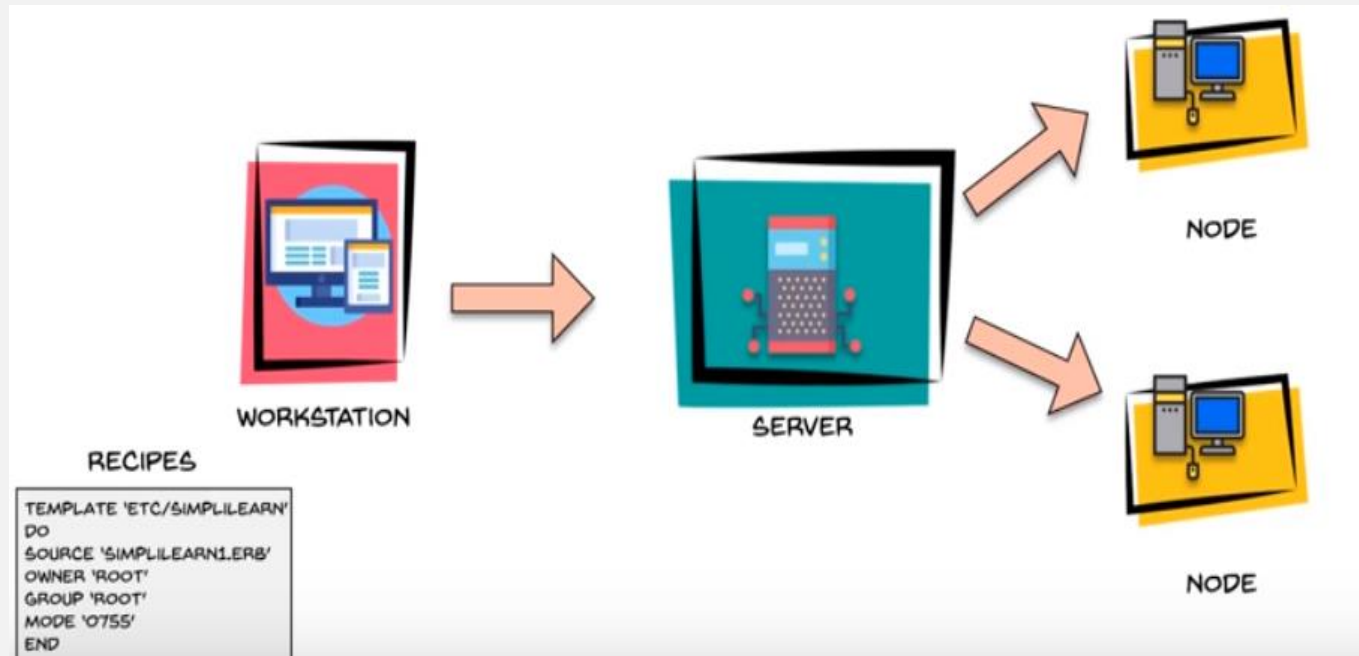
- ▲ **Code:** Basically the code what the system administrator write for configuring different system's.
- ▲ Codes needs to updated whenever the infrastructure needs new configuration or updates in operating system.
- ▲ **Infrastructure:** collect of system's and the server.
- ▲ As the required of companies change , we have upgrade our infrastructure.
- ▲ **People:** Teams to take care of your infrastructure.

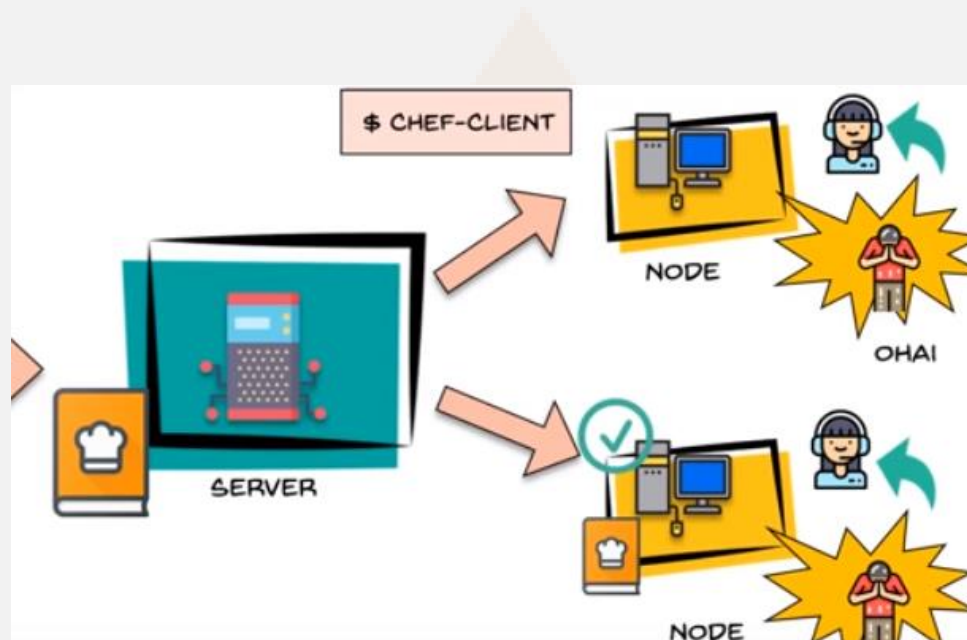
Chef follows under pull configuration

- ▲ A small software (called agent or client) is installed on every node. This agent/client will:
 - ▲ at regular intervals, get the configuration from the server
 - ▲ compare the configuration received from the server with the current configuration of the node
 - ▲ if there is any mis-match, take the steps required to match the configuration of the node with the configuration received from the server.



Chef architecture





- ▲ Chef has major components such as Workstation, Cookbook, Node, Chef-Client, and Chef-Server.

▲ Workstation

- ▲ The workstation is used to interact with Chef-server and also to interact with Chef-nodes.
- ▲ Workstation is a place where all the interaction takes place where Cookbooks are created, tested and deployed, and in workstation, codes are tested.

▲ Chef Server

- ▲ Chef server contains all configuration data and it stores cookbooks, recipes, and metadata that describe each node in the Chef-Client.
- ▲ Configuration details are given to node through Chef-Client.
- ▲ Any changes made must pass through the Chef server to be deployed. Prior to pushing the changes, it verifies that the nodes and workstation are paired with the server through the use of authorization keys, and then allow for communication between workstations and nodes.
- ▲ Chef server can be hosted on local or centralized server.

▲ **Chef Cookbook** - Cookbooks are fundamental working units of Chef, which consists of all the details related to working units, having the capability to modify configuration and the state of any system configured as a node on Chef infrastructure. Cookbooks can perform multiple tasks. Cookbooks contain values about the desired state of node.

▲ Key Components of a Cookbook

- ▲ Recipes
- ▲ Metadata
- ▲ Attributes
- ▲ Resources
- ▲ Templates
- ▲ Libraries
- ▲ Anything else that helps to create a system

▲ **Cookbooks**

- ▲ Cookbooks are created using Ruby language.
- ▲ A cookbook contains single or multiple recipes which specify resources to be used and in which order it is to be used.
- ▲ The cookbook contains all the details regarding the work and it changes the configuration of the Chef-Node.
- ▲ Cookbooks are created only on workstations.

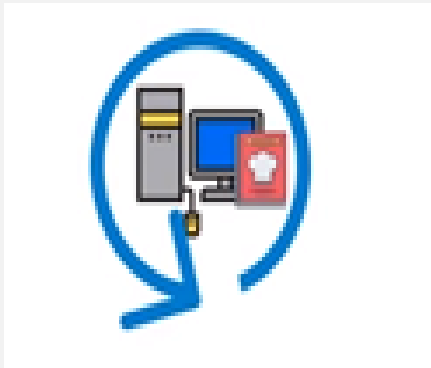
▲ **Nodes**

- ▲ Nodes are the systems that require the configuration.
- ▲ Nodes are managed by Chef and each node is configured by installing Chef-Client on it. Chef-Nodes are a machine such as physical, virtual, etc.

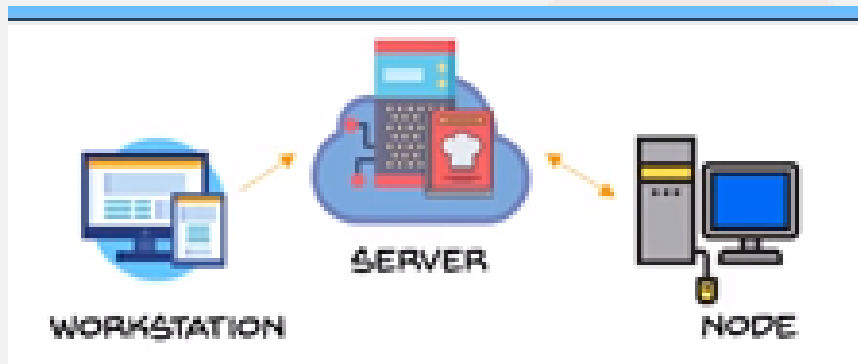


- ▲ Knife: Is a command line tool that uploads the cookbook to the server.
- ▲ Ohai: Ohai is used for determining the system state at beginning of Chef run in Chef-Client. It Collects all the system configuration data.

- ▲ Chef solo
- ▲ Chef solo has no remote server .The cookbook are located at local site it self.



- ▲ **Hosted chef**-Chef server is provided as a service on the cloud. No need to setup the server yourself.



- ▲ **Chef client/server:** This is the traditional chef architecture where a remote server is hosted that communicates between workstation and node



- ▲ **Private chef:** This is the enterprise version of chef where the server is hosted within the enterprise infrastructure.



- ▲ An **organization** is the top-level entity for role-based access control in the Chef Infra Server. Each organization contains the default groups (admins, clients, and users, billing_admins for the hosted Chef Infra Server), at least one user and at least one node (on which the Chef Infra Client is installed). The Chef Infra Server supports multiple organizations. The Chef Infra Server includes a single default organization that is defined during setup. Additional organizations can be created after the initial setup and configuration of the Chef Infra Server.
- ▲ A **group** is used to define access to object types and objects in the Chef Infra Server and also to assign permissions that determine what types of tasks are available to members of that group who are authorized to perform them. Groups are configured per-organization.
- ▲ Individual users who are members of a group will inherit the permissions assigned to the group. The Chef Infra Server includes the following default groups: admins, clients, and users. For users of the hosted Chef Infra Server

Environments.

- ▲ An environment is a way to map an organization's real-life workflow to what can be configured and managed when using Chef Infra.
- ▲ Chef environments allowing for nodes to be organized into different "groups" that define the role the node plays in the fleet. This allows for users to combine environments and versioned cookbooks to have different attributes for different nodes. For example, if testing a shopping cart, you may not want to test any changes on the live website, but instead, with a "development" set of nodes.
- ▲ Environments are defined in chef-repo/environments and saved as Ruby or JSON files.

▲ **Attributes**

- ▲ Attributes define specific values about a node and its configuration and are used by the Chef client to apply those attributes to nodes via its attribute list. The chef client can receive attributes from nodes, attribute files, recipes, environments, and roles.
- ▲ **Roles** in Chef are a logical way of grouping nodes.
- ▲ **Databag**- Chef data bags can be defined as an arbitrary collection of data which one can use with cookbooks.

Roles of chef in Devops

- ▲ Chef is for automating and managing the infrastructure.
- ▲ Chef IT automation can be done using various Chef DevOps products like Chef-server, Chef-client.
- ▲ Chef helps solve the problem by treating infrastructure as code. Rather than manually changing anything, the machine setup is described in a Chef recipe.

AWS OpsWorks



- ▲ **AWS OpsWorks** (Amazon Web Services **OpsWorks**) is a cloud computing service from Amazon Web Services (**AWS**) that manages infrastructure deployment for cloud administrators.
- ▲ The service automates deployment, configurations and operational tasks for distributed applications
- ▲ AWS OpsWorks is a configuration management service that helps you configure and operate applications in a cloud enterprise by using Puppet or Chef.
- ▲ OpsWorks lets you use Chef and Puppet to automate how servers are configured, deployed, and managed across your Amazon EC2 instances or on-premises compute environments.
- ▲ OpsWorks has three offerings, AWS Opsworks for Chef Automate, AWS OpsWorks for Puppet Enterprise, and AWS OpsWorks Stacks.

AWS Opsworks for Chef Automate

- ▲ AWS OpsWorks for Chef Automate is a fully managed configuration management service that hosts Chef Automate, [a suite of automation tools from Chef for configuration management, compliance and security, and continuous deployment.](#)
- ▲ OpsWorks also maintains your Chef server by automatically patching, updating, and backing up your server.
- ▲ **Cost and Licenses** - You are responsible for the cost of the AWS services used while running this Quick Start reference deployment. There is no additional cost for using the Quick Start.

AWS Opsworks for Puppet Enterprise



- ▲ AWS OpsWorks for Puppet Enterprise is a fully managed configuration management service that hosts Puppet Enterprise, [a set of automation tools from Puppet for infrastructure and application management](#).
- ▲ OpsWorks also maintains your Puppet master server by automatically patching, updating, and backing up your server. OpsWorks eliminates the need to operate your own configuration management systems or worry about maintaining its infrastructure.
- ▲ OpsWorks gives you access to all of the Puppet Enterprise features, which you manage through the Puppet console. It also works seamlessly with your existing Puppet code.

Benefits

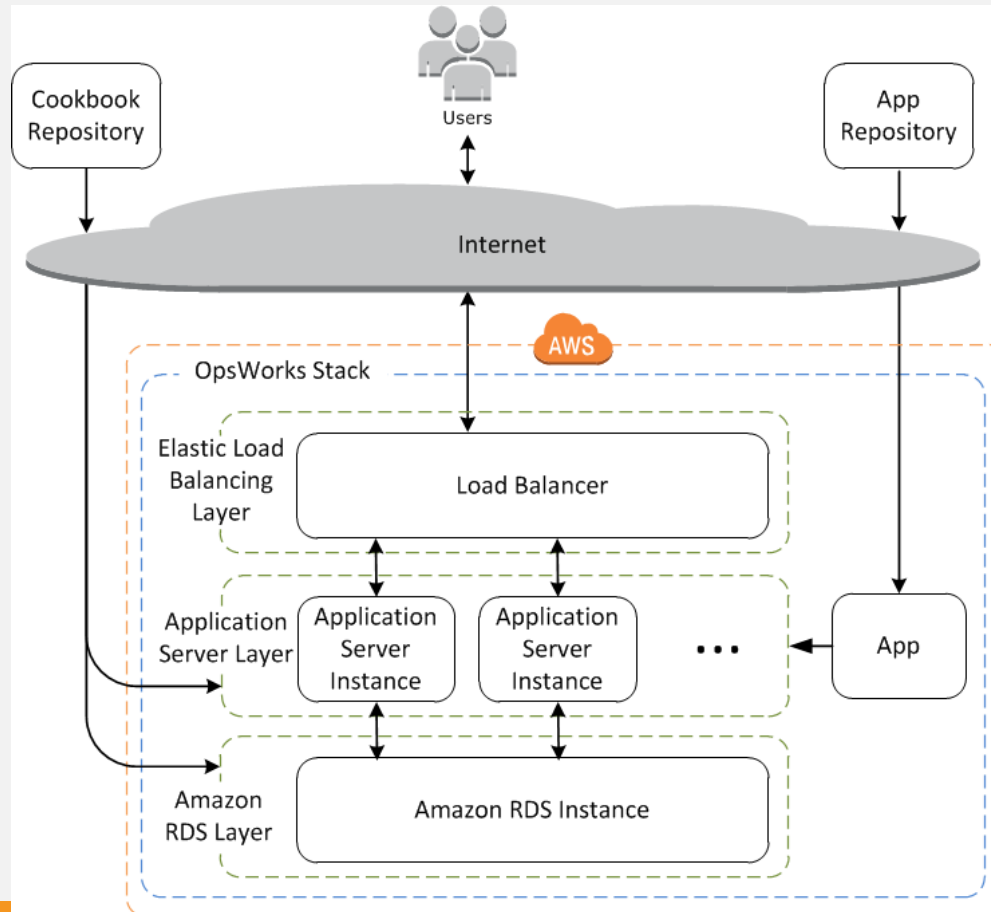
- Fully managed puppet master
- Programmable infrastructure
- Scaling made easy & SECURE
- Simple to manage hybrid environments

AWS Opsworks for AWS - Stacks

- ▲ AWS OpsWorks Stacks is an application and server management service.
- ▲ This allows you to automate tasks such as installing packages and programming languages or frameworks, configuring software, and more.
- ▲ A stack is a collection of **AWS resources** that you can manage as a single unit.
- ▲ It is basically a container for AWS resources—Amazon EC2 instances, Amazon RDS database instances, and so on—that have a common purpose and should be logically managed together.
- ▲ All the resources in a stack are defined by the stack's AWS CloudFormation template.
- ▲ With OpsWorks Stacks, you can model your application as a stack containing different layers, such as load balancing, database, and application server. Within each layer, you can provision Amazon EC2 instances, enable automatic scaling, and configure your instances with Chef recipes using Chef Solo.

▲ Layers

- ▲ You define the stack's constituents by adding one or more layers. A layer represents a set of Amazon EC2 instances that serve a particular purpose, such as serving applications or hosting a database server.
- ▲ For all stacks, AWS OpsWorks Stacks includes service layers, which represent the following AWS services.
 - ▲ Amazon Relational Database Service
 - ▲ Elastic Load Balancing
 - ▲ Amazon Elastic Container Service



- ▲ AWS CloudFormation provides a common language for you to describe and provision all the infrastructure resources in your cloud environment.
- ▲ CloudFormation allows you to use a simple text file to model and provision, in an automated and secure manner, all the resources needed for your applications across all regions and accounts.
- ▲ This file serves as the single source of truth for your cloud environment.

▲ Benefits

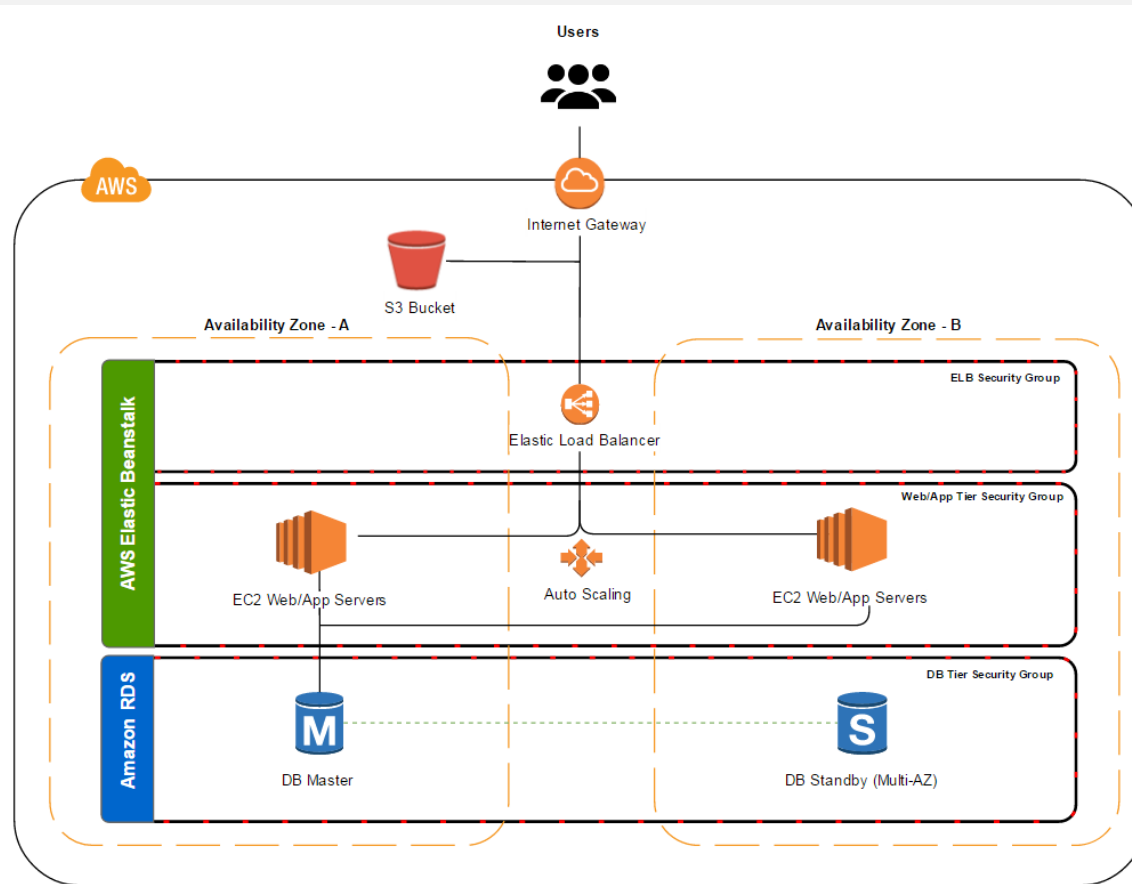
- ▲ Model it all
- ▲ Automate and deploy
- ▲ It's just code

- ▲ AWS CloudFormation allows you to model your entire infrastructure in a text file. This template becomes the single source of truth for your infrastructure.
- ▲ This helps you to standardize infrastructure components used across your organization, enabling configuration compliance and faster troubleshooting.

- ▲ AWS CloudFormation provisions your resources in a safe, repeatable manner, allowing you to build and rebuild your infrastructure and applications, without having to perform manual actions or write custom scripts.
- ▲ CloudFormation takes care of determining the right operations to perform when managing your stack, and rolls back changes automatically if errors are detected.

IT'S JUST CODE

- ▲ Codifying your infrastructure allows you to treat your infrastructure as just code. You can author it with any code editor, check it into a version control system, and review the files with team members before deploying into production.

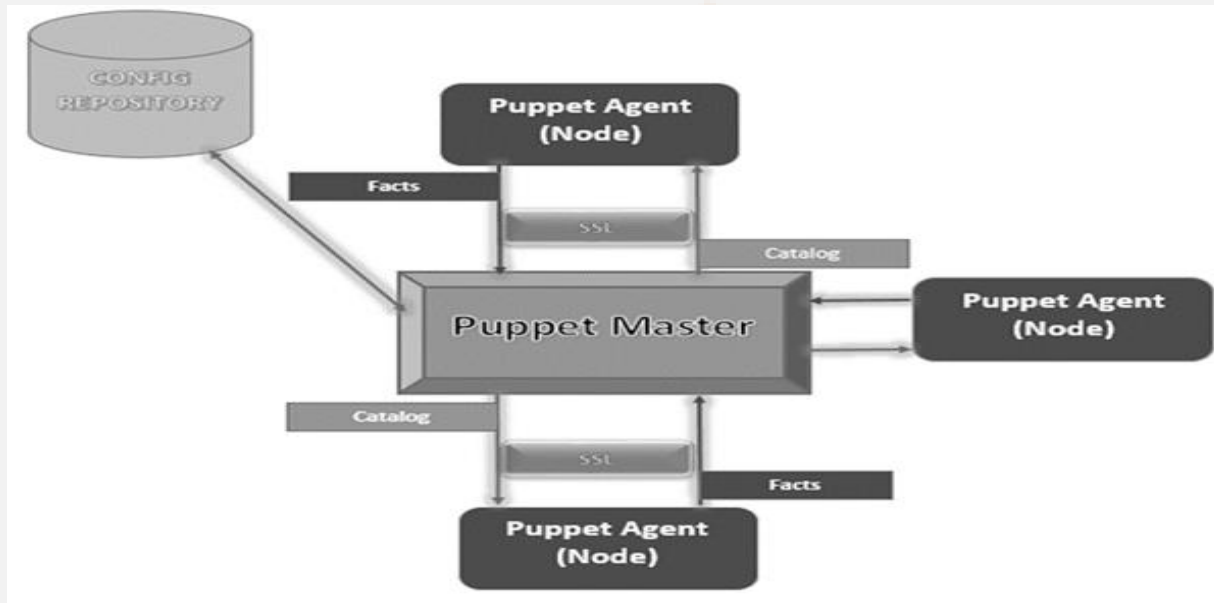


Puppet



- ▲ Puppet is a Configuration Management tool that is used for deploying, configuring and managing servers.
- ▲ Puppet performs following operations:
 - ▲ Defining distinct configurations for each and every host, and continuously checking and confirming whether the required configuration is in place and is not altered (if altered Puppet will revert back to the required configuration) on the host
 - ▲ Dynamic scaling-up and scaling-down of machines.
 - ▲ Providing control over all your configured machines, so a centralized (master-server or repo-based) change gets propagated to all, automatically.
- ▲ Puppet uses a Master Slave architecture in which the Master and Slave communicate through a secure encrypted channel with the help of SSL.
- ▲ Puppet works on the client server architecture, wherein we call the server as the Puppet master and the client as the Puppet node. This setup is achieved by installing Puppet on both the client and well as on all the server machines.

Puppet Architecture



- ▲ First of all, an agent node sends facts to the master or server and requests for a catalog.
- ▲ The master or server compiles and returns the catalog of a node with the help of some information accessed by the master.
- ▲ Then the agent applies the catalog to the node by checking every resource mentioned in the catalog. If it identifies resources that are not in their desired state, then makes the necessary adjustments to fix them. Or, it determines in no-op mode, the adjustments would be required to reconcile the catalog.
- ▲ And finally, the agent sends a report back to the master.

- ▲ **Puppet Master:** Puppet Master is the key mechanism which handles all the configuration related stuff. It applies the configuration to nodes using the Puppet agent.
- ▲ **Puppet Agents** are the actual working machines which are managed by the Puppet master. They have the Puppet agent daemon service running inside them.
- ▲ **Config Repository :** This is the repo where all nodes and server-related configurations are saved and pulled when required.

- ▲ **Facts** are the details related to the node or the master machine, which are basically used for analyzing the current status of any node. On the basis of facts, changes are done on any target machine. There are pre-defined and custom facts in Puppet.
- ▲ **Catalog** : All the manifest files or configuration which are written in Puppet are first converted to a compiled format called catalog and later those catalogs are applied on the target machine.

- ▲ Puppet Master contains the Puppet Slave's configuration details, and these are written in Puppet's native language.
- ▲ In Puppet, all the programs which are written using Ruby programming language and saved with an extension **of .pp are called manifests**. In general terms, all Puppet programs which are built with an intension of creating or managing any target host machine is called a manifest.

Manifest File Workflow

- ▲ Puppet manifest consists of the following components –
- ▲ **Files** (these are plain files where Puppet has nothing to do with them, just to pick them up and place them in the target location)
- ▲ **Resources** - Resources are the fundamental building blocks used to model system state in Puppet. They describe the desired end state of unique elements managed by Puppet on the system. Everything that Puppet manages is expressed as a resource.
- ▲ **Templates** (these can be used to construct configuration files on the node).
- ▲ **Nodes** (all the definition related to a client node is defined here)
- ▲ **Classes - Puppet classes** are defined as a collection of resources, which are grouped together in order to get a target node or machine in a desired state. These **classes** are defined inside **Puppet** manifest files which is located inside **Puppet** modules.

- ▲ Templates are documents that combine code, data, and literal text to produce a final rendered output. The goal of a template is to manage a complicated piece of text with simple inputs.
- ▲ In Puppet, you'll usually use templates to manage the content of configuration files (via the content attribute of the file resource type).

- ▲ Templates are written in a templating language, which is specialized for generating text from data.
- ▲ Puppet supports two templating languages:
 - **Embedded Puppet** (EPP) uses Puppet expressions in special tags. It's easy for any Puppet user to read, but only works with newer Puppet versions. (≥ 4.0 , or late 3.x versions with future parser enabled.)
 - **Embedded Ruby** (ERB) uses Ruby code in tags. You need to know a small bit of Ruby to read it, but it works with all Puppet versions.

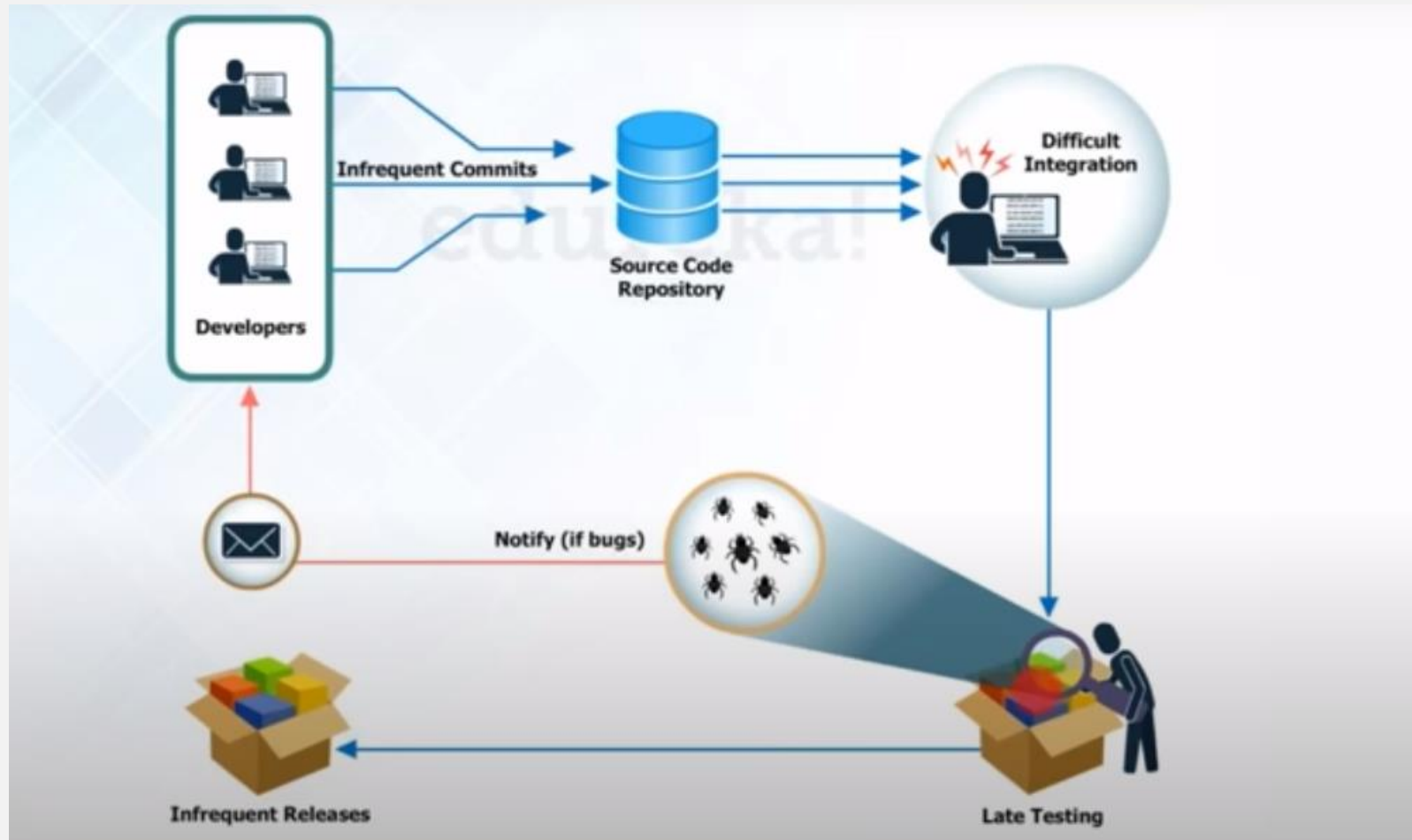
- ▲ the Puppet Forge is a repository of modules written both by Puppet and by the Puppet user community. These modules solve a wide variety of problems, and using them can save you time and effort.
- ▲ The puppet module tool provides a command line interface for managing modules from the Forge.





JENKINS

Process Before Continuous Integration

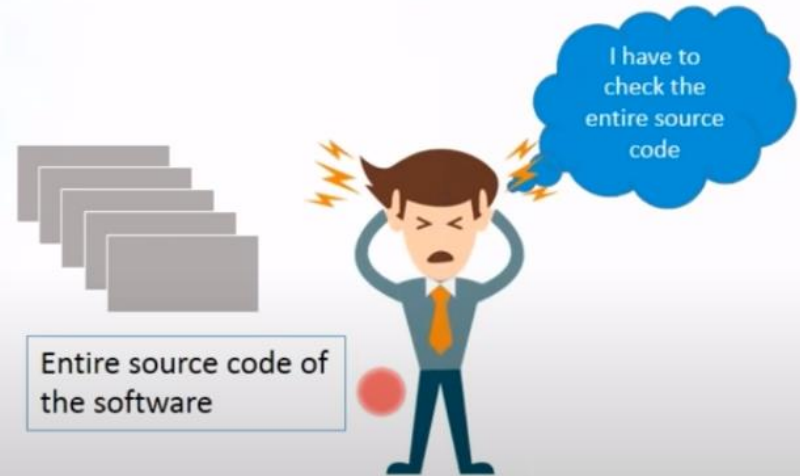


Process Before Continuous Integration

Developers have to wait till the complete software is developed for the test results.



If the test fails then locating and fixing bugs is very difficult. Developers have to check the entire source code of the software.



Software delivery process was slow



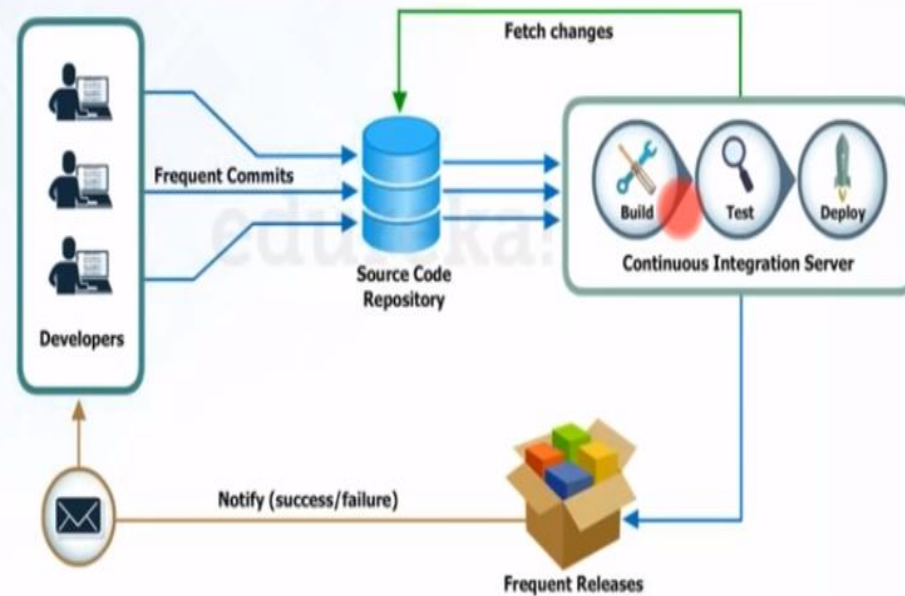
Continuous feedback pertaining to things like coding or architectural issues, build failures, test status etc. was not present

The Feedback loop
Build, Measure and
Learn



Continuous Integration to Rescue

- ❑ Since after every commit to the source code an auto build is triggered and then it is automatically deployed on the test server
- ❑ If the test results shows that there is a bug in the code then the developers only have to check the last commit made to the source code
- ❑ This also increases the frequency of new software releases
- ❑ The concerned teams are always provided with the relevant feedback



Before Continuous Integration

The entire source code was built and then tested.

Developers have to wait for test results

No Feedback

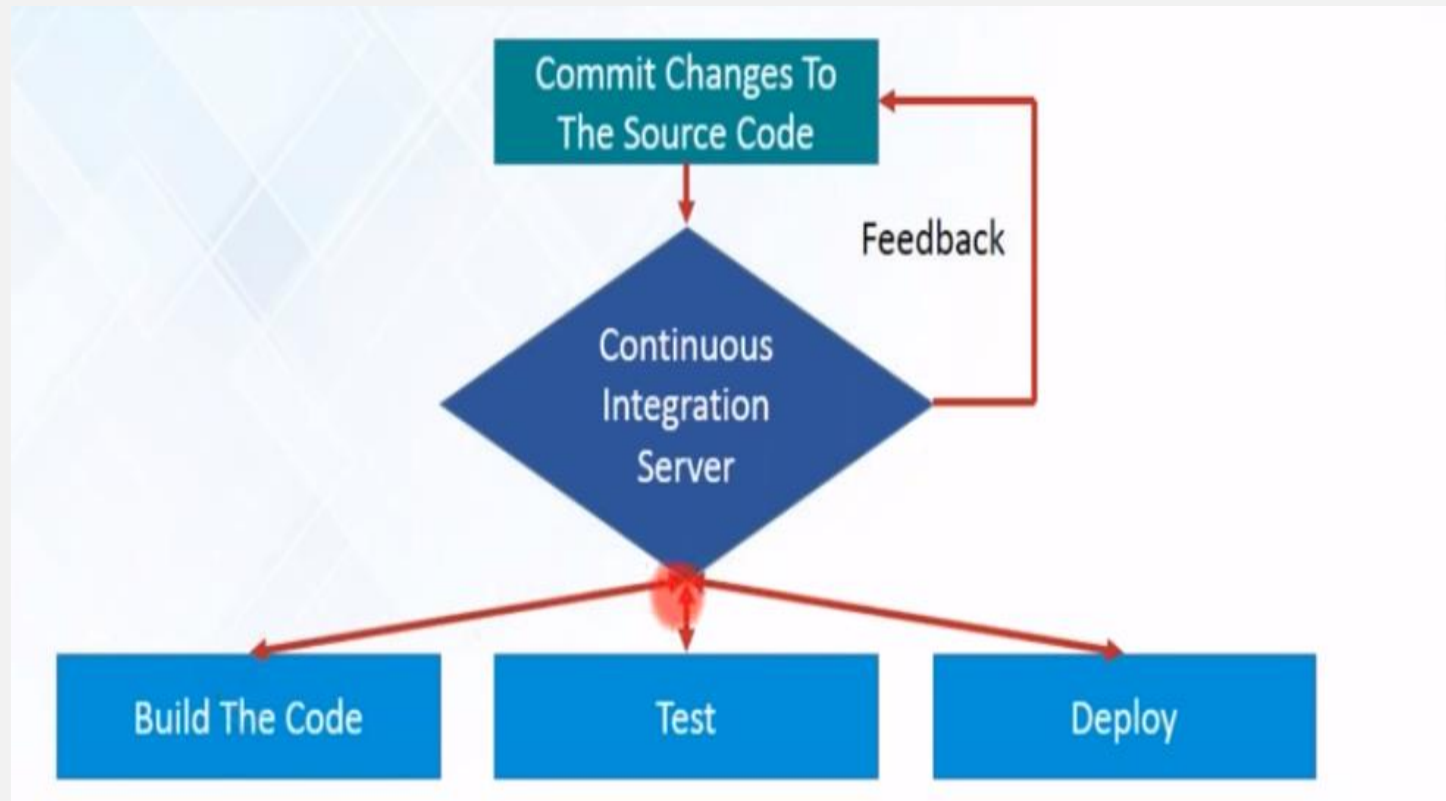
After Continuous Integration

Every commit made in the source code is built and tested.

Developers know the test result of every commit made in the source code on the run

Feedback is present

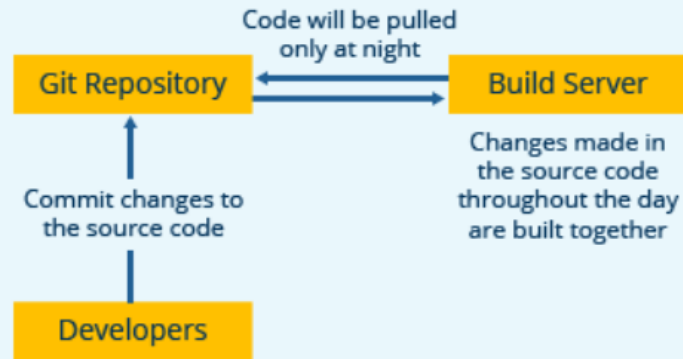
- ▲ Continuous Integration is a development practice where developers integrate code into a shared repository frequently where each integration is verified by an automated build and automated tests and run.
- ▲ It is the most important part of DevOps that is used to integrate various DevOps stages



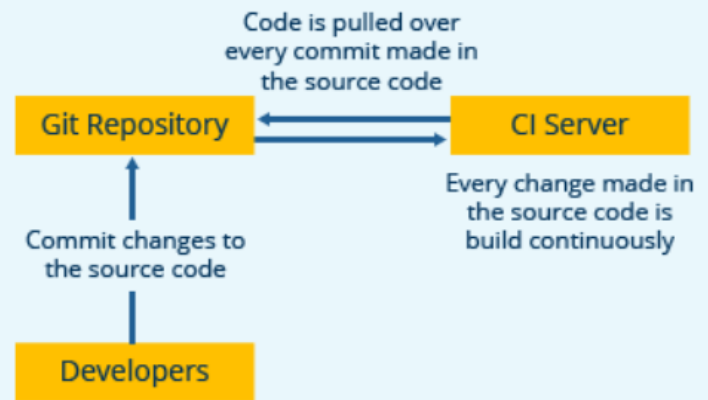
Continuous Integration Case Study-Nokia



Nightly build



Continuous Integration



- ▲ Let us imagine a scenario where the complete source code of the application was built and then deployed on test server for testing. It sounds like a perfect way to develop software, but, this process has many flaws.
 - ▲ Developers have to wait until the complete software is developed for the test results.
 - ▲ There is a high possibility that the test results might show multiple bugs. It was tough for developers to locate those bugs because they have to check the entire source code of the application.
 - ▲ It slows the software delivery process.
 - ▲ Continuous feedback pertaining to things like coding or architectural issues, build failures, test status and file release uploads was missing due to which the quality of software can go down.
 - ▲ The whole process was manual which increases the risk of frequent failure.



Continuous Integration With Jenkins



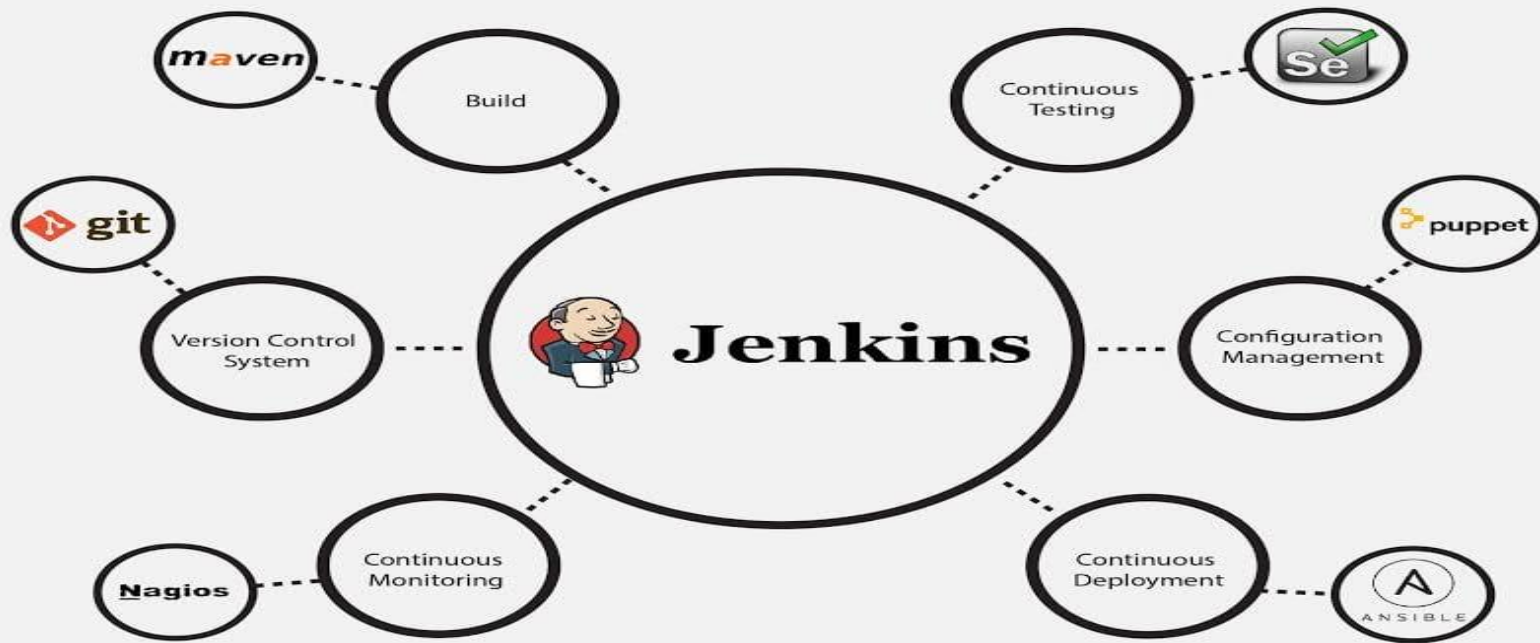
- ▲ The above diagram is depicting the following functions:
 - ▲ First, a developer commits the code to the source code repository. Meanwhile, the Jenkins server checks the repository at regular intervals for changes.
 - ▲ Soon after a commit occurs, the Jenkins server detects the changes that have occurred in the source code repository. Jenkins will pull those changes and will start preparing a new build.
 - ▲ If the build fails, then the concerned team will be notified.
 - ▲ If built is successful, then Jenkins deploys the built in the test server.
 - ▲ After testing, Jenkins generates a feedback and then notifies the developers about the build and test results.
 - ▲ It will continue to check the source code repository for changes made in the source code and the whole process keeps on repeating.

What is Jenkins?

- ▲ Jenkins is an open source automation tool written in Java with plugins built for Continuous Integration purpose.
- ▲ Jenkins is used to build and test your software projects continuously making it easier for developers to integrate changes to the project, and making it easier for users to obtain a fresh build.
- ▲ It also allows you to continuously deliver your software by integrating with a large number of testing and deployment technologies.
- ▲ Jenkins achieves Continuous Integration with the help of plugins. Plugins allows the integration of Various DevOps stages. If you want to integrate a particular tool, you need to install the plugins for that tool. For example: Git, Maven 2 project, Amazon EC2, HTML publisher etc.

- ▲ Advantages of Jenkins include:
- ▲ It is an open source tool with great community support.
- ▲ It is easy to install.
- ▲ It has 1000+ plugins to ease your work. If a plugin does not exist, you can code it and share with the community.
- ▲ It is free of cost.
- ▲ It is built with Java and hence, it is portable to all the major platforms.

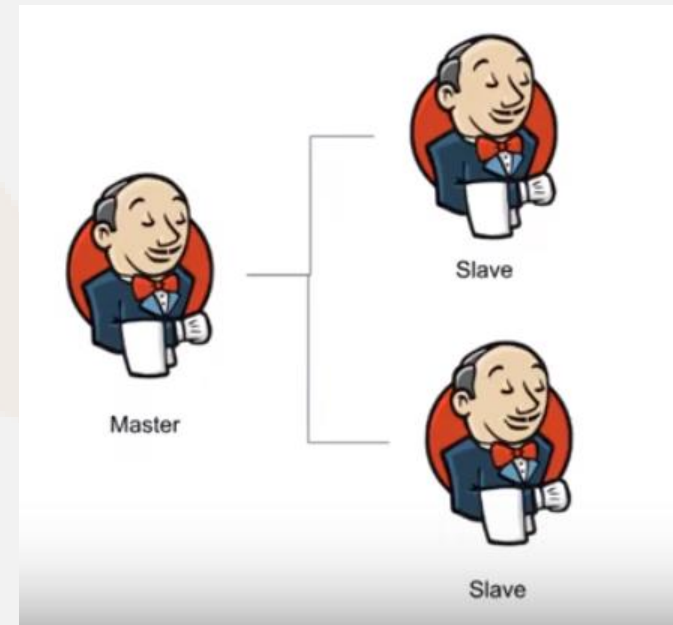
Before Jenkins	After Jenkins
The entire source code was built and then tested. Locating and fixing bugs in the event of build and test failure was difficult and time consuming, which in turn slows the software delivery process.	Every commit made in the source code is built and tested. So, instead of checking the entire source code developers only need to focus on a particular commit. This leads to frequent new software releases.
Developers have to wait for test results	Developers know the test result of every commit made in the source code on the run.
The whole process is manual	You only need to commit changes to the source code and Jenkins will automate the rest of the process for you.



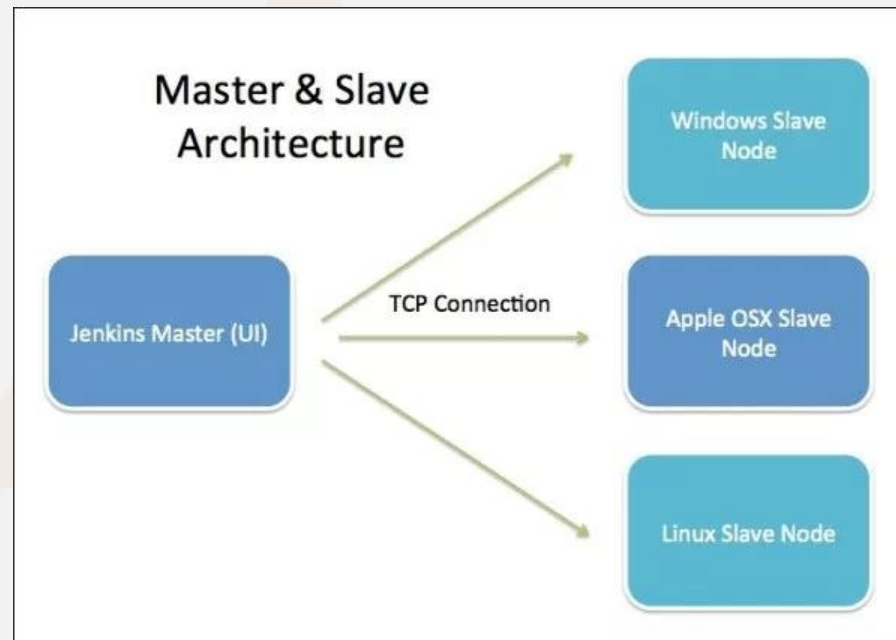
What is a build in software development?

- ▲ The process of **building software** is usually managed by a **build** tool. **Builds** are created when a certain point in **development** has been reached or the code has been deemed ready for implementation, either for testing or outright release. A **build** is also known as a **software build** or code **build**.

- Single Jenkins server was not enough to meet certain requirements like:
- Sometimes you might need several different environments to test your builds. This cannot be done by a single Jenkins server.
- If larger and heavier projects get built on a regular basis then a single Jenkins server cannot simply handle the entire load.
- To address the above stated needs, Jenkins distributed architecture was introduced.



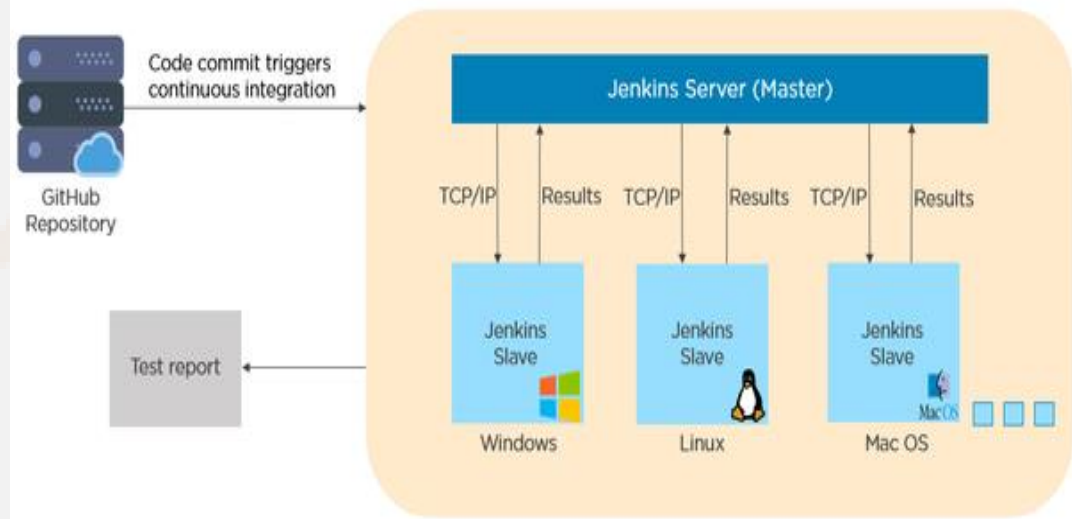
- ▲ Jenkins uses a Master-Slave architecture to manage distributed builds. In this architecture, Master and Slave communicate through TCP/IP protocol.



- ▲ Scheduling build jobs.
- ▲ Dispatching builds to the slaves for the actual execution.
- ▲ Monitor the slaves (possibly taking them online and offline as required).
- ▲ Recording and presenting the build results.
- ▲ A Master instance of Jenkins can also execute build jobs directly.

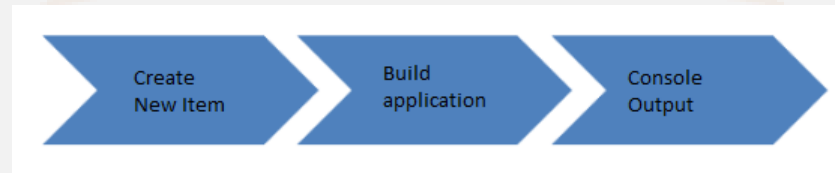
- ▲ A Slave is a Java executable that runs on a remote machine.
- ▲ It hears requests from the Jenkins Master instance.
- ▲ Slaves can run on a variety of operating systems.
- ▲ The job of a Slave is to do as they are told to, which involves executing build jobs dispatched by the Master.
- ▲ You can configure a project to always run on a particular Slave machine, or a particular type of Slave machine, or simply let Jenkins pick the next available Slave.

- ▶ Jenkins checks the Git repository at periodic intervals for any changes made in the source code.
- ▶ Each build requires a different testing environment which is not possible for a single Jenkins server. In order to perform testing in different environments Jenkins uses various Slaves as shown in the diagram.
- ▶ Jenkins Master requests these Slaves to perform testing and to generate test reports.



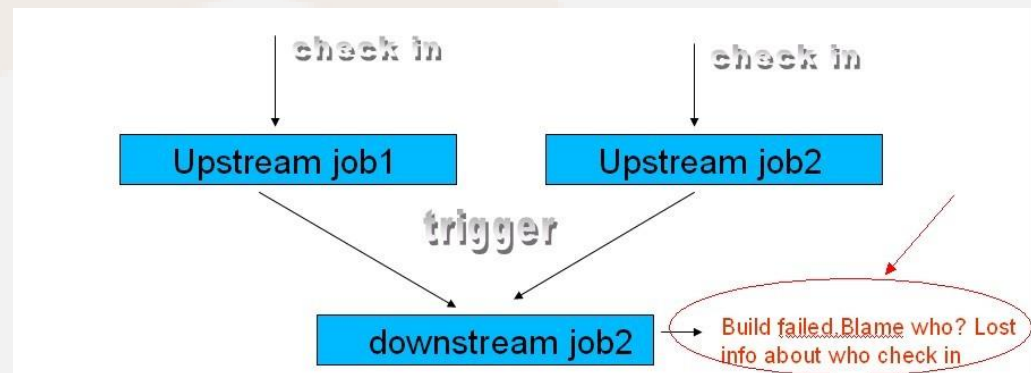
What is job in Jenkins?

- ▶ You are requesting Jenkins to perform any task on perform timeframe or immediate basis.
- ▶ **What is a Jenkins freestyle project?**
- ▶ A Jenkins project is a repeatable build job which contains steps and post-build actions. The types of actions you can perform in a build step or post-build action are quite limited.
- ▶ There are many standard plugins available within a Jenkins freestyle project to help you overcome this problem. They allow you to configure build triggers and offers project-based security for your Jenkins project.



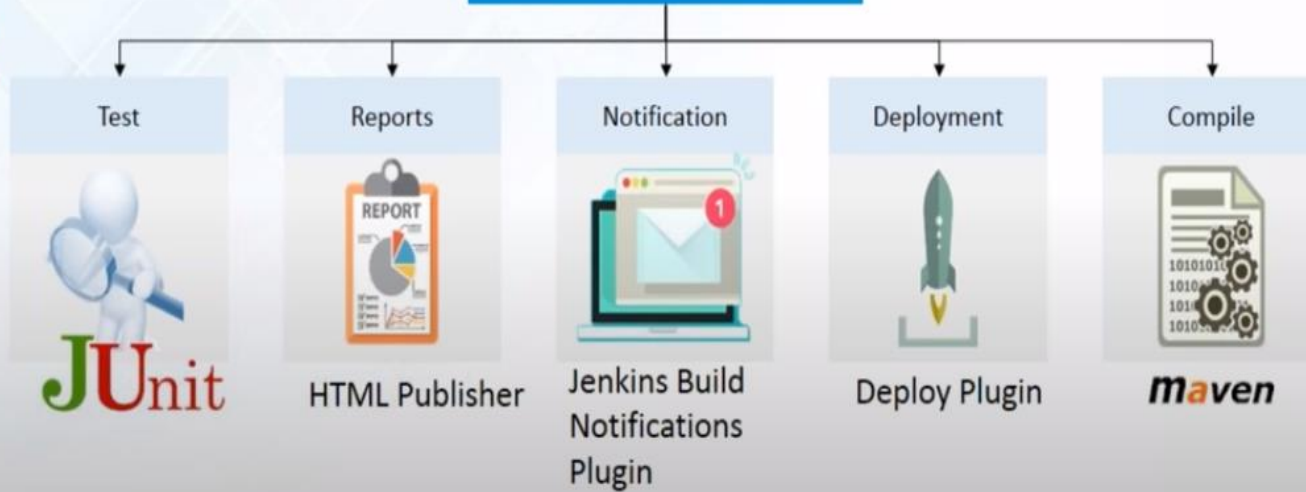
Upstream Job vs Downstream Job

- ▲ The upstream job is the one that is triggered before the actual job is triggered. The downstream job is the one that is triggered after the actual job is triggered.
- ▲ We can configure the actual job not to be triggered if the upstream job is failed.
- ▲ In the same way, we can configure the downstream job not to be triggered if the actual job is failed.
- ▲ **Order of job triggers:**
- ▲ Upstream Job -> Actual Job -> Downstream Job

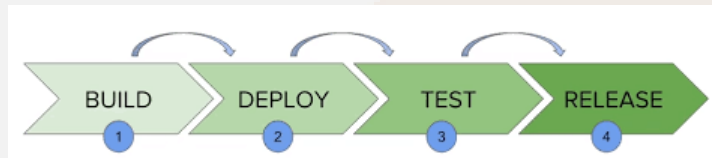


Jenkins supports plugins, which allow Jenkins to be extended to meet specific needs of individual projects

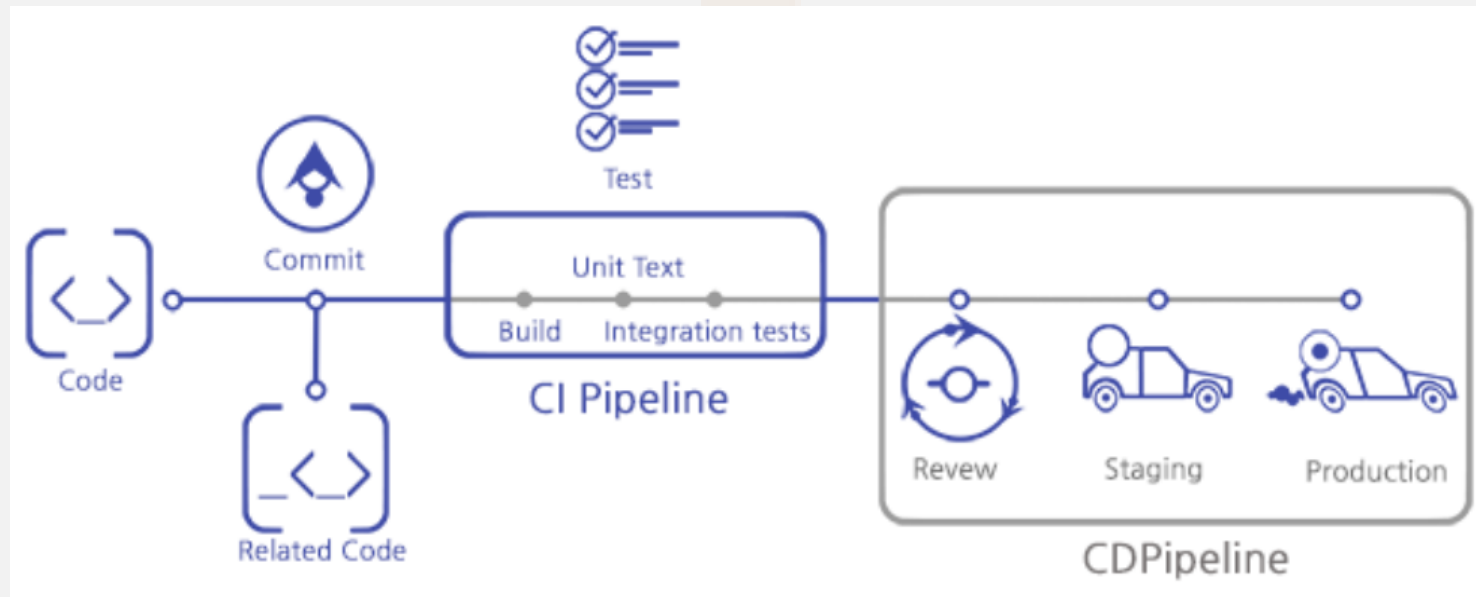
Plugin Categorization



- ▲ The **pipeline** is a set of instructions given in the form of code for continuous **delivery** and consists of instructions needed for the entire build process.
- ▲ With **pipeline**, you can build, test, and deliver the application. Node. The machine on which **Jenkins** runs is called a node.



CI pipeline and CD pipeline



▲ Pipeline

- ▲ This is a user defined block which contains all the processes such as build, test, deploy, etc. It is a collection of all the stages in a Jenkins file.

```
pipeline {  
}
```

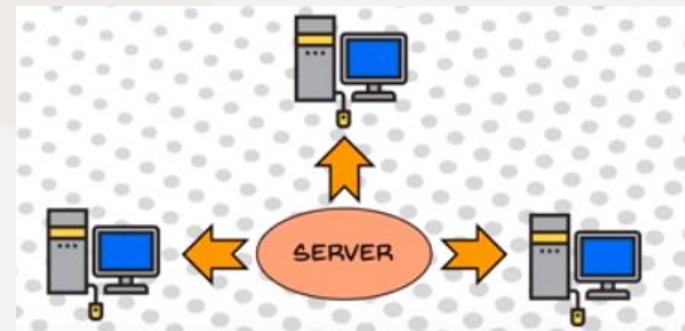
▲ Node

- ▲ A node is a machine that executes an entire workflow. It is a key part of the scripted pipeline syntax.

```
node {  
}
```

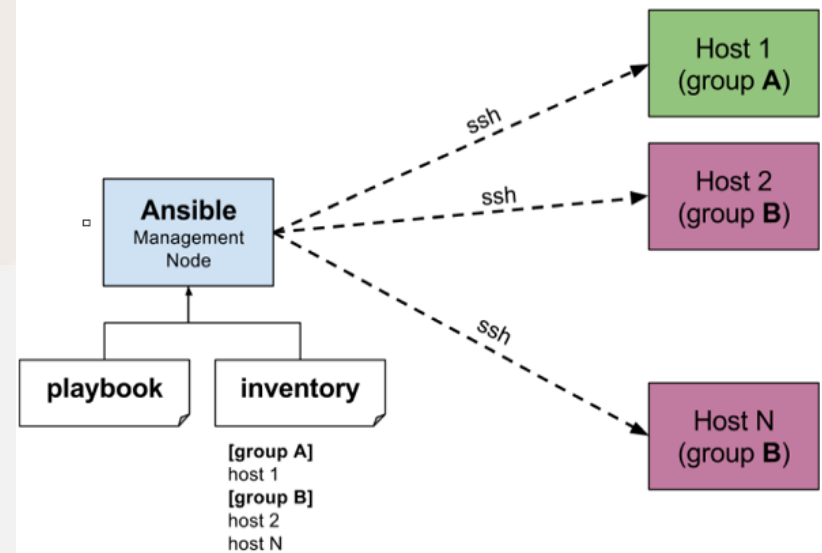
What is Ansible?

- ▶ Ansible is an open-source software provisioning, configuration management, and application-deployment .
- ▶ Ansible is easy to deploy because it does not use any agents or custom security infrastructure.
- ▶ Ansible uses playbook to describe automation jobs, and playbook uses very simple language i.e. **YAML**
- ▶ Its comes under push based configuration.



How Ansible Works?

- ▶ **Ansible works** by connecting to your nodes and pushing out small programs, called "**Ansible** modules" to them. **Ansible** then executes these modules (over SSH by default), and removes them when finished. Your library of modules can reside on any machine, and there are no servers, daemons, or databases required.



Understanding YAML

- ▲ “Yet another mark-up language”
- ▲ YAML uses simple key-value pair to represent the data. The dictionary is represented in key: value pair.
- ▲ **Note** – There should be space between : and value.

```
---  
james:  
  name: james john  
  rollNo: 34  
  div: B  
  sex: male  
...
```

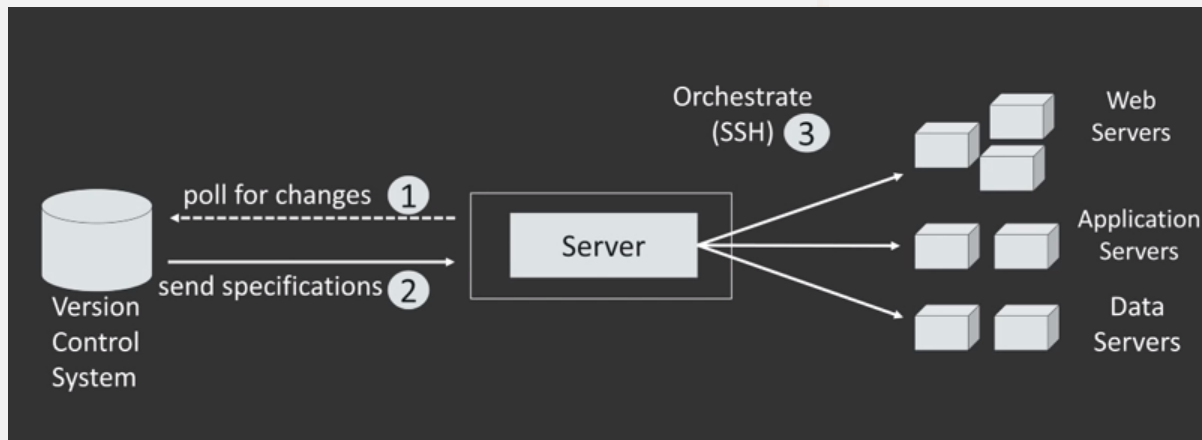
- ▲ The **Ansible inventory** file defines the hosts and groups of hosts upon which commands, modules, and tasks in a playbook operate. The file can be in one of many formats depending on your **Ansible** environment and plugins.
- ▲ The **inventory** file can list individual hosts or user-defined groups of hosts.

- ▲ **Ansible** ships with a number of **modules** (called the '**module** library') that can be executed directly on remote hosts or through Playbooks. Users can also write their own **modules**. These **modules** can control system resources, like services, packages, or files (anything really), or handle executing system commands.

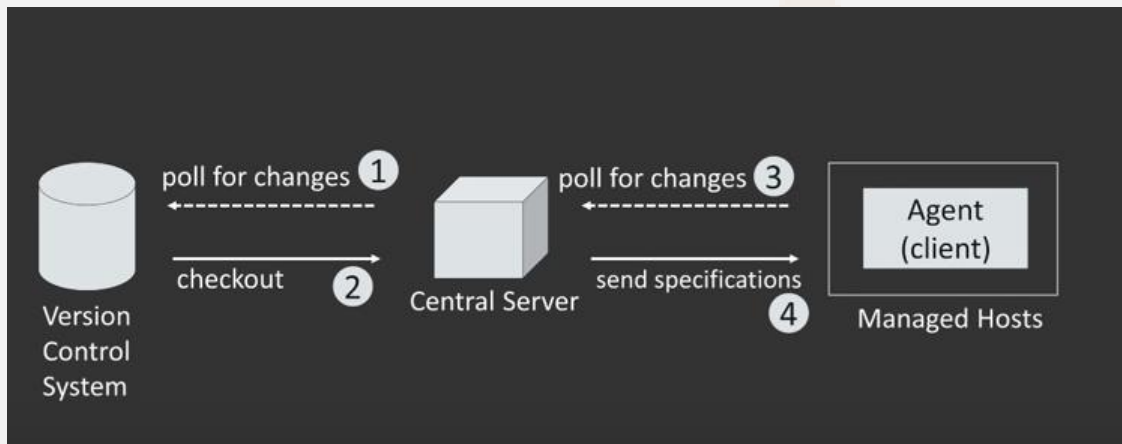
Features of ansible

- ▲ Its agentless
 - ▲ Its build on python
 - ▲ SSH
 - ▲ Push based
- 

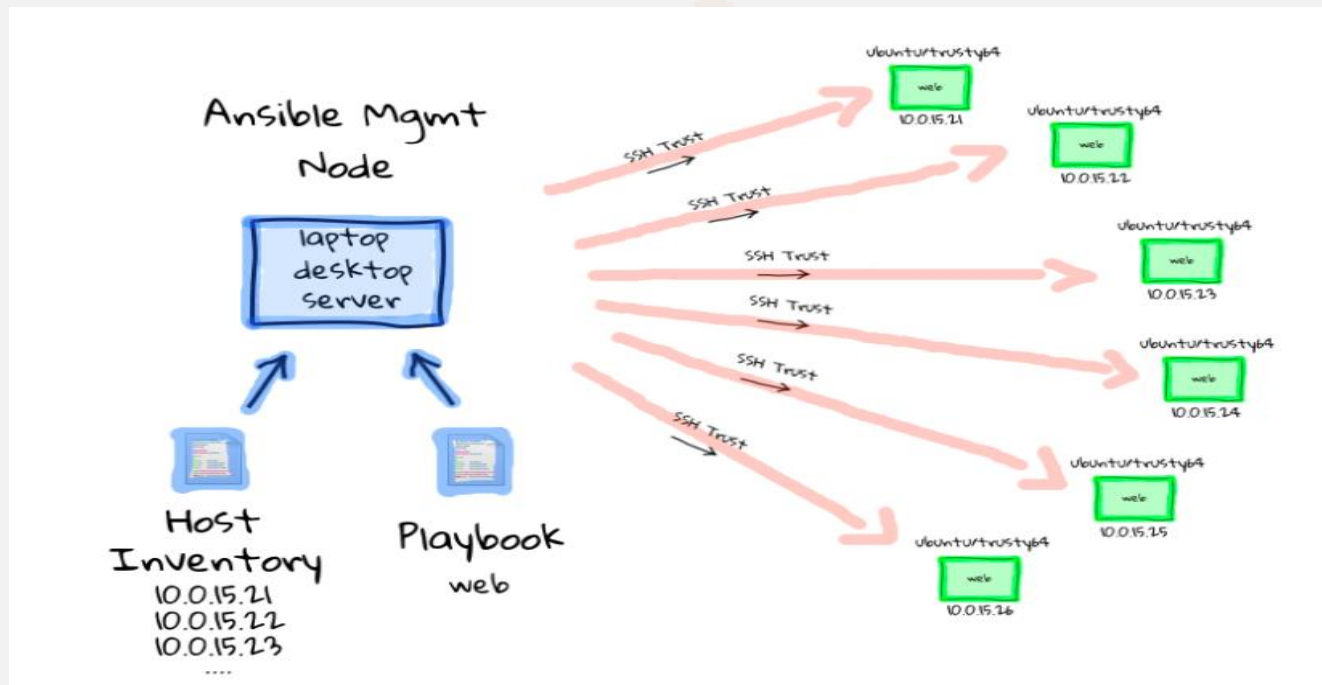
Push Based vs Pull Based

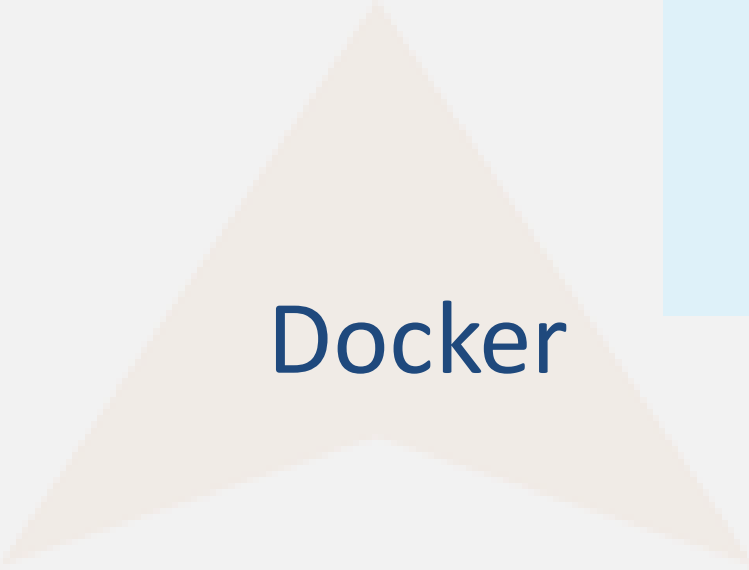


Pull



Ansible architecture





Docker



An application works in developer's laptop but not in testing or production. This is due to difference in computing environment between Dev, Test and Prod.



- ▲ Docker is a container management service. The keywords of Docker are develop, ship and run anywhere.
- ▲ Docker is for developers to easily develop applications, ship them into containers which can then be deployed anywhere.
- ▲ The initial release of Docker was in March 2013
- ▲ Docker is a client-server application that can be installed on Linux, Windows, and MacOS and that allows you to run Docker container.
- ▲ Containers are lightweight environments containing everything needed to run a specific application or part of an application. Multiple different containers can be run on one machine, so long as it has the Docker software installed.

Features of Docker

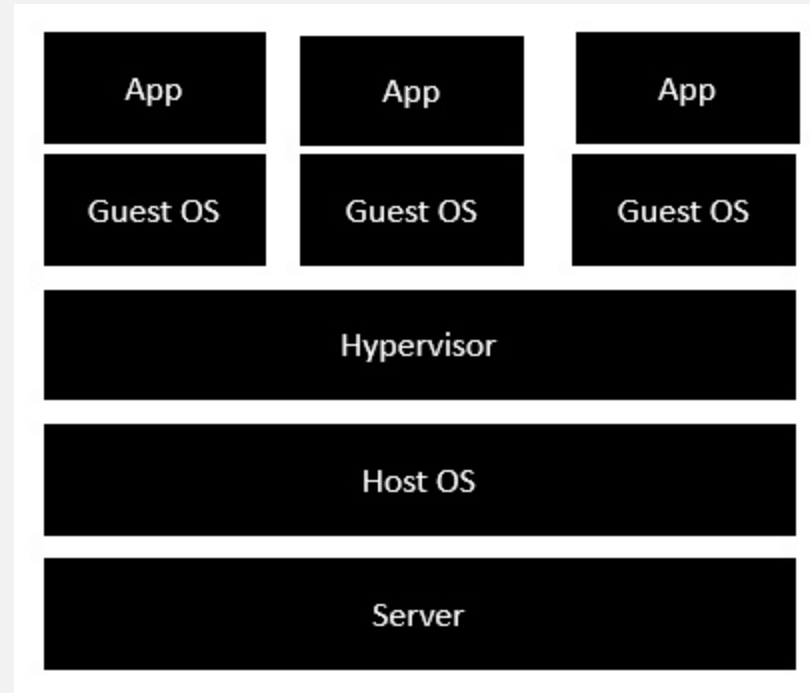
- ▶ Docker has the ability to reduce the size of development by providing a smaller footprint of the operating system via containers.
- ▶ With containers, it becomes easier for teams across different units, such as development, QA and Operations to work seamlessly across applications.
- ▶ You can deploy Docker containers anywhere, on any physical and virtual machines and even on the cloud.
- ▶ Since Docker containers are pretty lightweight, they are very easily scalable.

What can I use Docker for?

- ▲ Fast, consistent delivery of your applications
- ▲ Containers are great for continuous integration and continuous delivery (CI/CD) workflows.
- ▲ **Consider the following example scenario:**
- ▲ Your developers write code locally and share their work with their colleagues using Docker containers.
- ▲ They use Docker to push their applications into a test environment and execute automated and manual tests.
- ▲ When developers find bugs, they can fix them in the development environment and redeploy them to the test environment for testing and validation.
- ▲ When testing is complete, getting the fix to the customer is as simple as pushing the updated image to the production environment.

virtualization vs Containerization

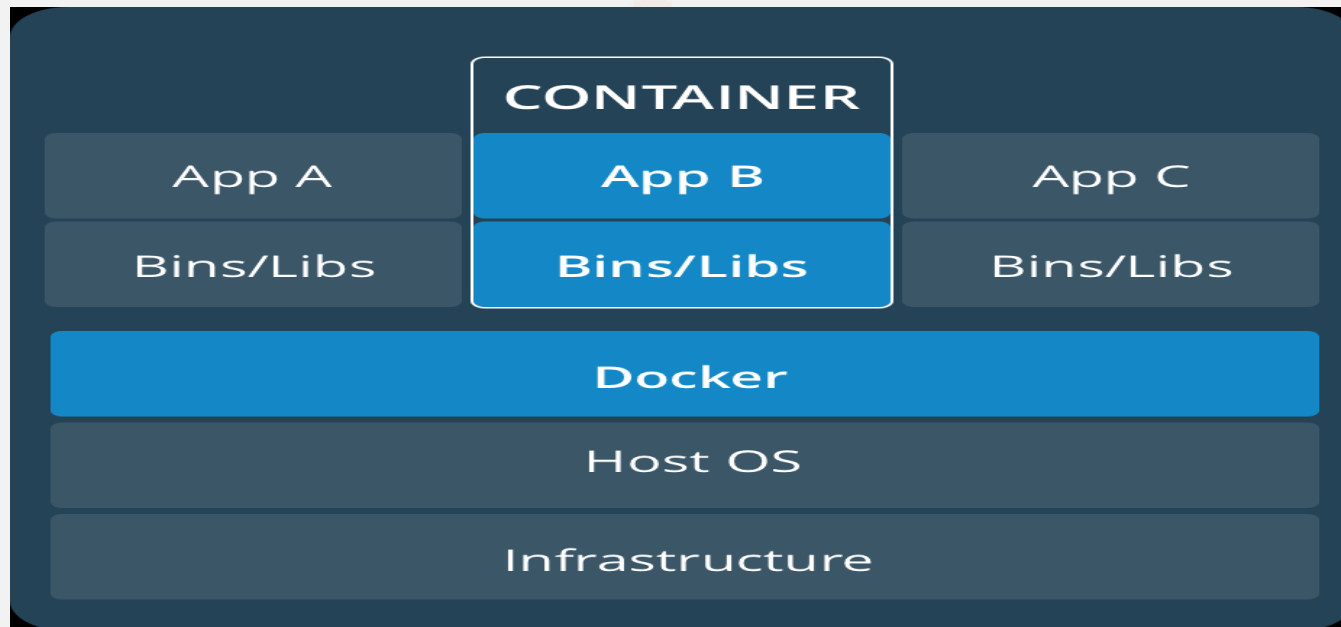
Virtualization.



- ▲ The **server** is the physical server that is used to host multiple virtual machines.
- ▲ The **Host OS** is the base machine such as Linux or Windows.
- ▲ The **Hypervisor** is either VMWare or Windows Hyper V that is used to host virtual machines.
- ▲ You would then install multiple operating systems as virtual machines on top of the existing hypervisor as Guest OS.
- ▲ You would then host your applications on top of each Guest OS.

Containerization

- ▶ Docker provides the ability to package and run an application in a loosely isolated environment called a container. The isolation and security allow you to run many containers simultaneously on a given host. Containers are lightweight and contain everything needed to run the application, so you do not need to rely on what is currently installed on the host.

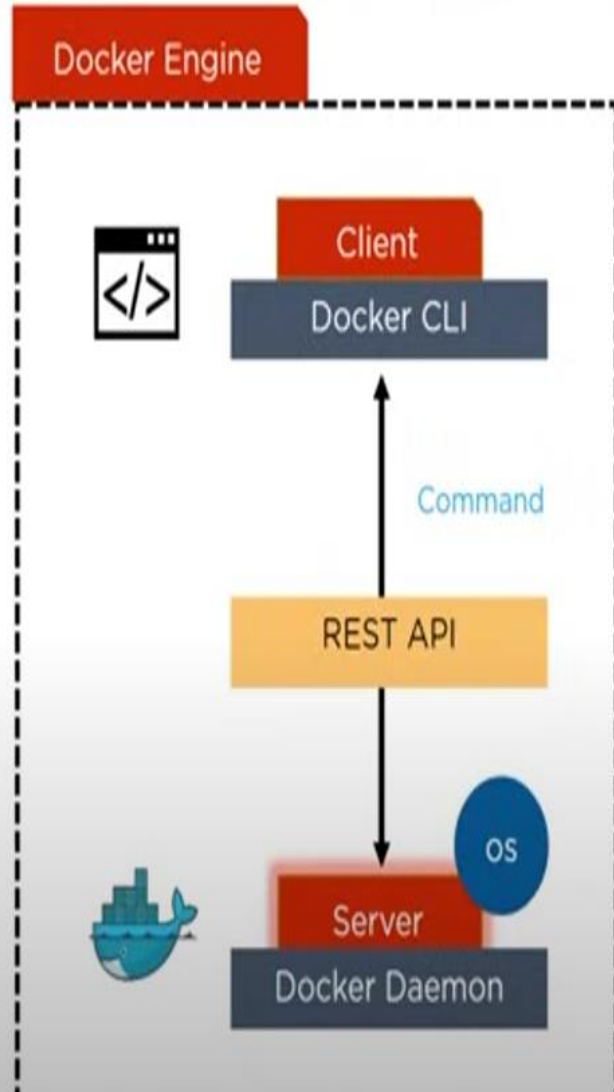


- ▶ The server is the physical server that is used to host multiple virtual machines. So this layer remains the same.
- ▶ The Host OS is the base machine such as Linux or Windows. So this layer remains the same.
- ▶ Now comes the new generation which is the Docker engine. This is used to run the operating system which earlier used to be virtual machines as Docker containers.
- ▶ All of the Apps now run as Docker containers.
- ▶ The clear advantage in this architecture is that you don't need to have extra hardware for Guest OS. Everything works as Docker containers.
- ▶ Applications can share the data easily

- ▲ Containerization is increasingly popular because containers are:
- ▲ **Flexible:** Even the most complex applications can be containerized.
- ▲ **Lightweight:** Containers leverage and share the host kernel, making them much more efficient in terms of system resources than virtual machines.
- ▲ **Portable:** You can build locally, deploy to the cloud, and run anywhere.
- ▲ **Loosely coupled:** Containers are highly self sufficient and encapsulated, allowing you to replace or upgrade one without disrupting others.
- ▲ **Scalable:** You can increase and automatically distribute container replicas across a datacenter.
- ▲ **Secure:** Containers apply aggressive constraints and isolations to processes without any configuration required on the part of the user.

Criteria	 Virtual Machine	Docker 
OS support	Occupies a lot of memory space	Docker Containers occupy less space
Boot-up time	Long boot-up time	Short boot-up time
Performance	Running multiple virtual machines leads to unstable performance	Containers have a better performance as they are hosted in a single Docker engine
Scaling	Difficult to scale up	Easy to scale up
Efficiency	Low efficiency	High efficiency
Portability	Compatibility issues while porting across different platforms	Easily portable across different platforms
Space allocation	Data volumes cannot be shared	Data volumes can be shared and reused among multiple containers

How does Docker work?



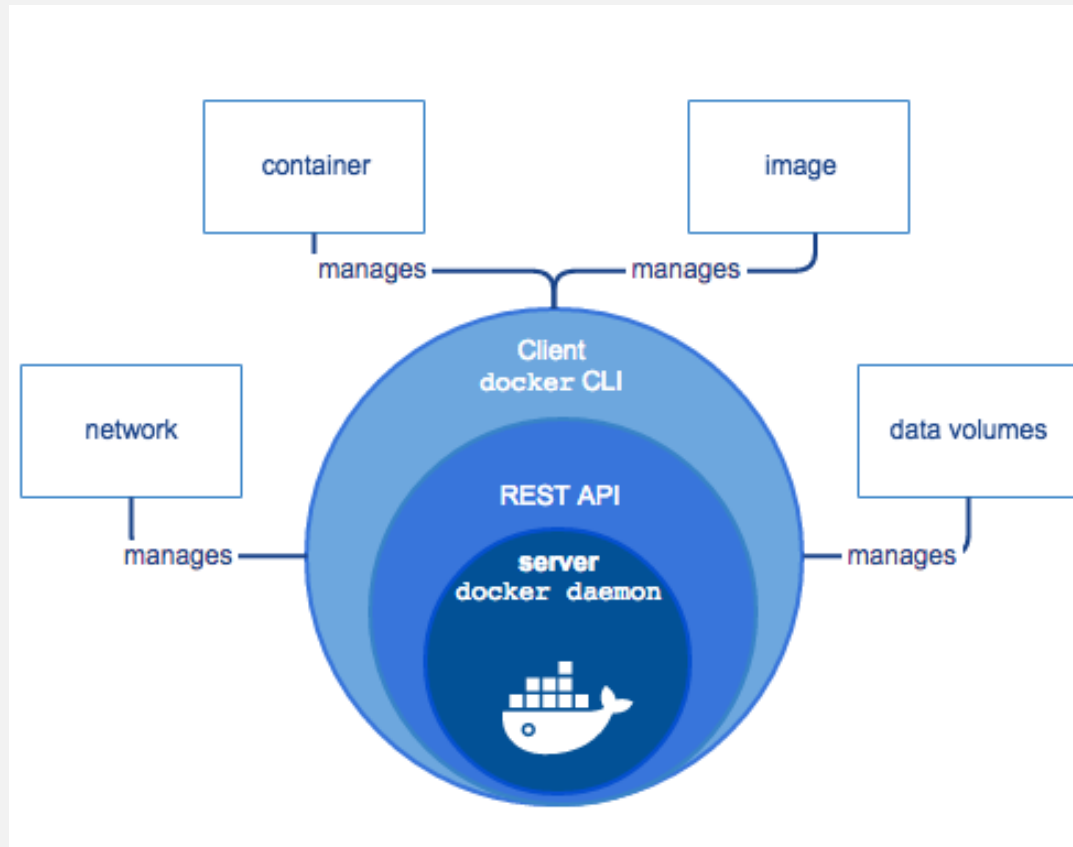
- Docker Engine or Docker is the base engine installed on your host machine to build and run containers using Docker components and services
- It uses a client-server architecture
- Docker Client and Server communicate using Rest API

What happens here?

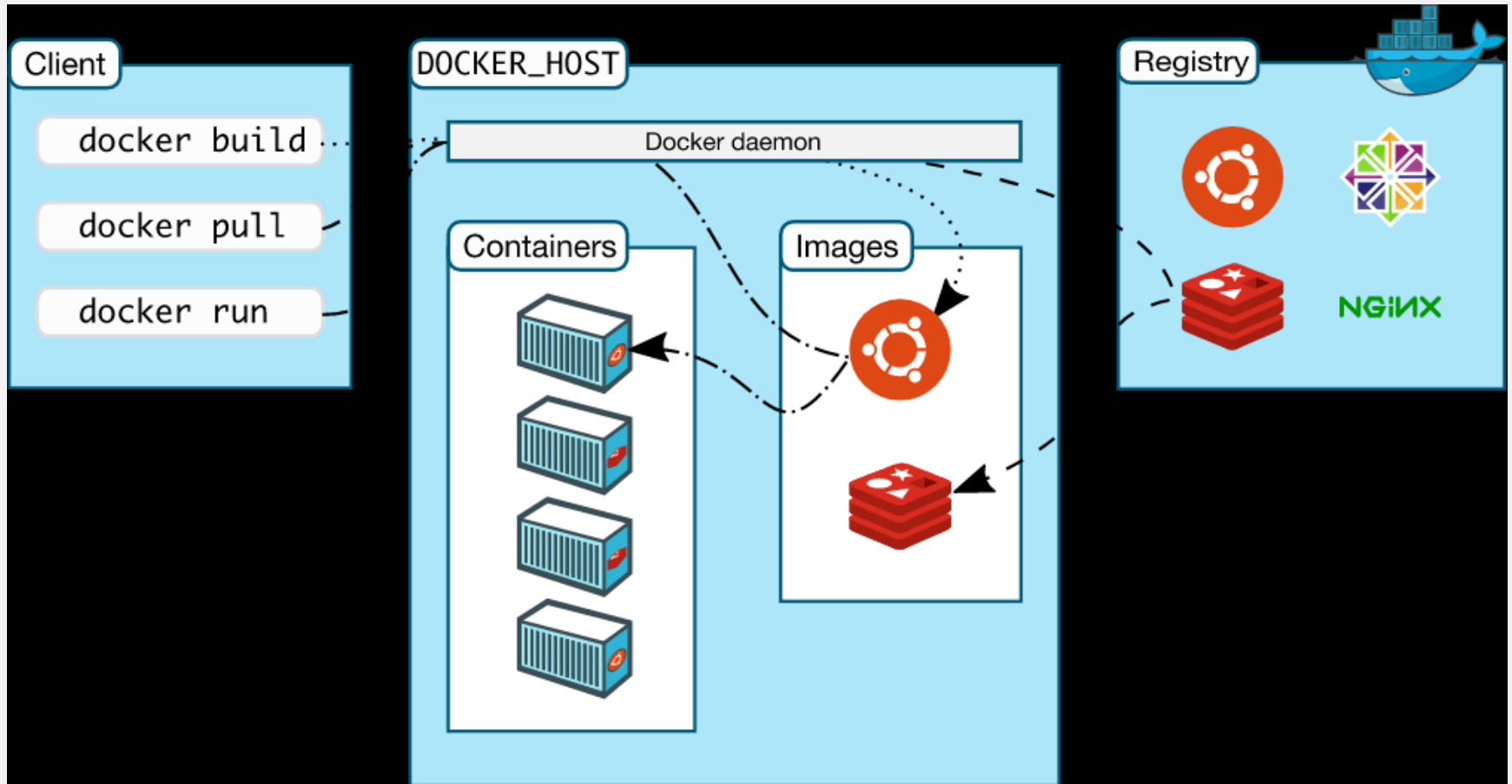
- Docker Client is a service which runs a command. The command is translated using REST API and is sent to the Docker Daemon (server)
- Then, Docker Daemon checks the client request and interacts with the operating system in order to create or manage containers

▲ Docker Engine

- ▲ Docker Engine is a client-server application with these major components:
 - ▲ A server which is a type of long-running program called a **daemon** process (the dockerd command).
 - ▲ A **REST API** which specifies interfaces that programs can use to talk to the daemon and instruct it what to do.
 - ▲ A command line interface (CLI) client (the docker command).
- ▲ The CLI uses the Docker REST API to control or interact with the Docker daemon through scripting or direct CLI commands. Many other Docker applications use the underlying API and CLI.
- ▲ The **daemon** creates and manages Docker *objects*, such as images, containers, networks, and volumes.



- ▶ Docker uses a client-server architecture. The Docker client talks to the Docker daemon, which does the heavy lifting of building, running, and distributing your Docker containers.
- ▶ The Docker client and daemon can run on the same system, or you can connect a Docker client to a remote Docker daemon. The Docker client and daemon communicate using a REST API, over UNIX sockets or a network interface.



▲ 1. The Docker client

- ▲ The Docker client (docker) is the primary way that many Docker users interact with Docker.
- ▲ When you use commands such as **docker run**, the client sends these commands to dockerd, which carries them out. The docker command uses the Docker API. The Docker client can communicate with more than one daemon.

▲ 2. Docker Host

- ▲ A Docker host is responsible for running the Docker daemon. Docker Daemon entertains API requests, including docker build and docker run amongst others. It also manages images, networks, containers, and other Docker objects. Daemons can communicate with each other to manage different Docker services.

▲ Docker registries

- ▲ A Docker registry stores Docker images. **Docker Hub** is a public registry that anyone can use, and Docker is configured to look for images on Docker Hub by default.
- ▲ You can even run your own private registry. If you use Docker Datacenter (DDC), it includes Docker Trusted Registry (DTR).
- ▲ When you use the `docker pull` or `docker run` commands, the required images are pulled from your configured registry.

▲ Docker Images

- ▲ An *image* is a read-only template with instructions for creating a Docker container.
- ▲ For example, you may build an image which is based on the ubuntu image, but installs the Apache web server and your application, as well as the configuration details needed to make your application run.
- ▲ To build your own image, you create a **Dockerfile** with a simple syntax for defining the steps needed to create the image and run it. Each instruction in a Dockerfile creates a layer in the image.
- ▲ When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt. This is part of what makes images so lightweight, small, and fast, when compared to other virtualization technologies.

▲ docker run hello-world

▲ Displaying Docker Images

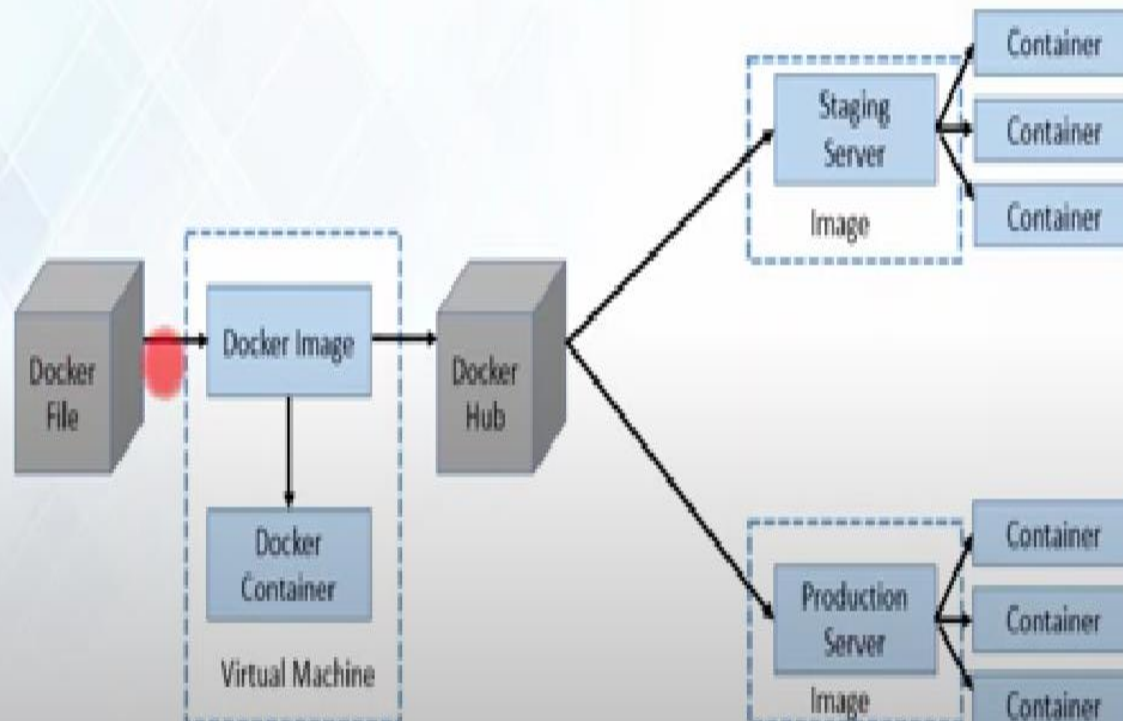
▲ docker images

- ▲ Containers are runnable instance of Docker images that can be run using the Docker run command.
- ▲ Listing of Containers
- ▲ `docker ps`
- ▲ This command is used to return the currently running containers.
- ▲ `docker ps -a`
- ▲ This command is used to list all of the containers on the system

- ▶ Docker Hub is a registry service on the cloud that allows you to download Docker images that are built by other communities. You can also upload your own Docker built images to Docker hub.



- Dockerfile builds a Docker image and that image contains all the project's code
- You can run that image to create as many Docker containers as you want
- Then this Image can be uploaded on Docker hub, from Docker hub any one can pull the image and build a container

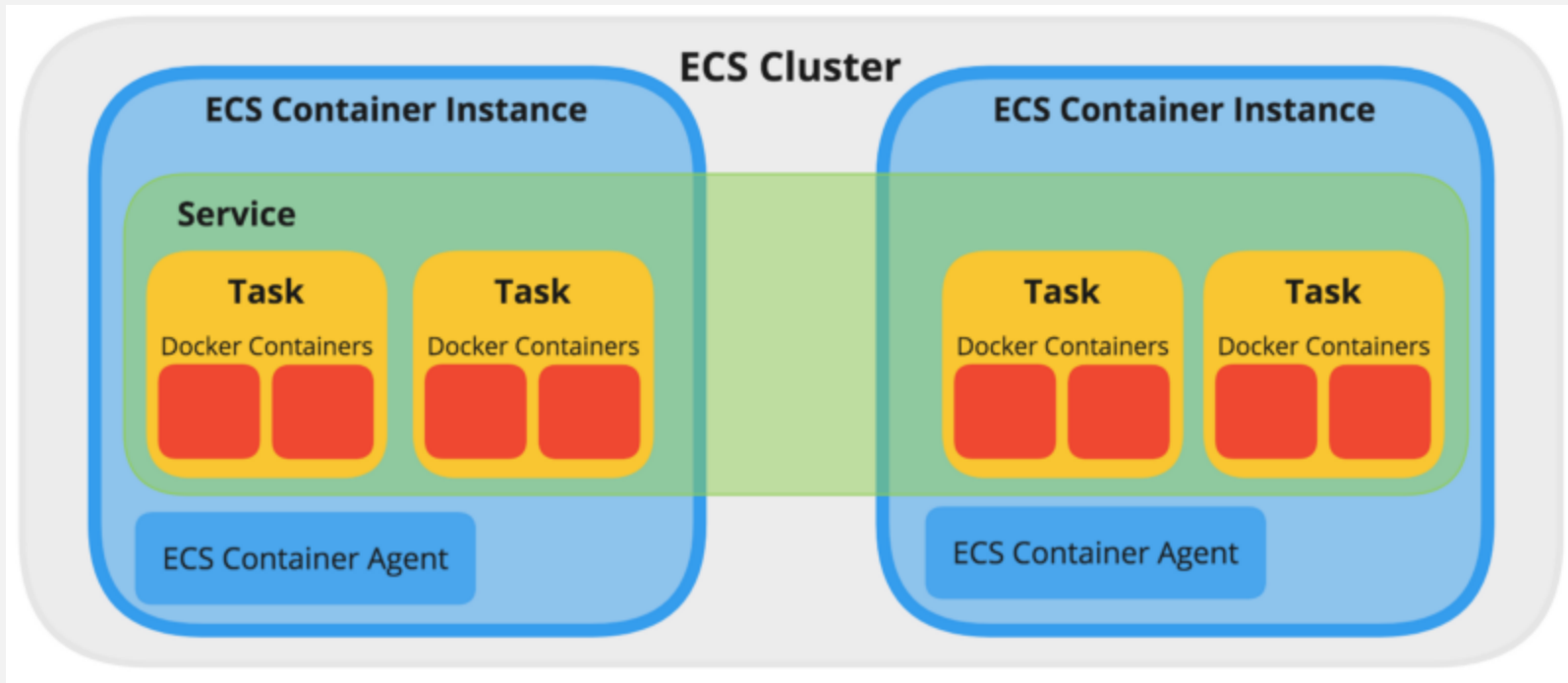


▲ Volumes

- ▲ The persisting data generated by docker and used by Docker containers are stored in Volumes. They are completely managed by docker through docker CLI or Docker API. Volumes work on both Windows and Linux containers.

- ▲ Docker takes care of the networking aspects so that the containers can communicate with other containers and also with the Docker Host. If you do an **ifconfig** on the Docker Host, you will see the Docker Ethernet adapter. This adapter is created when Docker is installed on the Docker Host.
- ▲ Listing All Docker Networks
- ▲ `docker network ls`

- ▲ Amazon Elastic Container Service (ECS) is, according to Amazon,
- ▲ ...a highly scalable, fast, container management service that makes it easy to run, stop, and manage Docker containers on a cluster.



An example ECS cluster, with one Service running four Tasks across two ECS Container Instances. A Cluster is a group of ECS Container Instances.

- ▲ **Clusters:** Clusters are the logical grouping of the tasks and services .
- ▲ **Task Definitions:** Task Definitions are text files where the user can describe one or more containers that form the applications.
- ▲ A task definition is like a blueprint for your application. Each time you launch a task in Amazon ECS, you specify a task definition. The service then knows which Docker image to use for containers, **how many containers to use in the task, and the resource allocation for each container.**



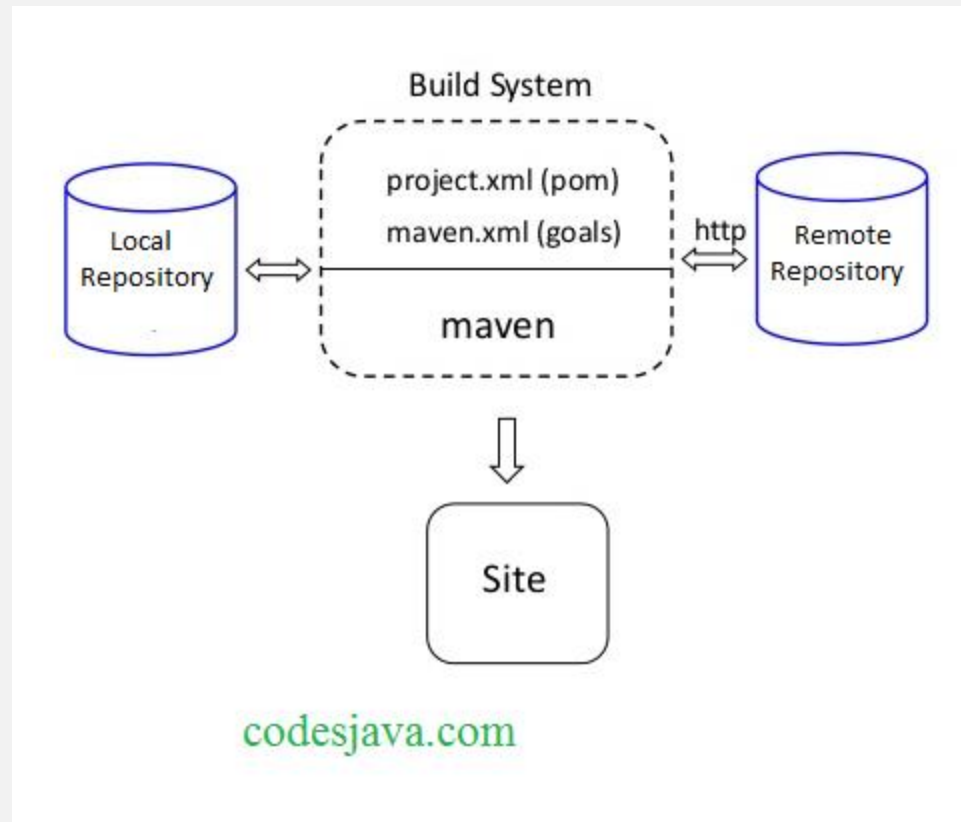
▲ Launch types of AWS ECS

- ▲ ECS can be launched in two following modes:
- ▲ **Fargate launch:** As discussed above, the *fargate launch type*, takes most of the responsibility from the users and takes in only the basic inputs like the CPU type, memory, and IAM policies from the user to run the application cluster.
- ▲ **EC2 Launch:** This is a more customizable launch type. Users are responsible for the number of instances in the cluster, scaling their cluster, and more. This allows you to be more in control of your clusters which may be required for security reasons.

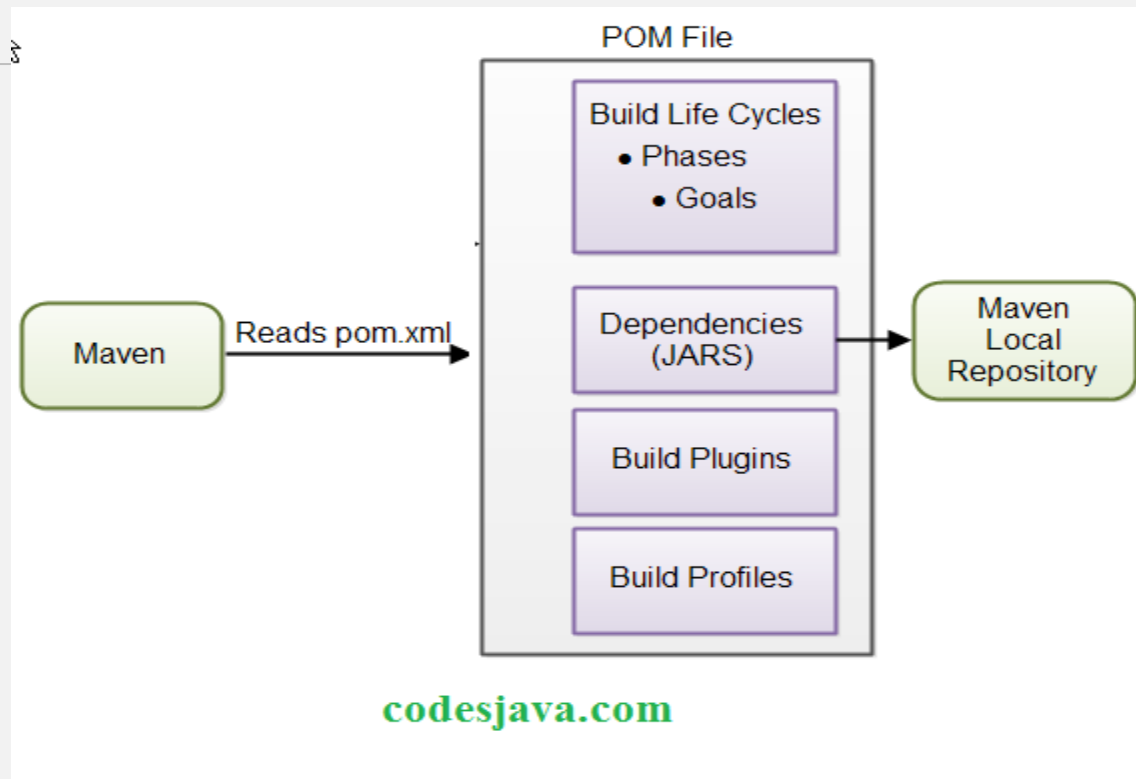
Maven



- ▲ Maven is a build automation tool used primarily for Java projects.
- ▲ Maven addresses two aspects of building software:
 - ▲ It describes how software is built.
 - ▲ it describes its dependencies.
 - ▲ Update the version



Maven overview diagram:



- Maven reads the pom.xml file.
- Downloads dependencies into local repository.
- Execute life cycles, build phases and goals.
- Execute plugins.

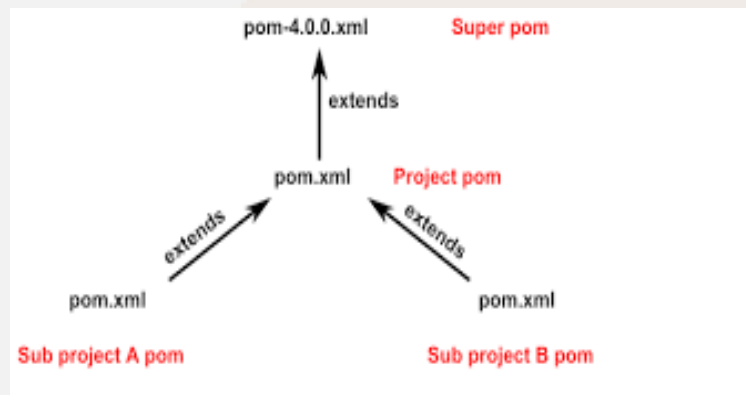
Overview about POM

- ▲ A Project Object Model or **POM** is the fundamental unit of work in Maven.
- ▲ It is an XML **file** that contains information about the project and configuration details used by Maven to build the project.
- ▲ Minimal POM requirement
 - ▲ project root
 - ▲ modelVersion - should be set to 4.0.0
 - ▲ groupId - the id of the project's group.
 - ▲ artifactId - the id of the artifact (project)
 - ▲ version - the version of the artifact under the specified group

```
1. <project>
2.   <modelVersion>4.0.0</modelVersion>
3.   <groupId>com.mycompany.app</groupId>
4.   <artifactId>my-app</artifactId>
5.   <version>1</version>
6. </project>
```

▲ Super POM

- ▲ The Super POM is Maven's default POM.
- ▲ All POMs inherit from a parent or default (despite explicitly defined or not).
- ▲ This base POM is known as the Super POM, and contains values inherited by default.
- ▲ Maven use the effective POM (configuration from super pom + project configuration) to execute relevant goal.



- ▲ Maven is based around the **central concept** of a build lifecycle. What this means is that the process for building and distributing a particular artifact (project) is clearly defined.
- ▲ Its necessary to learn a small set of commands to build any Maven project, and the POM will ensure they get the results they desired.
- ▲ There are three **built-in build lifecycles**: default, clean and site. The **default** lifecycle handles your **project deployment**, the **clean** lifecycle handles **project cleaning**, while the **site** lifecycle handles the creation of your project's **site documentation**.

- ▲ Each of these build lifecycles is defined by a different list of build phases, wherein a build phase represents a stage in the lifecycle.
- ▲ **validate** - validate the project is correct and all necessary information is available
- ▲ **compile** - compile the source code of the project
- ▲ **test** - test the compiled source code using a suitable unit testing framework. These tests should not require the code be packaged or deployed
- ▲ **package** - take the compiled code and package it in its distributable format, such as a JAR.
- ▲ **verify** - run any checks on results of integration tests to ensure quality criteria are met
- ▲ **install** - install the package into the local repository, for use as a dependency in other projects locally
- ▲ **deploy** - done in the build environment, copies the final package to the remote repository for sharing with other developers and projects.

- ▲ A Build profile is a set of configuration values, which can be used to set or override default values of Maven build.
- ▲ Using a build profile, you can customize build for different environments such as Production v/s Development environments.
- ▲ Profiles are specified in pom.xml file using its activeProfiles/profiles elements and are triggered in variety of ways.
- ▲ Profiles modify the POM at build time, and are used to give parameters different target environments (for example, the path of the database server in the development, testing, and production environments).

Types of Build Profile

Type	Where it is defined
Per Project	Defined in the project POM file, pom.xml
Per User	Defined in Maven settings xml file (%USER_HOME%/.m2/settings.xml)
Global	Defined in Maven global settings xml file (%M2_HOME%/conf/settings.xml)

▲ Maven is actually a plugin execution framework where every task is actually done by plugins. Maven Plugins are generally used to

- create jar file
- create war file
- compile code files
- unit testing of code
- create project documentation
- create project reports

- ▲ Java project can be compiled with the maven-compiler-plugin's compile-goal by running the following command.



```
mvn compiler:compile
```

Github

- GitHub is a Git repository hosting service, but it adds many of its own features. While Git is a command line tool, GitHub provides a Web-based graphical interface.
- It also provides access control and several collaboration features.



- Creating new repo
- Importing new repo
- New git **
- New org
- New project
- Fork

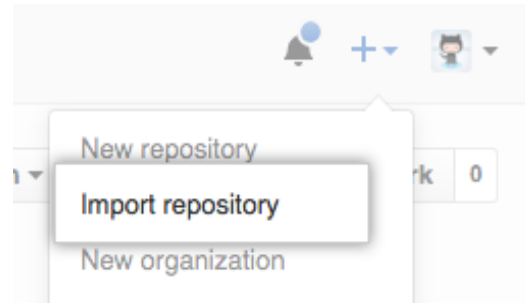
Creating new repo

- In revision control systems, a repository is a data structure which stores metadata for a set of files or directory structure.
- A central location in which data is stored and managed.



Importing new repo

- Just copying the repo from different repo to



New org

- Organization accounts allow your team to plan, build, review, and ship software — all while tracking bugs and discussing ideas.
- An **organization** is a collection of user accounts that owns repositories. **Organizations** have one or more owners, who have administrative privileges for the **organization**.

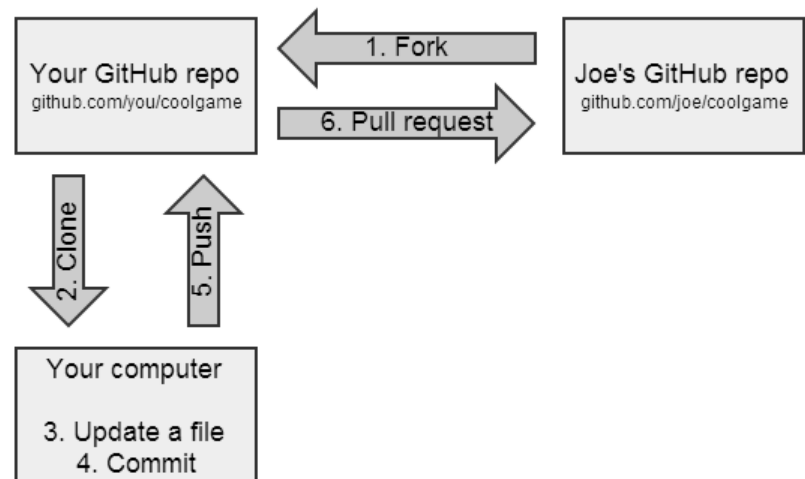
New project

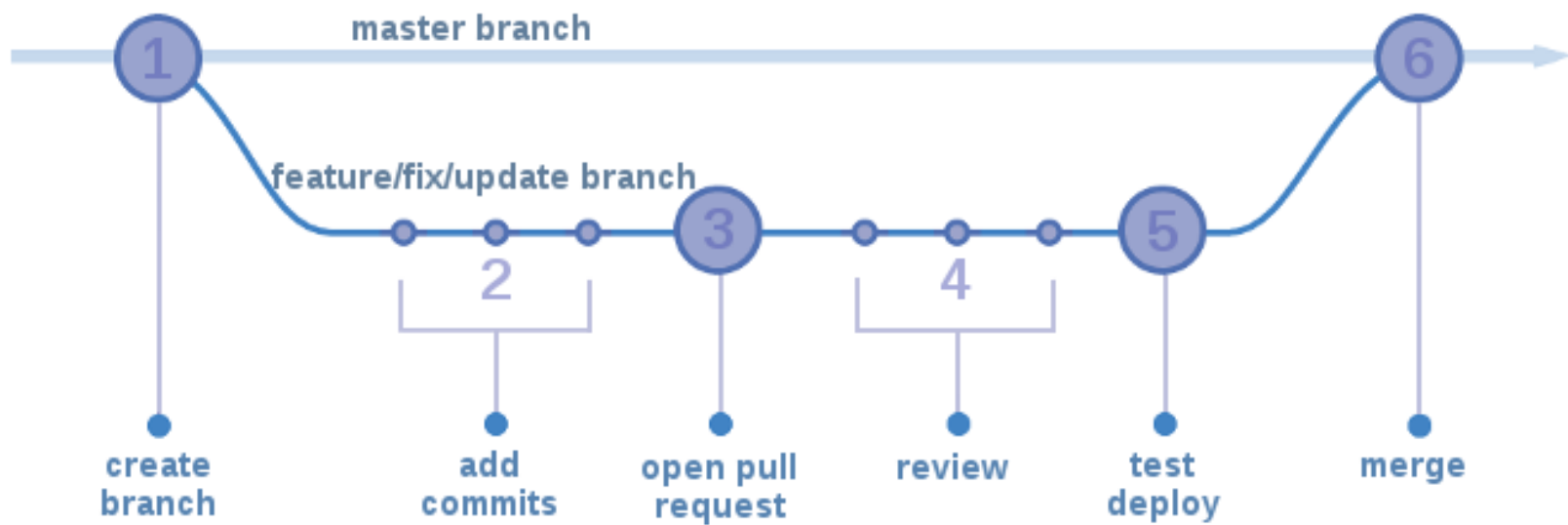
- **Project** boards on **GitHub** help you organize and prioritize your work. .You can create notes within columns to serve as task reminders, references to issues and pull requests from any repository on **GitHub**, or to add information related to the **project** board.

Template	Description
Basic kanban	Track your tasks with To do, In progress, and Done columns
Automated kanban	Cards automatically move between To do, In progress, and Done columns
Automated kanban with review	Cards automatically move between To do, In progress, and Done columns, with additional triggers for pull request review status
Bug triage	Triage and prioritize bugs with To do, High priority, Low priority, and Closed columns

Fork

- **Forking** Workflow **is** commonly used in public open-source projects. **Forking** **is** a **git** clone operation executed on a server copy of a projects repo.
- **Work is** done to complete the new feature and **git** commit **is** executed to save the changes. You then push the new feature branch to your remote forked repo.





GitHub news!!

[Starting January 7, 2019](#) - New GitHub: Announcing unlimited free private repos and unified Enterprise offering.

GitHub Free now includes unlimited private repositories. For the first time, developers can use GitHub for their private projects with up to three collaborators per repository for free.

- A **collaborator** is someone who has been granted write access to the main repository of a project.
- A contributor is a person who isn't explicitly a member of your organization or personal account, but wants to contribute to your project via pull requests and creating new issues.

GitHub Enterprise

Nagios®

- Nagios is an open source monitoring system for computer systems. It was designed to run on the Linux operating system and can monitor devices running Linux, Windows and Unix operating systems (OSes).
- Nagios software runs periodic checks on critical parameters of application, network and server resources.

Is Nagios free or not?

- Yes! Nagios XI is available free of charge for monitoring small environments.
- Nagios XI installations with a free license are limited to monitoring seven (7) hosts (nodes).
- There is no limitation on the number of services that can be monitored with a free license.

Nagios' benefits

- Scales well for simple, static setups.
- Easy to understand plugin architecture.
- Forks strive to maintain compatibility.
- Simple configuration language.
- Great ecosystem of plugins to extend functionality.

Why Continuous Monitoring on DevOps

- In the Manage phase of the DevOps life cycle, continuous monitoring of applications in production environments is typically implemented with application performance management (APM) solutions that intelligently monitor, analyze and manage cloud, on-premise and hybrid applications and IT infrastructure.
- These APM solutions enable you to monitor your users' experience and improve the stability of your application infrastructure.

- It helps identify the root cause of issues quickly to proactively prevent outages and keep users satisfied.
- With a DevOps approach, we are also seeing more customers broaden the scope of continuous monitoring into the staging, testing and even development environments.
- This is possible because development and test teams that are following a DevOps approach are striving to use production-like environments for testing as much as possible.

Nagios Core

- Nagios Core is the monitoring and alerting engine that serves as the primary application around which hundreds of Nagios projects are built.
- Nagios Core serves as the basic event scheduler, event processor, and alert manager for elements that are monitored.

Nagios Plugins

- **Plugins** are compiled executables or scripts (Perl scripts, shell scripts, etc.) that can be run from a command line to **check** the status of a host or service.
- **Nagios** uses the results from **plugins** to determine the current status of hosts and services on your network.

Nagios NRPE

- **Nagios** Remote Plugin Executor (**NRPE**) is a **Nagios** agent that allows remote system monitoring using scripts that are hosted on the remote systems.
- It allows for monitoring of resources such as disk usage, system load or the number of users currently logged in.

